

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

**«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ Н. Г. ЧЕРНЫШЕВСКОГО»**

Кафедра информатики и программирования

ГЕНЕРАЦИЯ ЗАГОЛОВКА ПО КРАТКОМУ СОДЕРЖАНИЮ ТЕКСТА

КУРСОВАЯ РАБОТА

студентки 3 курса 341 группы

направления 02.03.01 — Математическое обеспечение и администрирование
информационных систем

факультета КНиИТ

Загудалиной Вероники Павловны

Научный руководитель

ст. пр.

А. А. Казачкова

Заведующий кафедрой

доцент, к. ф.-м. н.

М. В. Огнева

Саратов 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 Теоретические основы моделей генерации	7
1.1 Сбор данных	7
1.2 Предобработка и анализ текста	7
1.3 Метрики оценки качества генерации текста	9
1.3.1 Метрика BLEU	9
1.3.2 Метрика Perplexity	10
1.3.3 Метрика ROUGE	11
1.3.4 Метрика METEOR	13
1.4 Эволюция подходов генерации	13
1.4.1 Шаблонные методы генерации заголовков	13
1.4.2 Языковые модели на основе n-грамм	14
1.4.3 Feedforward Neural Network Language Model (FNNLM)	16
1.4.4 Рекуррентные нейронные сети (RNN)	17
1.4.5 LSTM	20
1.4.6 Sequence-to-Sequence (Seq2Seq)	22
2 Работа с данными	25
2.1 Парсинг данных	25
2.1.1 Парсинг Проза.ру	26
2.1.2 Парсинг ЛитПричал	32
2.1.3 Парсинг Литрес	34
2.1.4 Парсинг Briefly	36
2.2 Предобработка и анализ данных	39
2.3 Аугментация данных	48
3 Написание и обучение моделей	55
3.1 Модель на основе n-gram	55
3.2 Feedforward Neural Network Language Model (FNNLM)	67

3.2.1	Подготовка обучающей выборки и формирование словаря	67
3.2.2	Архитектура нейросетевой модели FNNLM	69
3.2.3	Конфигурация и процедура обучения модели	70
3.2.4	Алгоритм генерации текста и стратегии инференса	72
3.2.5	Результаты и оценка качества генерации	74
3.3	Рекуррентная нейронная сеть (RNN)	76
3.3.1	Подготовка последовательностей и токенизация	76
3.3.2	Архитектура нейросетевой модели TitleRNN	78
3.3.3	Процесс обучения и механизм Teacher Forcing	80
3.3.4	Алгоритм авторегрессионной генерации и стратегия инференса	82
3.3.5	Результаты и оценка качества генерации	83
3.4	Реализация модели на базе архитектуры LSTM	85
3.4.1	Подготовка данных и структурирование последовательностей	85
3.4.2	Архитектура нейросетевой модели LSTMTitleGen	87
3.4.3	Процесс обучения	89
3.4.4	Стохастическая генерация	90
3.4.5	Результаты и оценка качества генерации	92
3.5	Seq2seq	94
3.5.1	Подготовка данных и токенизация	94
3.5.2	Компоненты нейросетевой модели	96
3.5.3	Класс Seq2Seq	98
3.5.4	Конфигурация и оптимизация процесса обучения	99
3.5.5	Алгоритм инференса	100
3.5.6	Результаты и сравнительный анализ	102
ЗАКЛЮЧЕНИЕ		104
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ		105
Приложение А Парсинг		109

Приложение Б	Работа с данными	122
Приложение В	Модель на основе n-gram.....	133
Приложение Г	Feedforward Neural Network Language Model.....	138
Приложение Д	Рекуррентные нейронные сети.....	146
Приложение Е	LSTM	152
Приложение Ж	Seq2seq.....	159

ВВЕДЕНИЕ

Современную эпоху характеризует стремительное развитие вычислительных технологий, глобальных сетей и цифровых коммуникаций, что ведёт к экспоненциальному росту числа пользователей Интернета и объёмов создаваемых данных. Согласно прогнозам, к 2025 году мировой объём цифровой информации достигнет 463 миллиардов гигабайт [1]. Столь масштабный рост требует новых подходов к обработке, анализу и визуализации данных, которые сегодня обеспечиваются, в первую очередь, технологиями искусственного интеллекта (ИИ), машинного обучения (ML) и обработки естественного языка (NLP). Эти инструменты автоматизируют работу с текстовой информацией, делая взаимодействие человека с цифровой средой более эффективным [2].

Особенно остро эта необходимость проявляется в сфере текстового контента — новостей, научных статей, художественных произведений и пользовательских публикаций, объёмы которых ежедневно увеличиваются. Для облегчения восприятия и повышения доступности такой информации требуются эффективные инструменты её обработки. Одним из ключевых среди них является автоматическая генерация заголовков, то есть создание краткого, выразительного текста, отражающего основную идею документа.

В новых условиях заголовки эволюционировали: из простого средства привлечения внимания они превратились в самостоятельный инструмент влияния на восприятие контента. Теперь они выполняют такие функции, как усиление вовлечённости, повышение кликабельности, поисковая оптимизация и адаптация под алгоритмы социальных сетей. Эта трансформация тесно связана с феноменом «заголовочного чтения», когда пользователи зачастую ограничиваются лишь заголовком, не обращаясь к полному тексту [3].

Целью курсовой работы является проектирование и разработка системы автоматической генерации заголовков для текстовых документов.

Задачи:

1. Изучить современные методы автоматической генерации заголовков и обзоры аналогичных исследований.
2. Рассмотреть библиотеки и инструменты для обработки естественного языка (NLP), включая парсинг текстов и подготовку данных.
3. Сбор и обработка текстовых данных:
 - а) Парсинг сайтов с литературными произведениями и публикациями.
 - б) Формирование датасета с текстами и соответствующими заголовками.
4. Разработка и обучение моделей генерации заголовков:
 - а) Реализация базовых моделей.
 - б) Реализация и обучение Seq2Seq-модели.
5. Оценка качества работы моделей.

1 Теоретические основы моделей генерации

1.1 Сбор данных

Парсинг веб-страниц – это процесс извлечения данных с веб-страниц, обычно с использованием специальных программ или скриптов.

Веб-страницы обычно написаны на языке HTML, который определяет структуру и содержание страницы. При парсинге веб-страниц данные извлекаются из HTML-кода, обрабатываются и преобразуются в удобный для использования формат, такой как текст, таблицы, JSON или XML.

Парсинг веб-страниц может включать в себя различные этапы, такие как загрузка HTML-кода страницы, поиск нужной информации среди различных тегов и атрибутов, обработка данных и сохранение их в нужном формате. Для парсинга веб-страниц часто используются специализированные библиотеки и инструменты, такие как BeautifulSoup, lxml, Scrapy, Selenium и другие.

Извлеченные данные могут использоваться для различных целей, таких как анализ, отслеживание изменений, автоматизация задач, создание персонализированных приложений или уведомлений.

Однако при использовании парсинга веб-страниц важно учитывать правила использования сайта и законы о защите данных, чтобы не нарушать авторские права или правила конфиденциальности [4].

1.2 Предобработка и анализ текста

Обработка естественного языка начинается с работы с сырыми данными, которые зачастую являются неструктурированными.

Эти данные часто содержат ошибки, неоднозначности или избыточность, что делает их сложными для анализа. Поэтому важным этапом является предварительная обработка текста, которая включает удаление лишних символов, преобразование регистра, устранение стоп-слов и другие шаги для приведения данных к стандартному виду.

Нормализация текста означает его преобразование в более удобную, стан-

дартную форму. Например, большая часть того, что мы собираемся делать с языком, основывается на выделении или токенизации слов из текста - задача токенизации [5].

Токенизация – это процесс разбиения фразы, предложения, абзаца или всего текстового документа на более мелкие единицы, например, отдельные слова или термины. Каждое из этих меньших подразделений называется токенами.

Перед обработкой естественного языка нужно определить слова, которые составляют строку символов. В связи с этим токенизация является основным шагом для работы с NLP. Важность токенизации обусловлена тем, что значение текста можно легко интерпретировать, анализируя слова, присутствующие в тексте.

Токенизированную форму можно использовать для подсчета базовых статистик, например, количества слов в тексте или частоты слова, как необходимый шаг перед более сложными шагами обработки текста [6].

Другой частью нормализации текста является лемматизация - задача определения того, что два слова имеют один и тот же корень, несмотря на их поверхностные различия. Например, слова пел, спетый и петь являются формами глагола петь. Слово петь является общей леммой этих слов, и лемматизатор переводит их все в «петь». Лемматизация необходима для обработки морфологически сложных языков, например, для стемминга русского языка. Под стеммингом понимается более простая версия лемматизации, в которой мы в основном просто удаляем суффиксы с конца слова.

В текстах часто встречаются слова, не влияющие на смысл и лишь создающие помехи при анализе эмоциональной окраски. Эти слова, известные как стоп-слова, включают в себя:

- Имена
- Числовые значения
- Вводные конструкции

- Служебные части речи (предлоги, частицы, союзы)
- Местоимения
- Междометия
- Ссылки
- Пунктуационные знаки

Для их фильтрации применяются следующие подходы:

Методы обработки

1. Регулярные выражения — эффективны для удаления пунктуации, цифр и других легко идентифицируемых элементов.
2. Предопределённые списки — популярные стоп-слова (предлоги, союзы, междометия и т.д.) заранее сохраняются в файлы, а затем загружаются в множество Python.

Алгоритм удаления

1. Текст разбивается на токены методом `tokenize`.
2. Каждый токен проверяется на вхождение в множество стоп-слов.
3. Совпадающие токены исключаются из дальнейшего анализа.

Этот подход минимизирует шум в данных и повышает точность обработки текста [7].

1.3 Метрики оценки качества генерации текста

1.3.1 Метрика BLEU

Метрика BLEU (Bilingual Evaluation Understudy) была предложена в работе Папинени. В качестве метода автоматической оценки качества. Основная идея метрики заключается в том, чтобы измерять степень совпадения n -грамм (последовательностей из n слов) между кандидатом и одним или несколькими эталонами, выполненными человеком [8].

Ключевые компоненты алгоритма BLEU включают:

1. Модифицированная n -граммная точность: для каждого размера n -граммы (обычно от 1 до 4) вычисляется точность. В отличие от обычной точ-

ности, модифицированный вариант предотвращает искусственное завышение оценки за счёт многократного учёта одинаковых слов в кандидате. Количество совпадений для каждой n -граммы ограничивается максимальным количеством её появлений в любом одном эталонном переводе.

2. Штраф за краткость: чтобы наказать слишком короткие кандидаты, которые могут иметь высокую n -граммную точность, но не передавать полный смысл, вводится штраф за краткость (Brevity Penalty, BP).
3. Агрегация: итоговая оценка BLEU представляет собой взвешенное геометрическое среднее модифицированных точностей для разных n , умноженное на штраф за краткость.

Итоговая формула для корпуса текстов имеет вид:

$$BLEU = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right) \quad (1)$$

где:

- p_n — модифицированная точность для n -грамм,
- w_n — вес, обычно $w_n = 1/N$ для равномерного взвешивания,
- N — максимальная длина n -граммы (по умолчанию 4),
- BP — штраф за краткость.

1.3.2 Метрика Perplexity

Для оценки качества языковых моделей часто используется показатель Perplexity (PPL) — функция вероятности, отражающая способность модели предсказывать слова в тестовом наборе данных. Чем лучше модель предсказывает последовательность слов, тем выше вероятность, которую она присваивает каждому слову, и тем ниже значение perplexity.

Идеальная языковая модель при этом должна для каждого слова в корпусе назначать вероятность 1 (для правильного слова) и 0 — для всех остальных. Однако на практике мы используем не абсолютные вероятности, а их нормированную форму, поскольку общая вероятность тестового набора убывает с

увеличением его длины.

Поэтому Perplexity служит нормированной метрикой, которая позволяет сравнивать модели, обученные на текстах различной длины. Она определяется как обратная вероятность тестового набора, нормированная по количеству слов (или токенов).

Для тестового корпуса $W = w_1, w_2, \dots, w_N$ perplexity вычисляется по формуле:

$$\text{Perplexity}(W) = P(w_1, w_2, \dots, w_N)^{-\frac{1}{N}} = \sqrt[N]{\frac{1}{P(w_1, w_2, \dots, w_N)}} \quad (2)$$

Используя правило цепочки вероятностей, можно разложить вероятность последовательности W на условные вероятности отдельных слов:

$$\text{Perplexity}(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_1, \dots, w_{i-1})}} \quad (3)$$

Из формулы (3) следует, что чем выше вероятность последовательности слов, предсказанная моделью, тем ниже perplexity. Следовательно, минимизация perplexity эквивалентна максимизации вероятности тестового корпуса, что делает метрику удобным показателем качества языковой модели.

Использование обратной вероятности связано с исходным определением perplexity, происходящим из коэффициента перекрёстной энтропии в теории информации. То есть perplexity имеет обратную зависимость от вероятности предсказанной последовательности [9].

1.3.3 Метрика ROUGE

Развитием метрик семейства BLEU, ориентированных преимущественно на показатель точности, стало появление группы метрик ROUGE, которые, помимо точного совпадения элементов текста, уделяют особое внимание измерению полноты (recall). Показатель полноты отражает долю n-грамм, совпадающих

между оригинальным и сгенерированным текстом, относительно общего числа n -грамм в исходном тексте.

Формально метрика ROUGE-N представляет собой recall на уровне n -грамм между сгенерированным (candidate) резюме и набором эталонных (reference) резюме. Значение ROUGE-N вычисляется следующим образом:

$$\text{ROUGE-N} = \frac{\sum_{S \in \{\text{Reference Summaries}\}} \sum_{\text{gram}_n \in S} \text{Count}_{\text{match}}(\text{gram}_n)}{\sum_{S \in \{\text{Reference Summaries}\}} \sum_{\text{gram}_n \in S} \text{Count}(\text{gram}_n)} \quad (4)$$

где:

- n — длина n -граммы;
- gram_n — конкретная n -грамма длины n ;
- $\text{Count}_{\text{match}}(\text{gram}_n)$ — максимальное количество вхождений данной n -граммы, встречающихся одновременно в сгенерированном тексте и в эталонных резюме;
- $\text{Count}(\text{gram}_n)$ — общее количество появлений этой n -граммы в эталонных текстах [10].

Помимо классической метрики ROUGE-N, существует целый ряд модификаций, позволяющих оценивать различные аспекты текстового сходства. Наиболее распространённые варианты включают:

- ROUGE-L, основанный на длине наибольшей общей подпоследовательности слов (LCS). Данный вариант учитывает порядок слов и хорошо подходит для оценки структурного сходства предложений.
- ROUGE-S, использующий совпадение так называемых «скип-биграмм» — пар слов, которые могут быть разделены произвольным количеством других слов. Такой подход позволяет обнаруживать частично совпадающие структуры, даже если порядок слов в тексте был изменён [11].

1.3.4 Метрика METEOR

Ещё одним инструментом для оценки качества автоматически создаваемого текста является метрика METEOR (Metric for Evaluation of Translation with Explicit ORdering). По своей сути она близка к ROUGE, однако отличается тем, что применяет заранее заданные правила выравнивания текста и учитывает совпадения не только на уровне точных слов, но и их синонимов. Такой подход позволяет более корректно оценивать случаи, когда система использует разные слова с одинаковым значением, а также принимать во внимание порядок слов в предложении, что делает оценку более точной.

В отличие от BLEU, метрика METEOR опирается на несколько типов соответствия — точное совпадение слов, совпадение по корням и совпадение по синонимам, что делает её более чувствительной к качеству генерации на уровне фраз и предложений. METEOR была создана как ответ на ограничения BLEU и стремится обеспечить более высокую согласованность автоматической оценки с человеческими суждениями.

По результатам экспериментов, использование METEOR на уровне словосочетаний показывает корреляцию с оценками экспертов 0,964, тогда как BLEU на тех же данных достигает лишь 0,817. На уровне отдельных предложений максимальная корреляция составляет 0,403 [11, 12].

1.4 Эволюция подходов генерации

1.4.1 Шаблонные методы генерации заголовков

Одним из первых подходов к автоматической генерации текста и заголовков были шаблонные (template-based) системы, широко применявшиеся в 1960–1990-х годах. Принцип их работы заключался в том, что задание структуры текста и заголовка осуществлялось человеком заранее, в виде набора правил или шаблонов. В дальнейшем система лишь подставляла в эти шаблоны фактические данные.

Основные особенности подхода:

1. Жёсткая структура: заголовок строился по заранее заданной грамматической схеме.
2. Подстановка данных: в качестве переменных использовались ключевые сущности текста — названия команд, счёт, место, время, имена участников.
3. Упрощение текста: для получения краткого заголовка из длинного текста часто применялись эвристики:
 - удаление предлогов, местоимений и вводных слов,
 - сохранение существительных и глаголов,
 - сокращение до ключевых событий.

Примеры ранних систем:

1. ELIZA (1966) — имитация психотерапевта, основанная на шаблонах и правилах подстановки. Хотя она не генерировала заголовки, её принцип работы показывает ограниченность и прямолинейность rule-based подхода [13].
2. PARRY (1972) — симулятор пациента с паранойей, также использовавший заранее прописанные сценарии [14].
3. SHRDLU (1972) — система Терри Винограда, демонстрировавшая взаимодействие на естественном языке в ограниченном «мире блоков»; применялась грамматика и правила синтаксиса [15].

Шаблонные методы активно использовались в первых новостных агрегаторах и спортивных отчётах. Такие заголовки генерировались автоматически на основе статистики матчей и заранее определённого набора шаблонов.

1.4.2 Языковые модели на основе n -грамм

Классические статистические языковые модели, основанные на n -граммах, опираются на подсчёт частот последовательностей из n токенов в корпусе. N -граммные языковые модели используют предположение Маркова, которое утверждает, что в контексте языкового моделирования вероятность следующе-

го слова в последовательности зависит только от предыдущего(их) слова(слов). В простейшей форме вероятность появления токена w_i при заданном контексте $w_{i-(n-1):i-1}$ оценивается как

$$P_n(w_i \mid w_{i-(n-1):i-1}) = \frac{\text{cnt}(w_{i-(n-1):i-1}w_i \mid \mathcal{D})}{\text{cnt}(w_{i-(n-1):i-1} \mid \mathcal{D})}, \quad (1)$$

где $\text{cnt}(\mathbf{w} \mid \mathcal{D})$ — количество появлений n -граммы $\mathbf{w}$ в обучающем корпусе $\mathcal{D}$, а n — заранее заданный гиперпараметр. Для $n = 1$ контекст $w_{i-(n-1):i-1}$ определяется как пустая строка ε , количество которой равно $|\mathcal{D}|$.

Однако на практике такая наивная модель сталкивается с проблемой разреженности данных: числитель в формуле может быть равен нулю, что приводит к бесконечной перплексии. Одним из способов решения этой проблемы является стратегия backoff [16]: если числитель равен нулю, уменьшают n на единицу и повторяют процесс до тех пор, пока числитель не станет положительным. Следует отметить, что backoff не формирует корректное распределение $P_n(* \mid w_{i-(n-1):i-1})$, поскольку эффективное n зависит от конкретного w_i , поэтому требуется дополнительное снижение вероятностей для нормализации распределения, например, метод Katz [17].

Исторически n -граммовые модели реализовывались через построение таблицы подсчёта n -грамм по обучающему корпусу. Каждая уникальная n -грамма сохраняется вместе с числом её появлений. Размер таких таблиц растёт почти экспоненциально с увеличением n . Например, оценка показывает, что таблица для 5-грамм на корпусе из 1,4 триллиона токенов займёт примерно 28 ТиБ дискового пространства. Поэтому традиционные n -граммовые модели ограничены малыми значениями n , чаще всего $n = 5$ [18, 19]. Малое значение n приводит к потере более богатого контекста, что снижает предсказательную способность таких моделей [20].

1.4.3 Feedforward Neural Network Language Model (FNNLM)

Одним из первых и наиболее влиятельных подходов к построению нейронных языковых моделей является нейронная вероятностная языковая модель (Neural Probabilistic Language Model), предложенная Йошуа Бенджио и соавторами в 2003 году. Эта модель является прямой (feedforward) нейронной сетью, обучаемой предсказывать вероятность следующего слова по фиксированному контексту из $n-1$ предыдущих слов.

Основная проблема традиционных моделей типа n -грамм заключается в «проклятии размерности»: число возможных последовательностей слов растёт экспоненциально, поэтому большинство контекстов в тестовых данных отсутствуют в обучающей выборке. В результате требуется агрессивное сглаживание или сокращение контекста, что ухудшает качество генерации. Йошуа Бенджио указывает, что n -граммы фактически «склеивают» короткие фрагменты текста и не умеют учитывать сходство слов или более дальние зависимости между ними.

Для решения этих ограничений Бенджио и соавторы предложили два ключевых механизма:

1. Обучение распределённых представлений слов: Каждому слову сопоставляется обучаемый вектор признаков — embedding. Тем самым слова с похожими синтаксическими и семантическими ролями оказываются близко друг к другу в пространстве признаков.
2. Моделирование вероятности следующего слова с помощью feedforward-сети Вероятность следующего слова вычисляется как гладкая функция от векторных представлений слов контекста. Поскольку функция непрерывна, небольшое изменение входа ведёт к небольшому изменению вероятности, что обеспечивает генерализацию на новые сочетания слов.

Feedforward-LM состоит из двух основных частей:

1. Слой представлений слов

Матрица C размера $|V| \times m$, где каждая строка — embedding слова.

Для входного окна $(w_{t-n+1}, \dots, w_{t-1})$ векторы конкатенируются: $x = (C(w_{t-1}), \dots$

2. Нейронная сеть предсказания

Сеть с одним скрытым слоем и функцией активации $\tanh$: $y = b + Wx + U \tanh(d + Hx)$, где y_i — логиты вероятности для слова i .

Выход нормализуется через softmax:

$$P(w_t = i | w_{t-1}, \dots) = \frac{e^{y_i}}{\sum_j e^{y_j}}.$$

Таким образом, модель одновременно обучает:

- параметры сети,
- распределённые векторные представления слов.

Работа показывает, что использование FFNN-LM позволяет:

1. эффективно бороться с проклятием размерности,
2. учитывать дальние зависимости (до 5–6 слов в окне),
3. переносить знания за счёт сходства слововых embedding’ов (пример: предложение с «dog» имеет высокую вероятность, даже если обучалась модель на варианте с «cat»).

На реальных корпусах (Brown, AP News) FFNN-модели значительно снизили перплексию по сравнению с n-граммами — до 20–30%, что сделало подход архитектурно революционным [21].

Несмотря на успех, модель FNNLM остается ограниченной фиксированным размером входного окна, что не позволяет учитывать глобальные связи в длинных предложениях.

1.4.4 Рекуррентные нейронные сети (RNN)

Процесс человеческого мышления характеризуется непрерывностью: понимание каждого нового слова базируется на контексте предыдущих, что обеспечивает устойчивость мыслей. Традиционные нейронные сети прямого распространения лишены этой особенности, что является их существенным недостатком.

ком при работе с последовательными данными. Рекуррентные нейронные сети (RNN) решают данную проблему за счет наличия внутренних циклов, позволяющих информации сохраняться [22].

Рекуррентная сеть — это любая сеть, содержащая циклы в своих связях, что означает зависимость значения некоторого узла от его собственных более ранних выходных данных. Эти архитектуры служат фундаментом для более сложных подходов, таких как сети с долгой краткосрочной памятью (LSTM).

В отличие от традиционного подхода на основе скользящего окна, последовательности в RNN обрабатываются по одному элементу за раз. Для вычисления выходного значения y_t для входа x_t необходимо значение активации скрытого слоя h_t .

Ключевым отличием RNN является рекуррентная связь, которая дополняет входные данные для вычислений на скрытом слое значением этого же слоя из предыдущего момента времени. Скрытый слой h_{t-1} обеспечивает форму памяти или контекста, который кодирует результаты предыдущей обработки и информирует решения на более поздних этапах

Математически процесс вычисления скрытого состояния h_t и выходного вектора y_t описывается следующими уравнениями:

$$h_t = g(Uh_{t-1} + Wx_t)$$

$$y_t = f(Vh_t)$$

Где:

1. $W \in \mathbb{R}^{d_h \times d_{in}}$ — матрица весов входного вектора;
2. $U \in \mathbb{R}^{d_h \times d_h}$ — матрица весов рекуррентных соединений;
3. $V \in \mathbb{R}^{d_{out} \times d_h}$ — матрица весов выходного слоя;
4. g и f — функции активации.

Для задач классификации или языкового моделирования выход y_t обычно вычисляется через функцию softmax для получения распределения вероятно-

стей по классам:

$$y_t = \text{softmax}(Vh_t)$$

Рекуррентную сеть можно представить как несколько копий одной и той же сети, каждая из которых передает сообщение преемнику. Этот процесс называется «развертыванием» сети во времени. При развертывании становится очевидно, что слои юнитов копируются для каждого временного шага и имеют разные значения, однако соответствующие матрицы весов (W, U, V) остаются общими для всех моментов времени.

Подобная цепочечная природа делает RNN естественной архитектурой для работы со списками и последовательностями. Это подтверждается успехами в таких областях, как распознавание речи, машинный перевод и аннотирование изображений [9].

Теоретически скрытый слой способен хранить информацию, охватывающую контекст до самого начала последовательности. Однако на практике обучение стандартных RNN установлению связей между данными с большим временным разрывом оказывается крайне сложным.

Например, в коротком контексте («облака плывут по...») предсказание слова «небу» тривиально. Но в сложных сценариях, где релевантная информация отделена от места использования длинным фрагментом текста, RNN теряют способность к обучению. Фундаментальные причины этих трудностей были исследованы в работах Хохрайтера (1991) и Бенджио (1994).

Неспособность стандартных рекуррентных сетей переносить критически важную информацию на большие расстояния обусловлена двумя основными причинами:

1. **Функциональная перегрузка:** Скрытые слои и определяющие их веса вынуждены одновременно решать две задачи: предоставлять информацию для текущего решения и обновлять контекст для будущих шагов.
2. **Проблема затухающих градиентов:** В процессе обратного распростра-

нения ошибки во времени (BPTT) скрытые слои подвергаются многократному умножению, что при длинных последовательностях часто приводит к стремлению градиентов к нулю.

Для решения проблемы долговременных зависимостей используются архитектуры LSTM, которые будут рассмотрены далее [22].

1.4.5 LSTM

Сети долгой краткосрочной памяти (Long Short-Term Memory, LSTM) решают проблему управления контекстом, разделяя её на две подзадачи: удаление ненужной информации и добавление данных, необходимых для будущих решений.

Основное отличие LSTM заключается в их способности обучаться долговременным зависимостям «по умолчанию», что является их базовым поведением, а не результатом сложной настройки. В то время как стандартные RNN состоят из цепочки повторяющихся модулей с простой структурой (например, один слой с функцией $\tanh$), модуль LSTM содержит четыре взаимодействующих слоя, организованных особым образом.

Ключевым элементом архитектуры является состояние ячейки — горизонтальная линия, проходящая через верхнюю часть диаграммы модуля. Состояние ячейки можно сравнить с конвейерной лентой, которая проходит через всю цепочку с минимальными линейными взаимодействиями, что позволяет информации беспрепятственно протекать по ней, сохраняясь на протяжении длительных интервалов.

LSTM управляет информацией в состоянии ячейки с помощью структур, называемых гейтами. Они строятся по общей схеме: слой прямого распространения, за которым следует сигмоидальная функция активации и поэлементное умножение. Сигмоида выводит значения, близкие к 0 или 1, действуя как бинарная маска: значения, соответствующие единицам, проходят почти без изменений, а соответствующие нулям — стираются.

Процесс обновления состояния в блоке LSTM включает следующие этапы:

1. **Гейт забывания:** На основе предыдущего скрытого состояния h_{t-1} и текущего входа x_t принимается решение, какую информацию следует удалить из состояния ячейки. Например, в языковой модели при появлении нового подлежащего сеть может «забыть» грамматический род предыдущего субъекта.

$$f_t = \sigma(U_f h_{t-1} + W_f x_t)$$

$$k_t = c_{t-1} \odot f_t$$

2. **Гейт добавления:** Состоит из двух частей: сигмоидальный «входной гейт» определяет, какие значения будут обновлены, а слой $\tanh$ создает вектор новых кандидатов $\tilde{C}_t$.

$$g_t = \tanh(U_g h_{t-1} + W_g x_t)$$

$$i_t = \sigma(U_i h_{t-1} + W_i x_t)$$

$$j_t = g_t \odot i_t$$

3. **Обновление вектора контекста:**

$$c_t = j_t + k_t$$

4. **Выходной гейт:** Формирует финальное скрытое состояние h_t на основе фильтрации состояния ячейки. После применения $\tanh$ к состоянию ячейки оно умножается на результат выходной сигмоиды.

$$o_t = \sigma(U_o h_{t-1} + W_o x_t)$$

$$h_t = o_t \odot \tanh(c_t)$$

Скрытое состояние h_t является выходным значением LSTM на каждом временном шаге и может использоваться как вход для последующих слоев или для формирования финального результата сети [9].

Существует множество модификаций базовой архитектуры LSTM. Среди них выделяются «peerhole connections» (связи-глазки), позволяющие гейтам учитывать текущее состояние ячейки, и архитектуры со связанными гейтами забывания и добавления.

Одной из наиболее популярных упрощенных версий является Gated Recurrent Unit (GRU), предложенный Cho et al. (2014). В GRU гейты забывания и входа объединены в единый «гейт обновления», а состояния ячейки и скрытого слоя совмещены, что делает модель менее требовательной к вычислительным ресурсам.

Несмотря на эффективность LSTM, современные исследования указывают на следующий важный этап в развитии рекуррентных архитектур — механизмы внимания (Attention). Внимание позволяет сети на каждом шаге выбирать наиболее релевантные фрагменты из всей совокупности доступной информации, что значительно расширяет возможности обработки длинных и сложных последовательностей [22].

Основываясь на предоставленных вами материалах, предлагаю следующий текст для теоретической части вашей курсовой работы. Он логически продолжает раздел о нейросетевых архитектурах и описывает модель Seq2Seq строго по указанной статье.

1.4.6 Sequence-to-Sequence (Seq2Seq)

Глубокие нейронные сети обладают высокой вычислительной мощностью, однако их применение ограничено задачами, где входные и целевые данные могут быть представлены векторами фиксированной размерности. Это является существенным ограничением, поскольку многие важные задачи, такие как машинный перевод или вопросно-ответные системы, лучше всего выражаются в

виде последовательностей, длины которых заранее не известны. Стандартные рекуррентные нейронные сети (RNN) способны преобразовывать последовательности, если выравнивание между входами и выходами известно заранее, но их сложно применить к задачам, где длины входных и выходных последовательностей различаются и имеют сложные немонотонные связи. Для решения общих задач преобразования последовательности в последовательность может быть использована архитектура на базе сетей долгой краткосрочной памяти (LSTM).

Основная идея данного подхода заключается в использовании одной сети LSTM для чтения входной последовательности с целью получения векторного представления большой фиксированной размерности. Затем используется вторая глубокая сеть LSTM для извлечения целевой последовательности из этого вектора. Вторая LSTM по сути является языковой моделью на базе рекуррентной нейронной сети, работа которой обусловлена входной последовательностью. Общая схема данного процесса представлена на рисунке 1.

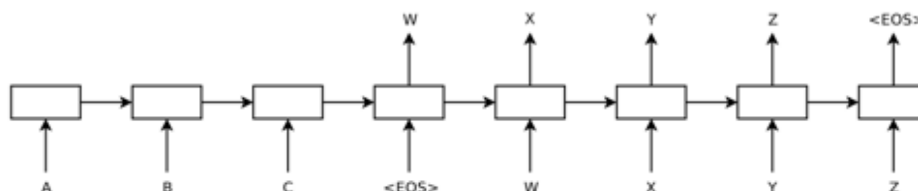


Рисунок 1 – Архитектура Sequence-to-Sequence

С математической точки зрения, цель модели — оценить условную вероятность $p(y_1, \dots, y_{T'} | x_1, \dots, x_T)$, где $(x_1, \dots, x_T)$ — входная последовательность, а $(y_1, \dots, y_{T'})$ — соответствующая ей выходная последовательность, длина которой T' может отличаться от длины входа T . Модель вычисляет эту вероятность в два этапа:

1. Сначала получается фиксированное векторное представление v входной последовательности, которое является последним скрытым состоянием первой LSTM.

2. Затем вычисляется вероятность выходной последовательности с использованием стандартной формулировки языковой модели LSTM, начальное скрытое состояние которой устанавливается равным вектору v :

$$p(y_1, \dots, y_{T'} | x_1, \dots, x_T) = \prod_{t=1}^{T'} p(y_t | v, y_1, \dots, y_{t-1}).$$

Важным условием является то, что каждое предложение должно заканчиваться специальным символом конца предложения «» (end-of-sentence). Это позволяет модели определять распределение вероятностей для последовательностей всех возможных длин. Практическая реализация данной архитектуры имеет три важные особенности:

1. В модели используются две разные LSTM-сети: одна для входной последовательности («кодировщик»), другая для выходной («декодировщик»). Это увеличивает количество параметров модели при незначительных вычислительных затратах и делает естественным одновременное обучение на нескольких языковых парах.
2. Глубокие сети LSTM значительно превосходят поверхностные, поэтому на практике применяется архитектура с четырьмя слоями.
3. Обращение порядка слов во входном предложении (при сохранении прямого порядка в целевом предложении) существенно улучшает качество работы модели. Это простое преобразование данных вводит множество краткосрочных зависимостей: первые слова исходного предложения оказываются очень близко к первым словам целевого предложения.

Данная архитектура также обладает полезным свойством: она учится преобразовывать входные предложения переменной длины в фиксированное векторное представление, в котором предложения со схожим смыслом находятся близко друг к другу, а с различным смыслом — далеко. Представления чувствительны к порядку слов, но при этом остаются относительно инвариантными к замене активного залога на пассивный.

2 Работа с данными

2.1 Парсинг данных

Веб-скрапинг (web-scraping) – это автоматизированное извлечение информации с веб-страниц. Например, сбор контактов из онлайн-каталога можно отнести к скрапингу. Поскольку ручной сбор больших массивов данных неэффективен, для этой задачи используют специальные программы – веб-скраперы.

С правовой точки зрения веб-скрапинг, как правило, не считается нарушением закона, так как он работает с общедоступными данными.

С ростом цифровизации значение веб-скрапинга увеличилось. Согласно исследованию компании, специализирующейся на информационной безопасности, более 50% (52%) интернет-трафика (без учета аудио- и видеопотоков) генерируется ботами – автоматизированными системами [23].

Для построения и обучения моделей машинного обучения требуется корпус данных, соответствующий предметной области исследования. Так как готовые наборы данных не всегда удовлетворяют требованиям по объёму, актуальности и тематике, было принято решение осуществить автоматизированный сбор текстовой информации из открытых источников. Основная цель парсинга — формирование репрезентативного набора текстов, который может быть использован для последующей предобработки и обучения моделей.

В качестве источников данных были выбраны:

1. Сайты с произведениями:
 - Проза.ру
 - ЛитПричал
2. Литрес — сайт по продаже книг, в котором для каждой книги писалась аннотация.
3. Брифли — сайт с краткими пересказами произведений.

Критериями выбора послужили: доступность данных, достаточный объём информации, а также структурированность представления материалов. Для

разработки парсеров были использованы следующие инструменты и библиотеки Python:

- requests — для получения HTML-кода страниц;
- BeautifulSoup — для анализа и извлечения информации из HTML;
- fake_useragent — для имитации различных User-Agent при отправке запросов и снижения риска блокировки;
- json — для сохранения собранных данных в структурированном формате;
- time, datetime — для управления задержками между запросами и работы с датами публикаций;
- tqdm — для визуализации прогресса выполнения парсинга.

Для каждого сайта был написан отдельный парсер. Перед написанием была изучена структура HTML документа каждого из сайтов для корректного сбора нужной информации. Рассмотрим подробно каждый.

2.1.1 Парсинг Проза.ру

Основной целью парсинга являлось получение текстов вместе с метаданными для последующей предобработки и обучения моделей машинного обучения.

Парсер реализован как класс ProzaRuParser со следующими основными методами:

Метод `__init__`: при создании объекта класса задаются базовый URL, заголовки запроса, время для ожидания ответа от сайта, задержка для запросов, чтобы не перегружать сайт, и файл для сохранения данных:

```
def __init__(self, base_url: str = "https://proza.ru/texts/list.html", delay: float =  
    ↪ 1.5):  
    self.base_url = base_url  
    self.headers = {"User-Agent": UserAgent().random}  
    self.timeout = 10  
    self.delay = delay  
    self.output_file = "../data/temp_data/json/data_proza_ru.json"  
  
    with open(self.output_file, 'w', encoding='utf-8') as f:  
        json.dump({}, f, ensure_ascii=False, indent=4)
```

Метод `_get_page`: загружает страницу и возвращает BeautifulSoup-объект.

```
time.sleep(self.delay)
response = requests.get(url, headers=self.headers, timeout=self.timeout)
response.raise_for_status()
return BeautifulSoup(response.content, "html.parser")
```

Метод `get_all_forms()`: получает все категории, представленные на сайте Proza.ru.

Сначала он загружает HTML-код основной страницы с помощью вспомогательного метода `_get_page()`. Далее из полученного документа извлекается блок с разделами произведений, после чего метод проходит по каждому подразделу и собирает ссылки на все категории. Для каждой категории сохраняется название и полная ссылка на соответствующую страницу. В результате метод возвращает словарь, где ключом является название категории, а значением — URL страницы.

```
def get_all_forms(self) -> Dict[str, str]:
    """Получает названия и ссылки на разделы с малыми формами."""
    soup = self._get_page(self.base_url)
    if not soup:
        return {}

    works_block = soup.find('ul', attrs={'type': 'square', 'style':
        ↳ 'color:#404040'})
    all_forms = works_block.find_all('ul', attrs={'type': 'square'})
    data_all_forms = {}
    for form in all_forms:
        category = form.find_all('a')
        for link in category:
            title = link.text.strip()
            full_link = "https://www.proza.ru" + link['href']
            data_all_forms[title] = full_link

    return data_all_forms
```

Метод `clean_text()` выполняет очистку HTML-контента и извлечение чистого текста из страницы.

Сначала метод создаёт объект BeautifulSoup для анализа HTML-кода. Затем удаляются все нерелевантные элементы, такие как: `iframe`, `img`, `script`, `style`, рекламные блоки (`div.ads`, `div.advertisement`) и блоки видео (`div.video-blk`, `div.video-block`). Дополнительно удаляются скрытые элементы с классом,

содержащим 'hidden'. После удаления всех лишних элементов текст извлекается с сохранением разделения строк, а пустые строки удаляются. Метод возвращает очищенный текст в виде строки.

```
def clean_text(self, html: str) -> str:
    soup = BeautifulSoup(html, 'html.parser')
    for element in soup(['iframe', 'img', 'script', 'style',
                        'div.video-blk', 'div.video-block',
                        'div.ads', 'div.advertisement']):
        element.decompose()

    for div in soup.find_all('div', class_=lambda x: x and 'hidden' in x):
        div.decompose()

    clean_text = soup.get_text(separator='\n', strip=True)
    lines = [line.strip() for line in clean_text.split('\n') if line.strip()]
    return '\n'.join(lines)
```

Метод `get_works()` выполняет сбор всех произведений в рамках выбранной категории на сайте Proza.ru.

Сначала метод загружает HTML-страницу категории с помощью вспомогательного метода `_get_page()`. Затем из документа извлекаются блоки с перечнем произведений. Для каждого произведения метод получает:

1. имя автора (`author`);
2. заголовок произведения (`title`);
3. ссылку на полную страницу (`link`);
4. текст произведения (`text`), который загружается и очищается с помощью метода `get_text()`.

Собранные данные сохраняются в словарь, где ключом является имя автора, а значением — словарь с заголовком, ссылкой и текстом произведения. После обработки каждого произведения данные также добавляются в JSON-файл с помощью метода `_update_output_file()`, а между запросами делается задержка для предотвращения блокировки.

Метод возвращает словарь с произведениями категории.

```
def get_works(self, url: str, category_title: str) -> Dict[str, Dict[str, str]]:
    soup = self._get_page(url)
```

```

if not soup:
    return {}

works_block = soup.find_all('ul', attrs={'type': 'square',
↪   'style': 'color:#404040'})
data_small_works = {}

for works_list in works_block:
    for work in works_list.find_all('li'):
        work_data = work.find('a')
        if not work_data:
            continue
        try:
            author = work.find('a', attrs={'class': 'poemlink'}).text.strip()
            title = work_data.text.strip()
            link = "https://www.proza.ru" + work_data['href']
            text = self.get_text(link)

            data_small_works[author] = {
                'link': link,
                'title': title,
                'text': text
            }
            self._update_output_file(category_title, title, data_small_works[title])
            time.sleep(self.delay)
        except Exception as e:
            print(f"Ошибка при обработке произведения: {e}")
            continue

return data_small_works

```

Метод `parse_by_dates()` выполняет сбор произведений за указанный период по дате публикации.

На вход метод получает:

1. `start_date` — дата начала периода в формате 'YYYY-MM-DD';
2. `end_date` — дата окончания периода в том же формате;
3. `category_title` — название категории произведений;
4. `topic` — идентификатор темы на сайте.

Метод преобразует даты в объекты `datetime` и организует цикл, который проходит по каждому дню периода в обратном порядке. Для каждой даты формируется URL с указанием дня, месяца, года и темы. Затем вызывается метод `get_works()`, который собирает все произведения на этой странице. Собранные данные добавляются в общий словарь `result`. В конце работы метод возвращает

словарь со всеми произведениями за указанный период, структурированный по авторам и названиям.

```
def parse_by_dates(self, start_date: str, end_date: str, category_title: str, topic: str)
    ↪ -> Dict[str, dict]:
    current_date = datetime.strptime(start_date, "%Y-%m-%d")
    end_date = datetime.strptime(end_date, "%Y-%m-%d")
    result = {}

    while current_date >= end_date:
        date_str = current_date.strftime("%Y-%m-%d")
        print(f"\nОбработка даты: {date_str}")

        day = current_date.strftime("%d")
        month = current_date.strftime("%m")
        year = current_date.strftime("%Y")

        url = f"{self.base_url}?day={day}&month={month}&year={year}&topic={topi_
            ↪ c}"
        data_day = self.get_works(url, category_title)
        result.update(data_day)

        current_date -= timedelta(days=1)

    return result
```

Метод `get_all_work()` обеспечивает полный сбор всех форм и произведений внутри них на сайте Proza.ru.

Сначала метод получает список всех категорий форм с помощью метода `get_all_forms()`. Далее для каждой категории выполняются следующие действия:

1. вызывается метод `get_works()` для получения всех произведений, доступных на странице категории;
2. извлекается идентификатор темы (`topic`) из URL категории;
3. вызывается метод `parse_by_dates()` для сбора произведений за определённый период по дате публикации;
4. объединяются результаты обоих методов в один словарь `merged_works`, который содержит все произведения категории;
5. обновлённый словарь добавляется в общий словарь `all_data`, где ключом является название категории.

В результате метод возвращает полный словарь, структурированный по категориям, авторам и названиям произведений.

```
def get_all_work(self) -> Dict[str, Dict[str, Dict[str, str]]]:
    """Получает все малые формы и произведения внутри них."""
    all_data = {}
    all_forms = self.get_all_forms()

    for all_form_title, all_form_link in all_forms.items():
        print(f"\nОбработка категории: {all_form_title}")
        works = self.get_works(all_form_link, all_form_title)
        topic = re.search(r'topic=(\d+)', all_form_link).group(1)
        works_by_data = self.parse_by_dates("2025-07-18", "2025-07-11",
            ↪ all_form_title, topic)
        merged_works = {**works, **works_by_data}
        all_data[all_form_title] = merged_works
        time.sleep(self.delay)

    return all_data
```

Метод `_update_output_file()` отвечает за обновление JSON-файла при добавлении нового произведения.

Он выполняет следующие действия:

1. считывает существующие данные из файла JSON;
2. проверяет, существует ли категория произведения, и при необходимости создаёт новый словарь для неё;
3. добавляет или обновляет запись с заголовком произведения и его данными (author, title, text, link);
4. сохраняет обновлённый словарь обратно в JSON-файл.

В случае ошибки при чтении или записи файла метод выводит сообщение об ошибке.

```
def _update_output_file(self, category: str, title: str, work_data: dict):
    """Обновляет JSON-файл, добавляя новое произведение"""
    try:
        with open(self.output_file, 'r', encoding='utf-8') as f:
            existing_data = json.load(f)

        if category not in existing_data:
            existing_data[category] = {}
        existing_data[category][title] = work_data
```

```

with open(self.output_file, 'w', encoding='utf-8') as f:
    json.dump(existing_data, f, ensure_ascii=False, indent=4)

except Exception as e:
    print(f" Ошибка при обновлении файла: {e}")

```

Метод `save_to_json()` сохраняет весь собранный корпус данных в отдельный JSON-файл.

Он принимает на вход словарь с данными и имя файла для сохранения. В случае успешного сохранения выводится сообщение с подтверждением. Если возникает ошибка при записи файла, метод выводит сообщение об ошибке.

```

def save_to_json(self, data: dict, filename: str = "data/data_proza_ru.json"):
    """Сохраняет данные в JSON-файл."""
    try:
        with open(filename, 'w', encoding='utf-8') as f:
            json.dump(data, f, ensure_ascii=False, indent=4)
        print(f"\nДанные сохранены в {filename}")
    except Exception as e:
        print(f"Ошибка при сохранении JSON: {e}")

```

2.1.2 Парсинг ЛитПричал

Данный парсер реализует сбор текстов и метаданных с сайта «ЛитПричал» и принципиально отличается от парсинга Proza.ru рядом особенностей архитектуры ресурса. Основным объектом структуры являются жанры, а не категории. Это определяет общую логику обхода страниц.

Парсер реализован в виде класса `LitPrichalParser`, где в конструкторе задаются базовый URL, заголовки и таймауты запросов

```

def __init__(self, base_url: str = "https://www.litprichal.ru"):
    self.base_url = base_url
    self.headers = {"User-Agent": UserAgent().random}
    self.timeout = 10

```

Ключевым отличием является метод `get_genres()`. Он извлекает с главной страницы списка прозы все доступные жанровые разделы. Жанры представлены в блоках:

```
<div class="col-sm-6 col-md-4">...</div>
```


Каждый жанр содержит одну или несколько ссылок на подразделы. Метод собирает названия и формирует абсолютные ссылки через `urljoin`.

```
def get_genres(self) -> Dict[str, Dict[str, str]]:
    url = f"{self.base_url}/prose.php"
    soup = self._get_page(url)

    genre_blocks = soup.find_all("div", class_="col-sm-6 col-md-4")
    for block in genre_blocks:
        for genre_link in block.find_all("a"):
            name = genre_link.text.strip()
            link = urljoin(self.base_url, genre_link.get("href"))
            genres[name] = {"link": link}
```

В отличие от Proza.ru, где категории группировались в отдельные HTML-списки, здесь структура линейно распределена по bootstrap-блокам.

Сайт ЛитПричал использует постраничную навигацию внутри жанров. Для определения количества страниц применяется метод `_get_page_count()`.

```
def _get_page_count(self, genre_url: str) -> int:
    pagination = soup.find("ul", class_="pagination")
    pages = pagination.find_all("li")
    last_page = int(pages[-1].find("a").get("href").strip("/").split("/")[-1].replace("p",
↵    ""))
```

Таким образом, логика сбора на ЛитПричале строится на последовательном переборе страниц.

Произведения собираются методом `get_books()`. На каждой странице ищутся элементы:

```
<div class="col-md-6 x2">...</div>
```

Из каждого извлекаются заголовок, ссылка и имя автора.

Для извлечения текста используется метод `get_text()`. Он выбирает второй блок `div.col-md-12 x2`, где находится основной текст.

```
text_blocks = soup.find_all("div", class_="col-md-12 x2")
return self.clean_text(str(text_blocks[1]))
```

На Proza.ru текстовая часть извлекалась из более сложной структуры страницы; здесь доступ проще и локализован в одном блоке.

Метод `clean_text()` структурно похож на аналогичный в Proza.ru, отличается отсутствием обработки некоторых специфичных элементов.

В отличие от Proza.ru отсутствует инкрементальное обновление JSON-файла. Все данные собираются в оперативной памяти и записываются по завершении.

Финальный сбор производится методом `parse_all_in_genre()`. Он получает все жанры, затем вызывает `get_books()` для каждого:

```
for genre_name, genre_data in genres.items():
    books = self.get_books(genre_data["link"])
    result[genre_name] = books
```

В целом логика данного парсинга адаптирована под архитектуру сайта ЛитПричал и нацелена на сбор произведений по жанрам с учётом пагинации, без анализа дат публикации и без инкрементального сохранения данных.

2.1.3 Парсинг Литрес

Парсер сайта ЛитРес предназначен для сбора метаданных о книгах и получения коротких текстовых аннотаций со страниц произведений. В отличие от сайтов Proza.ru и ЛитПричал, ресурс ЛитРес содержит преимущественно коммерческий контент и использует более сложную верстку, что усложняет извлечение информации.

Парсер реализован в виде класса `LitresParser`. При инициализации задаются базовый URL жанровой страницы, заголовки HTTP-запросов, таймауты и путь для сохранения результатов в JSON-файл.

```
def init(self, url: str):
    self.litres_url = "https://www.litres.ru"
    self.base_url = url
    self.headers = {"User-Agent": UserAgent().random}
    self.timeout = 10
    self.filename = "/data/temp_data/litres.json"
```

Основной вспомогательный метод `_get_page()` отвечает за загрузку HTML-страницы. Он осуществляет запрос к серверу с задержкой, обрабатывает сетевые ошибки и возвращает объект `BeautifulSoup`:

```
def _get_page(self, url: str) -> Optional[BeautifulSoup]:
    time.sleep(1.5)
    response = requests.get(url, headers=self.headers, timeout=self.timeout)
    response.raise_for_status()
    return BeautifulSoup(response.content, "html.parser")
```

Для обхода каталога и извлечения списка книг используется метод `get_books()`. Он поддерживает пагинацию: на вход передается целевое количество страниц, после чего метод формирует URL вида: `<base_url>?page=<номер>`

На каждой странице ищутся карточки произведений:

```
<div class="Art-module__3wrtfG__content
→ Art-module__3wrtfG__content_full">...</div>
```

Из каждого блока извлекаются:

- заголовок книги;
- ссылка на страницу произведения;
- текст аннотации (через метод `get_text()`).

При этом предусмотрена проверка на дублирование: если книга уже присутствует в ранее сохраненных данных, повторная обработка не выполняется.

```
for book in books:
    info = book.find("a", class_="ArtInfo-module__Y-DtKG__title")
    title = info.text.strip()
    link = self.litres_url + info.get("href")
    if title in books_data:
        continue
```

Извлечение текстовой информации организовано в методе `get_text()`. Аннотация находится внутри блока:

```
<div class="BookDetailsAbout-module__p8ABVW__truncate">...</div>
```

и затем уточняется вложенный элемент:

```
truncated = block.find("div", class_="Truncate-module__FwxwPG__truncated")
return truncated.text if truncated else None
```

В отличие от Proza.ru и ЛитПричала, на ЛитРес нельзя получить полный текст произведения из-за авторских прав и ограничений доступа. Поэтому парсер извлекает только доступные публичные аннотации.

Сохранение результатов производится инкрементально после каждой обработанной страницы. Метод `save_to_json()` записывает текущий словарь в файл:

```
def save_to_json(self, data: Dict, filename: str = None) -> None:
    with open(filename or self.filename, "w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=4)
```

Перед обработкой выполняется загрузка уже имеющегося JSON-файла:

```
def _load_existing_data(self) -> Dict:
    try:
        with open(self.filename, "r", encoding="utf-8") as f:
            return json.load(f)
    except:
        return {}
```

Таким образом, структура данных постепенно дополняется, что позволяет продолжать сбор даже при прекращении работы или сетевых ошибках.

В целом логика парсинга ЛитРес характеризуется следующими особенностями:

- опора на пагинацию по параметру `page`;
- сохранение данных после каждой итерации;
- невозможность получения полного текста произведения;
- извлечение только открытых аннотаций;
- проверка на дубликаты по названию книги.

Такой подход обеспечивает устойчивость обработки и минимальные потери данных при длинных сериях запросов.

2.1.4 Парсинг Briefly

Парсер, реализованный для ресурса `Briefly.ru`, предназначен для извлечения кратких пересказов произведений мировой литературы.

Функциональность организована в классе `ParsingBriefly`. В конструкторе задаются базовый URL, заголовки HTTP-запросов, сетевые параметры и путь сохранения результатов:

```
def init(self, url: str):
    self.briefly_url = "https://briefly.ru"
    self.base_url = url
    self.headers = {"User-Agent": UserAgent().random}
    self.timeout = 5
    self.filename = "../data/temp_data/json/briefly.json"
```

Получение HTML-страниц осуществляется вспомогательным методом `_get_page` который генерирует задержки между запросами для имитации поведения человека:

```
def _get_page(self, url: str) -> Optional[BeautifulSoup]:
    self.human_delay()
    response = requests.get(url, headers=self.headers, timeout=self.timeout)
    return BeautifulSoup(response.content, "html.parser")
```

Использование искусственных задержек снижает риск блокировки со стороны сервера.

Метод `get_all_data()` обходит карточки культур (например, русская, французская, античная), находящиеся на корневой странице Briefly.ru. Для каждой культуры извлекаются имена авторов:

```
cultures_cards = soup.find_all("a", class_="visited-hidden")
for culture in cultures_cards[7:]:
    name = culture.get_text(strip=True)
    link = self.briefly_url + culture.get("href")
    data_culture = self.get_authors(link, name)
```

Таким образом обеспечивается последовательный обход каталога.

Парсер использует метод `get_works()`, который распознаёт две возможные структуры страницы:

- `works_index` — содержит список полных и кратких пересказов;
- `author_works` — альтернативное оформление профиля автора.

Дополнительно выполняется фильтрация технических названий (например, «глава», «том», «действие»), чтобы не загружать фрагменты произведений:

```
if any(word in title.lower() for word in ["глава", "том", "действие"]):
    continue
```

Извлечение текста реализовано в методе `get_text()`, который учитывает различные варианты вёрстки страниц:

- краткие пересказы (pending) находятся в `div.microsummary__content`;
- полные пересказы (published) — в `p.microsummary__content`;
- если формат иной — применяется резервный поиск в `divtext`.

Перед сохранением нежелательные рекламные элементы удаляются:

```
for ad in main_div.find_all("div", class_="honey"):
    ad.decompose()
```

Завершающий текст формируется объединением всех параграфов.

Отличительной особенностью является метод `human_delay()`, который:

- создаёт случайные задержки между запросами;
- с небольшой вероятностью инициирует длинную паузу;
- делает парсер менее «роботоподобным».

```
delay = random.uniform(base, base + var)
if random.random() < long_pause_prob:
    time.sleep(random.uniform(5, 15))
```

Подобный механизм снижает риск антибот-ограничений.

Для сохранения прогресса используется подход постепенной записи в JSON-файл:

```
self.save_to_json(all_data)
```

В случае прерывания процесса данные не теряются, и загрузка продолжается с последнего сохранённого состояния.

Разработка обладает рядом отличительных черт:

- глубокая вложенность навигации (культура → автор → произведения);
- обработка нескольких шаблонов вёрстки страниц;
- фильтрация неполных текстов (глав, томов, действий);
- удаление рекламных блоков;
- антибот-стратегия (случайные паузы);

— инкрементальное сохранение данных.

Благодаря этому достигается устойчивость работы при обработке большого количества страниц и минимизируется риск блокировок.

2.2 Предобработка и анализ данных

На каждом из этих сайтов тексты имели различный формат представления, поэтому на первом этапе требовалось привести их к единому виду и очистить от лишней информации.

Были написаны вспомогательные функции для открытия и преобразования данных из формата JSON в DataFrame.

Были реализованы две функции:

1. `open_json()` — открывает JSON-файл и возвращает его содержимое в виде словаря.

```
def open_json(input_file: str) -> dict:
    with open(input_file) as json_file:
        data = json.load(json_file)
    return data
```

2. `json_to_csv()` и `json_to_csv_lp()` — преобразуют данные разных форматов (в зависимости от сайта-источника) в таблицу с двумя основными столбцами:

— `title` — название произведения,

— `text` — текст произведения или его описание.

```
def json_to_csv_lp(json_data: dict) -> DataFrame:
    rows = []
    for category, works in json_data.items():
        for author, work_data in works.items():
            rows.append({
                "title": work_data.get("title", ""),
                "text": work_data.get("text", "")
            })

    df = pd.DataFrame(rows)
    return df
```

```
def json_to_csv(json_data: dict) -> DataFrame:
    rows = []
    for title, text in json_data.items():
        rows.append({
```

```

        "title": title,
        "text": text
    })

    df = pd.DataFrame(rows)
    return df

```

Далее все таблицы были объединены в один общий датасет:

```
data = pd.concat([briefly, litprichal, proza_ru, litres], ignore_index=True)
```

Проверка количества записей показала, что данные объединились без потерь.

На этапе первичной очистки все текстовые данные были приведены к нижнему регистру и очищены от лишних пробелов:

```
data = data.apply(lambda col: col.str.lower().str.strip())
```

Затем были удалены пропущенные значения (NaN) и пустые строки. Это позволило избавиться от записей, в которых отсутствовали либо текст, либо название.

Для текстов описаний была создана функция `clean_text()`, которая выполняла следующие действия:

- удаляла ссылки (http), HTML-теги и спецсимволы;


```
text = re.sub(r"http\S+", "", text)
```
- заменяла несколько пробелов на один;


```
text = re.sub(r"<[>]+>", "", text)
```
- оставляла только буквы, цифры и основные знаки пунктуации.


```
text = re.sub(r"[^\w\s,!?-]", " ", text)
```

Результат — тексты были приведены к единому читаемому виду, пригодному для анализа и подачи модели.

Во многих названиях встречались элементы нумерации, такие как «том 2», «глава 5», «эпизод 3» и т.п. Для их удаления была реализована функция `clean_title_completely`.

`clean_title_completely(title)` — функция для «полной» очистки названия произведения от любых форм нумерации (тома, главы, части, эпизоды и т.п.), а также от сопутствующего шумового мусора (скобки с числами, служебные символы, лишняя пунктуация и т.д.). Результат — компактная читабельная строка, пригодная как целевая метка для обучения модели генерации заголовков.

```
cleaned = remove_complex_volume_numbers(title)
cleaned = str(cleaned).lower().strip()
```

`Complex_patterns` используется для удаления более сложных конструкций нумерации, включающих сочетания нескольких типов меток (том, часть, эпизод, серия, глава, книга) и двух и более числовых значений.

```
complex_patterns = [
    r' \ (? \ s* \ d+ \ s+ (? : эпизод | серия | глава | часть ) \ s+ \ d+ \ s+ (? : том | книга | т \ . ) \ s* \ d _
    ↪ * \ s* \ . ? \ ) ? ',
    r' \ (? \ s* \ d+ \ s+ (? : том | книга | т \ . ) \ s+ \ d+ \ s+ (? : эпизод | серия | глава | часть ) \ s* \ d _
    ↪ * \ s* \ . ? \ ) ? ',
    r' \ b \ d+ \ s+ \ d+ \ s+ (? : том | часть | книга | эпизод | серия ) \ b ',
    r' \ b (? : том | часть | книга | эпизод | серия ) \ s+ \ d+ \ s+ \ d+ \ b ',
    r' \ b \ d+ [ - \ . , ] \ s* \ d+ \ s+ (? : том | часть | книга ) ',
    r' \ b (? : том | часть | книга ) \ s+ \ d+ [ - \ . , ] \ s* \ d+ \ b ',
]
```

```
for pattern in complex_patterns:
    cleaned = re.sub(pattern, ' ', cleaned, flags=re.IGNORECASE)
```

1. `\ (? \ s* \ d+ \ s+ (? : эпизод | серия | глава | часть ) \ s+ \ d+ \ s+ (? : том | книга | т \ . ) \ s* \ d* \ s* \ . ? \ ) ?`

— Ищет сложные конструкции вида: число + тип контента + число + тип издания;

— Например, (12 серия 3 том), 5 глава 2 книга.

2. `\ (? \ s* \ d+ \ s+ (? : том | книга | т \ . ) \ s+ \ d+ \ s+ (? : эпизод | серия | глава | часть ) \ s* \ d* \ s* \ . ? \ ) ?`

— Обратный порядок: число + тип издания + число + тип контента;

— Например, (3 том 12 серия), 2 книга 5 глава.

3. `\ b \ d+ \ s+ \ d+ \ s+ (? : том | часть | книга | эпизод | серия ) \ b`

— Ищет два числа подряд + тип контента/издания;

- Например, 12 3 том, 5 2 книга.
- 4. `\b(?:том|часть|книга|эпизод|серия)\s+\d+\s+\d+\b`
 - Обратный порядок: тип контента/издания + два числа;
 - Например, том 12 3, книга 5 2.
- 5. `\b\d+[-\.,]\s*\d+\s+(?:том|часть|книга)`
 - Ищет числа с разделителями + тип издания;
 - Например, 12-3 том, 5.2 книга.
- 6. `\b(?:том|часть|книга)\s+\d+[-\.,]\s*\d+\b`
 - Обратный порядок: тип издания + числа с разделителями;
 - Например, том 12-3, книга 5.2, часть 10,1.

Basic_patterns используется для того, чтобы удалить распространённые сочетания слов-меток (том, часть, глава, книга, эпизод, серия, выпуск, т.) вместе с последующими/предшествующими цифрами или римскими цифрами.

```
basic_patterns = [
    r '\s*(?:том|часть|книга|т\.|vol\.|?)эпизод|серия|глава|выпуск)\s*[ivxlcdm0-9]+' ,
    r '\s*[ivxlcdm0-9]+\s*(?:том|часть|книга|т\.|vol\.|?)эпизод|серия|глава|выпуск)' ,
    r '\s*\d+[-\.,]? \s*(?:том|часть|книга|глава)' ,
    r '\s*(?:том|часть|книга|глава)[-\.,]? \s*\d+' ,
]
for pattern in basic_patterns:
    cleaned = re.sub(pattern, ' ', cleaned, flags=re.IGNORECASE)
```

1. `\s*(?:том|часть|книга|т\.|vol\.|?)эпизод|серия|глава|выпуск)\s*[ivxlcdm0-9]+`
 - Ищет слово типа том (или сокращение т., vol.) + пробелы + число (арабское) или римское (ivxlcdm);
 - `\s*` до и после — чтобы удалить предшествующий пробел и не оставлять двойных пробелов.
2. `\s*[ivxlcdm0-9]+\s*(?:том|часть|книга|т\.|vol\.|?)эпизод|серия|глава|выпуск)`
 - Обратная форма: число перед словом;
 - Например, 2 том, iii том.
3. `\s*\d+[-\.,]? \s*(?:том|часть|книга|глава)`

- Захватывает варианты с разделителями между числом и словом;
- Например, 10-том, 10, том, 10. том.

4. `\s*(?:том|часть|книга|глава)[-\.]? \s*\d+`

- Аналогичный случай, когда разделитель стоит после слова;
- Например, том-10, том. 10.

Для того чтобы удалить текстовые порядковые обозначения (глава вторая, часть первая и т.д.) был написан паттерн:

```
number_words = ["первая", "вторая", "третья", ... , "двадцатая"]
text_num_pattern = r'\b(?:глава|часть|том|эпизод|серия|книга|выпуск)\s+(?:' +
    ↪ '"|'.join(number_words) + r')\b'
cleaned = re.sub(text_num_pattern, ' ', cleaned, flags=re.IGNORECASE)
```

Для удаления форм с русскими окончаниями (падежные и порядковые суффиксы): 2-я глава, 3й том, 4ая часть, том 5-ая и т.п., были написаны паттерны:

```
cleaned = re.sub(
    r'\b\d+[-]?(?:я|й|е|ой|ая|ое|ые|ых)?\s+(?:глава|часть|том|книга|серия|эпизод|в |
    ↪ ыпуск)\b',
    ' ', cleaned, flags=re.IGNORECASE
)
cleaned = re.sub(
    r'\b(?:глава|часть|том|книга|серия|эпизод|выпуск)\s+\d+[-]?(?:я|й|е|ая|ое|ые|
    ↪ ых)?\b',
    ' ', cleaned, flags=re.IGNORECASE
)
```

1. `[-]?` учитывает разные дефисы/тире (обычный - и длинный —);
2. `(?:я|й|е|ой|ая|ое|ые|ых)?` — возможные окончания, которые часто встречаются у порядковых числительных.

Удаление символа «№»:

```
cleaned = re.sub(r'№', ' ', cleaned)
```

Также были удалены скобки с числами и одиночных чисел на конце/начале

```
cleaned = re.sub(r'\([^\)]*\d+[^\)]*\)', ' ', cleaned)
cleaned = re.sub(r'\s+\d+\s*\.?\$', ' ', cleaned)
cleaned = re.sub(r'^\d+\s+', ' ', cleaned)
```

1. `\([^\)]*\d+[^\)]*\)` — удаляет любые круглые скобки, в которых есть цифры, например (том 2), (№10, переработанное);

2. `\s+\d+\s*\.?\$` — удаляет числа, стоящие в конце строки, возможно с точкой;
3. `^\d+\s+` — удаляет ведущие числа в начале строки.

После всего были удалены прочие нежелательные символы и нормализованы пробелы

```
cleaned = re.sub(r'[\^\w\s.,!?-]', ' ', cleaned)
cleaned = re.sub(r'\s+', ' ', cleaned)
cleaned = cleaned.strip(' ,.-')
```

1. `r'[\^\w\s.,!?-]'` — удаляет всё, кроме букв / цифр / подчёрки (`\w`), пробелов, и набора разрешённых знаков `. , ! ? -`. Это убирает лишние спецсимволы, кавычки, другие скобки, символы валют и т.п.
2. Затем сводим последовательные пробелы к одному и убираем ведущие / замыкающие пробелы и пунктуацию `, . -`.

В результате очистки было изменено 8666 названий из 32085.

После предобработки были просмотрены пропуски в данных. Просто NaN не было, зато встречались тексты и названия, в которых не было никаких символов. Также в текстах часто встречалась заглушка «описание отсутствует». Все такие строки были удалены.

```
data = data[data.text != ""].reset_index(drop=True)
data = data[data.title != ""].reset_index(drop=True)
data = data[data.text != "описание отсутствует"].reset_index(drop=True)
```

В некоторых строках встречался нерусский текст. Так как датасет собирается для модели генерации на русском языке, такие строки были почищены, чтобы не вносить модели лишний шум. Для этого была написана функция `keep_only_russian`, которая оставляла только русские символы, цифры и пунктуацию в тексте. Если после этого появлялись пропуски, то они удалялись.

```
def keep_only_russian(text):
    if pd.isna(text):
        return ""
    text = re.sub(r"[^А-Яа-яЁё0-9\s.,!?-]", "", text)
    text = re.sub(r"\s+", " ", text).strip()
```

```

return text

data.text = data.text.apply(keep_only_russian)
data.cleaned_title = data.cleaned_title.apply(keep_only_russian)

data = data[(data.text != "") & (data.cleaned_title != "")].reset_index(drop=True)

```

После первичной обработки данных были дополнительно проверены строки, содержащие текст. В датасете встречались случаи, когда строка формально не была пустой, однако не содержала никаких значимых символов — только пробелы или пунктуацию. Такие строки не несут полезной информации и потенциально могут добавить шум при обучении языковой модели.

Чтобы отфильтровать подобные случаи, была написана вспомогательная функция `is_meaningful`, которая проверяет, содержит ли текст хотя бы один букво-цифровой символ (на русском или английском языке).

Также при парсинге на некоторых сайтах в начале текста встречалась лишняя ведущая пунктуация (например, «...», «—», «***»). Для устранения этого была реализована функция `clean_leading_punct`, которая удаляет начальные небуквенные символы с помощью регулярного выражения.

После применения этих функций датасет очищается от нерелевантных строк. Далее выполняется нормализация текста:

1. Замена всех символов, не относящихся к слову, пробелу или базовой пунктуации, на пробел;
2. Устранение избыточных пробелов.

В результате тексты становятся более однородными и удобными для дальнейшей обработки.

```

def is_meaningful(text):
    return bool(re.search(r"[А-Яа-яА-Zа-z0-9]", text))

def clean_leading_punct(text):
    return re.sub(r"^[^\wА-Яа-я0-9]+", "", text).strip()

data = data[data.text.apply(is_meaningful)].reset_index(drop=True)
data.text = data.text.str.replace(r"^[^\w\s,!?-]", " ", regex=True)
data.text = data.text.str.replace(r"\s+", " ", regex=True).str.strip()

```

После нормализации была проведена проверка на дубликаты. Сначала было посчитано их общее количество (430), затем — количество повторяющихся значений отдельно по тексту (762) и названию (2127).

Оказалось, что дубликаты названий встречаются достаточно часто — это допустимо, так как одни и те же произведения могут быть размещены на разных сайтах и иметь одинаковые заголовки.

Однако более критичны случаи, когда совпадает текст, а названия различаются. Это означает, что одно и то же произведение было спарсено несколько раз с разных источников. Такие строки следует удалить, чтобы не дублировать обучающие данные и не вносить перекося в модель.

Для этого были удалены все повторяющиеся тексты, оставив от каждого дубликата только один экземпляр.

```
data.duplicated().sum()
data.text.duplicated().sum(), data.title.duplicated().sum()

data = data[~data.text.duplicated(keep=False)].reset_index(drop=True)
data.duplicated().sum()
```

Таким образом, датасет был очищен от нерелевантных строк и повторяющихся текстов, что положительно скажется на качестве обучения генеративной модели.

На следующем этапе был проведён анализ длины заголовков (названий произведений). Сначала было вычислено распределение частот появления каждого уникального заголовка, после чего для каждого названия была рассчитана длина — количество слов, содержащихся в нём. Это позволило сгруппировать данные и оценить, какие длины встречаются чаще всего.

Полученное распределение визуализировано в виде столбчатой диаграммы. По графику 2 видно, что в датасете присутствуют названия, состоящие из большого количества слов. Такие случаи обычно характерны либо для метаданных, случайно попавших в заголовок, либо для чрезмерно подробных описаний. Поскольку задача — обучить модель генерировать короткие и понятные художе-

ственные названия, слишком длинные заголовки могут вносить шум и снижать качество обучения.

```
data["title_len"] = data.title.apply(lambda x: len(str(x).split()))
length_counts = data.groupby('title_len').size()

plt.figure(figsize=(12,6))
length_counts.plot(kind='bar')
plt.xlabel("Длина названия (слов)")
plt.ylabel("Количество названий")
plt.title("Распределение длин названий в датасете без аугментации")
plt.show()
```



Рисунок 2 – Распределение длин названий в датасете без аугментации

После анализа было принято решение ограничить длину названий до 10 слов, поскольку более длинные примеры встречаются редко и не являются репрезентативными. Соответствующие строки были удалены, а датасет пересчитан.

Далее были рассчитаны дополнительные статистики: количество примеров, число уникальных заголовков, средняя длина текста и средняя длина заголовков. Эти значения позволяют оценить «средний масштаб» данных и убедиться, что после фильтрации структура выборки остаётся адекватной для обучения модели.

После этого вспомогательные столбцы `title_len` и `text_len` были удалены, а финальный датасет сохранён в CSV-файл для дальнейшего использования.

ного добавления новых источников. Это особенно важно при обучении моделей обработки естественного языка, так как наличие более широкого набора текстов способствует лучшей генерализации и снижению переобучения.

Перед аугментацией исходные данные были корректно разделены на тренировочную и валидационную выборки с учётом ограничения на повторяющиеся названия. Это позволило избежать потенциальной утечки информации: валидационная выборка после повторного разбиения не содержит текстов с одинаковыми заголовками, что улучшает объективность последующей оценки модели.

```
train_df, val_df = train_test_split(data, test_size=0.2, random_state=42)
```

После предварительного анализа стало видно, что в первоначальную валидационную выборку попали несколько текстов с одинаковыми названиями. Это создаёт риск искажения оценки качества модели, если одно и то же произведение (или его вариации) встречается и в обучении, и в валидации, модель может фактически «узнать» его структуру, а не обобщать.

Чтобы избежать подобной утечки данных, было выполнено контролируемое разделение датасета.

Для этого реализована функция `split_with_controlled_test_size`, которая:

1. Группирует записи по названию произведения.
2. Перемешивает список уникальных заголовков.
3. Для каждого заголовка случайным образом выбирает ровно один пример в валидационную выборку (пока не будет набран заданный размер).
4. Остальные примеры заголовка попадают в обучение.

Таким образом мы гарантируем, что один и тот же заголовок не встречается в обеих выборках одновременно, что обеспечивает более честную оценку генеративной модели.

```
def split_with_controlled_test_size(data, target_test_size=0.2, random_state=42):
```

```

np.random.seed(random_state)

title_groups = data.groupby('title').apply(lambda x: x.index.tolist()).to_dict()

unique_titles = list(title_groups.keys())
np.random.shuffle(unique_titles)

train_indices = []
test_indices = []

target_test_count = int(len(data) * target_test_size)

for title in unique_titles:
    indices = title_groups[title]

    if len(test_indices) < target_test_count:
        test_idx = np.random.choice(indices, 1)[0]
        test_indices.append(test_idx)
        train_indices.extend([idx for idx in indices if idx != test_idx])
    else:
        train_indices.extend(indices)

return data.iloc[train_indices], data.iloc[test_indices]

```

После выполнения функции было выведено распределение размеров выборок, что позволило убедиться в корректности деления:

```

Общий размер данных: 29815
Тренировочная выборка: 23852 записей (80.0%)
Валидационная выборка: 5963 записей (20.0%)

```

После формирования тренировочной выборки был выполнен этап аугментации. Важно подчеркнуть, что аугментировать нужно только train-набор, чтобы избежать утечки информации в валидацию, которая служит для объективной оценки качества модели.

Поэтому перед началом процедуры индекс тренировочного датасета был сброшен.

Для увеличения количества обучающих примеров текст каждой записи был разбит на небольшие смысловые фрагменты. Разбиение происходит по предложениям с фиксированной длиной блока: в данном случае — по три предложения на каждый фрагмент. Это позволяет:

1. увеличить объём обучающих данных;

2. дать модели больше разнообразия контекста;
3. улучшить способность генерации коротких связных отрывков.

Для этого была реализована функция `split_text_into_chunks`, использующая токенизацию предложений:

```
def split_text_into_chunks(text, sentences_per_chunk=3):
    sentences = sent_tokenize(text, language="russian")
    chunks = []
    for i in range(0, len(sentences), sentences_per_chunk):
        chunk = " ".join(sentences[i:i + sentences_per_chunk])
        chunks.append(chunk)
    return chunks
```

Каждый `chunk` получает тот же заголовок, что и оригинальный текст, так как принадлежит одному произведению.

В итоге формируется новый датасет `augmented_dataset`, размер которого значительно превышает исходный `train`-набор.

После генерации `chunk`'ов был выполнен знакомый по предыдущим этапам этап очистки

1. Удаление строк, не содержащих значимых символов (букв или цифр);
2. Удаление лишней пунктуации в начале строки;
3. Замена нежелательных символов на пробел;
4. Нормализация пробелов и удаление хвостовых отступов.

После разбиения некоторые `chunk`'и оказались слишком короткими или малоинформативными. Такие строки не несут полезной контекстной нагрузки и могут ухудшить качество обучения, особенно при генерации связного текста.

Поэтому было добавлено дополнительное условие фильтрации:

```
augmented_dataset = augmented_dataset[augmented_dataset.text.str.strip().str.len()
→ > 10]
```

Это позволяет:

1. Отсеять «обрывки»;
2. Исключить заглушки;
3. Сохранить только содержательные обучающие примеры.

После аугментации тренировочных данных (разбиения текстов на фрагменты) образовался существенно расширенный набор записей. Однако дальнейший анализ выявил ряд потенциальных проблем, которые необходимо устранить, чтобы избежать ухудшения качества обучения модели.

Из-за разбиения длинных текстов на множество фрагментов отдельные заголовки (title) начали появляться слишком часто. Это создает дисбаланс и может привести модель к переобучению на популярные заголовки, снижая её способность обобщать.

Чтобы равномерно распределить данные, было принято решение оставить не более 500 фрагментов на каждый заголовок:

```
max_per_title = 500
augmented_dataset = augmented_dataset.groupby("title").head(max_per_title).reset_index(drop=True)
```

Таким образом, наиболее длинные произведения перестают доминировать в обучающем корпусе, а модель получает более стабильный распределённый по заголовкам набор данных.

После обрезки по количеству записей появился небольшой процент точных дубликатов, где совпадали и text, и title. Такие записи никак не обогащают датасет и фактически дублируют сигнал для модели, поэтому были удалены. Это позволяет снизить помощь модели «запоминать» конкретные фрагменты.

Следующий обнаруженный источник шума — случаи, когда один и тот же текстовый фрагмент встречается под разными названиями. Это может привести к обучению модели ложным связям: одинаковый контент начинает ассоциироваться с разными заголовками, что затрудняет обучение генерации.

Такие записи были удалены с помощью фильтрации по дубликатам на уровне столбца text:

```
augmented_dataset = augmented_dataset[~augmented_dataset.text.duplicated(keep=False)].reset_index(drop=True)
```

В завершение был выполнен дополнительный анализ:

```
dup_titles = (
    augmented_dataset.groupby("text")["title"]
```

```

        .nunique()
        .reset_index()
        .query("title > 1")
    )

```

Он показывает, сколько фрагментов всё ещё имеют более одного уникального заголовка. После проведённой фильтрации количество таких случаев больше не было, что подтверждает корректность предпринятых действий.

После выполнения аугментации тренировочного набора данных было важно оценить, как изменилось распределение длин заголовков (в словах). Для этого были рассчитаны длины названий в трёх выборках:

- Исходный датасет без аугментации;
- Тренировочная выборка после аугментации;
- Валидационная выборка.

Измерение длины выполнялось по количеству слов в заголовке.

Далее была построена сравнительная гистограмма, представленная на рисунке 4, отображающая распределение полученных значений.



Рисунок 4 – Сравнение распределения длин названий до и после аугментации

По графику можно заметить следующие особенности:

1. Исходное распределение

- Распределение достаточно равномерное в области коротких названий.

- Количество примеров сравнительно небольшое, так как это необработанный датасет без разбиения текста на части.

2. train после аугментации

- Сильно увеличилось общее количество примеров.
- Распределение визуально сохраняет схожую форму: короткие названия встречаются значительно чаще, чем длинные.
- Это закономерно, так как аугментация увеличивает количество текстовых фрагментов, но название у каждого chunk'а остаётся прежним.

3. test/val после аугментации

- Количество названий здесь существенно меньше, так как валидация purposely не аугментируется.
- Распределение по длинам заголовков остаётся очень близким к исходному, что важно для корректной оценки модели.

3 Написание и обучение моделей

3.1 Модель на основе n-gram

Для подготовки текста к обучению модели используется функция `tokenize`, реализующая базовый этап предобработки входных данных. Её задача — преобразовать сырой текст в последовательность токенов, пригодных для последующего анализа и построения n-граммной модели.

На первом шаге весь текст приводится к нижнему регистру. Это устраняет различия между одинаковыми словами, записанными с разным использованием заглавных букв, и уменьшает размер словаря.

Затем выполняется очистка строки с помощью регулярного выражения. Из текста удаляются все символы, не являющиеся буквами латиницы или кириллицы, цифрами или пробелами.

После очистки строка разбивается на отдельные токены по пробелам. Результатом работы функции является список слов, упорядоченных в соответствии с исходным текстом. Эти токены далее используются для построения обучающих последовательностей и формирования n-грамм.

```
def tokenize(text: str) -> List[str]:  
    """Функция самой простой предобработки текста, основанной на разбиении на  
    ↪ токены по пробелам."""  
    text = text.lower()  
    text = re.sub(r"[^a-zA-яё0-9\s]", "", text)  
    return text.split()
```

Для решения задачи генерации названий по входному тексту используется класс `TitleNgramModel`, реализующий статистическую n-граммную модель. Она обучается на парах (текст + название) и оценивает вероятность появления следующего слова на основе нескольких предыдущих.

В методе `__init__` задаются основные настройки и внутренние структуры данных, необходимые для работы модели:

1. `n_gram` — порядок модели, определяющий, сколько предыдущих токенов используется как контекст при прогнозировании следующего слова.

- Например, при `n_gram = 3` модель опирается на биграммный контекст;
2. `ngrams` — словарь, в котором каждому контексту сопоставляется счётчик слов, появляющихся после него. Это ядро модели: оно хранит частоты всех наблюдаемых `n`-грамм;
 3. `context_counts` — количество встреч каждого контекста; Используется при вычислении вероятностей;
 4. `vocab` — словарь уникальных токенов, включающий наиболее частотные слова корпуса и специальные служебные символы;
 5. Специальные токены:
 - `<s>` — начало последовательности;
 - `<title>` — разделитель, отделяющий исходный текст от целевого названия;
 - `</s>` — конец последовательности;
 - `<unk>` — токен для слов, отсутствующих в словаре.
 6. `smoothing` — стратегия сглаживания вероятностей (по умолчанию используется Лапласовское сглаживание). Сглаживание предотвращает нулевые вероятности для слов, отсутствующих в обучающих `n`-граммах;
 7. `alpha` — коэффициент, применяемый в формулах сглаживания;
 8. `lower_order_models` — модели меньшего порядка (например, для 4-граммной модели автоматически создаются 1-, 2- и 3-граммные версии). Они используются для методов типа `linear interpolation` или `backoff`, позволяющих улучшать генерацию при редких или отсутствующих контекстах.

```
class TitleNgramModel:
    """Модель n-грамм, обучающаяся по парам (текст → название)."""

    def __init__(self, n_gram: int = 4, smoothing: str | None = 'laplace', alpha: float
        ↪ = 1.):
        self.n_gram = n_gram
        self.ngrams = defaultdict(Counter)
        self.context_counts = defaultdict(int)
        self.vocab = set()
        self.start_token = "<s>"
        self.title_token = "<title>"
        self.end_token = "</s>"
```



```

self.unknown_token = "<unk>"
self.smoothing = smoothing
self.alpha = alpha
self.lower_order_models = {}

```

Метод `create_ngrams` отвечает за построение последовательностей n -грамм, которые позднее используются при обучении модели. Он принимает на вход список токенов и преобразует его в набор перекрывающихся отрезков длины n , каждый из которых описывает локальный контекст в тексте.

```

def create_ngrams(self, tokens: List[str]) -> list[tuple[str, ...]]:
    """Создание n-грамм с учётом начала и конца."""
    tokens = [self.start_token] + tokens + [self.end_token]
    return [tuple(tokens[i:i + self.n_gram]) for i in range(len(tokens) - self.n_gram + 1)]

```

Дальше идет функция полного обучения моделей.

Метод `train` выполняет полное построение n -граммной модели на основе набора пар (текст $\rightarrow$)..

1. Токенизация данных и формирование общего корпуса:

Первым шагом выполняется преобразование исходных пар в единый поток токенов. Для каждого текста и соответствующего ему названия выполняется разбиение на слова, после чего элементы объединяются в одну последовательность, разделённую специальным маркером `<title>`.

```

all_tokens = []
for text, title in pairs:
    text_tokens = tokenize(text)
    title_tokens = tokenize(title)
    combined = text_tokens + [self.title_token] + title_tokens
    all_tokens.extend(combined)

```

2. Формирование словаря токенов (vocabulary):

После сбора всех токенов подсчитываются частоты слов, и формируется словарь модели. В него включаются только те слова, которые встретились более одного раза, что помогает уменьшить число редких элементов.

```

token_counts = Counter(all_tokens)
self.vocab = set(token for token, count in token_counts.items() if count > 1)
self.vocab.update([self.start_token, self.title_token, self.end_token,
    ↪ self.unknown_token])

```

Слова, отсутствующие в словаре, будут заменяться на специальный токен `<unk>`.

3. Построение моделей меньшего порядка:

Если основная модель использует n -граммы порядка $n > 1$, то автоматически создаются модели всех меньших порядков (1-граммная, 2-граммная и т.д.).

Они будут применяться для линейной интерполяции или backoff-алгоритмов.

```
if self.n_gram > 1:
    for n in range(1, self.n_gram):
        self.lower_order_models[n] = TitleNgramModel(n, self.smoothing, self.alpha)
        self.lower_order_models[n].train(pairs)
```

4. Подготовка данных и построение n -грамм:

На следующем этапе текст и заголовок повторно токенизируются, но теперь с учетом словаря: редкие слова заменяются `<unk>`.

```
text_tokens = [token if token in self.vocab else self.unknown_token for token in
    ↪ text_tokens]
title_tokens = [token if token in self.vocab else self.unknown_token for token in
    ↪ title_tokens]
combined = text_tokens + [self.title_token] + title_tokens
ngrams_list = self.create_ngrams(combined)
```

5. Подсчёт частот контекстов и слов:

Для каждой n -граммы фиксируется её контекст — первые $n-1$ слов — и следующее слово. Модель накапливает статистику, необходимую для расчёта вероятностей.

```
for ngram in ngrams_list:
    context = ngram[:-1]
    next_word = ngram[-1]
    self.ngrams[context][next_word] += 1
    self.context_counts[context] += 1
```

В результате модель получает полный набор частот, на которых далее строятся вероятности предсказания следующего слова.

Для удобства работы с датасетом реализован вспомогательный метод `train_from_csv`, который выполняет загрузку данных, их фильтрацию и передачу в основной тренировочный цикл модели. Метод обеспечивает удобную

интеграцию n-граммной модели с табличными корпусами.

На вход подаётся путь к CSV-файлу, содержащему пары (текст, название).

Файл читается в формате `pandas.DataFrame`:

```
df = pd.read_csv(csv_path)
if limit:
    df = df.head(limit)
```

Параметр `limit` позволяет ограничить размер обучающей выборки, что полезно на этапе отладки.

Далее осуществляется проход по строкам таблицы.

Используется индикатор выполнения `tqdm`, который позволяет отслеживать сколько осталось времени до полного прохода.

```
pairs = []
for _, row in tqdm(df.iterrows(), total=len(df), desc="Обучение модели"):
    text = getattr(row, text_col, None)
    title = getattr(row, title_col, None)
    if text and title:
        pairs.append((text, title))
```

В каждой строке извлекаются значения из столбцов `text` и `title` (их имена можно менять через параметры функции).

Если в строке отсутствует одно из полей, пара пропускается.

В результате формируется список, полностью соответствующий тому формату данных, который использует основной метод обучения `train`.

После подготовки пар выполняется вызов базового алгоритма обучения.

В n-граммной модели ключевым этапом является оценка условной вероятности появления следующего слова при заданном контексте. В реализации предусмотрено несколько вариантов вычисления вероятностей: без сглаживания, со сглаживанием Лапласа и с линейной интерполяцией. Каждый из них решает разные проблемы статистической модели.

1. Простая вероятностная оценка (без сглаживания):

```
def get_simple_probability(self, context: tuple, word: str) -> float:
    context_counts = self.ngrams.get(context, {})
    total = sum(context_counts.values())
    return context_counts[word] / total if total else 0.0
```

- Вероятность оценивается по формуле: $P(w_t | context) = \frac{C(context, w_t)}{C(context)}$ где
- $C(context, w_t)$ — число раз, когда после данного контекста появилось слово w_t ,
 - $C(context)$ — сколько раз этот контекст встречался в данных.

Особенности:

- Это самый точный вариант для часто встречающихся n-грамм.
- Главный недостаток — нулевые вероятности для комбинаций, которые не встречались в обучении. При генерации текста это приводит к «обрыву» предсказаний.

2. Сглаживание Лапласа (Add-one):

Для борьбы с нулевыми вероятностями используется метод Лапласовского сглаживания:

```
def get_laplace_probability(self, context: tuple, word: str) -> float:
    context_counts = self.ngrams.get(context, {})
    total = sum(context_counts.values())
    vocab_size = len(self.vocab)

    count_word = context_counts.get(word, 0)
    return (count_word + self.alpha) / (total + self.alpha * vocab_size)
```

Принцип метода:

Сглаживание Лапласа добавляет единицу к частоте каждого слова: $P(w_t | context) = \frac{C(context, w_t) + \alpha}{C(context) + \alpha \cdot |V|}$ где

- $\alpha = 1()$,
- $|V|$ — размер словаря.

3. Линейная интерполяция (с моделями меньшего порядка):

Метод `get_linear_interpolation_probability` улучшает качество модели за счёт комбинирования вероятностей разных порядков:

```
def get_linear_interpolation_probability(self, context: tuple, word: str) -> float:
    if self.n_gram == 1 or not self.lower_order_models:
        return self.get_laplace_probability(context, word)

    lambda_current = 0.6
    lambda_backoff = 0.4

    current_prob = self.get_laplace_probability(context, word)
```

```

backoff_context = context[1:] if len(context) > 1 else tuple()
backoff_model = self.lower_order_models[self.n_gram - 1]
backoff_prob = backoff_model.get_probability(backoff_context, word)

return lambda_current * current_prob + lambda_backoff * backoff_prob

```

Математическая формула: $P = \lambda P_n + (1 - \lambda) P_{n-1}$, где

- P_n — вероятность в модели порядка n ,
- P_{n-1} — вероятность в модели меньшего порядка ($n-1$),
- λ — вес старшей модели.

Так как редкие контексты приводят к недостоверным оценкам. Интерполяция позволяет «страховаться» моделями меньшего порядка, которые более устойчивы статистически.

4. Универсальный метод выбора вероятности:

Общий интерфейс `get_probability` выбирает метод в зависимости от параметров модели:

```

def get_probability(self, context: tuple, word: str) -> float:
    if word not in self.vocab:
        word = self.unknown_token

    if self.smoothing == 'laplace':
        return self.get_laplace_probability(context, word)
    elif self.smoothing == 'linear':
        return self.get_linear_interpolation_probability(context, word)
    elif self.smoothing is None:
        return self.get_simple_probability(context, word)
    else:
        return self.get_laplace_probability(context, word)

```

Далее идут методы для генерации названия.

Метод `generate_with_backoff` вычисляет распределение вероятностей слов для заданного контекста, используя стратегию постепенного уменьшения длины контекста.

Если полная n -грамма отсутствует в тренировочных данных, метод постепенно сокращает контекст, пока не найдёт подходящий.

Если подходящего контекста нет вовсе, используется равномерное распределение по словарю.

1. Инициализация контекста и предельной глубины отката:

Сначала фиксируется исходный контекст и максимальная допустимая глубина backoff-уменьшения.

Если глубина не указана вручную, она устанавливается равной $n - 1$, что позволяет модели последовательно пройти через все возможные контексты меньшего порядка.

```
if max_depth is None:
    max_depth = self.n_gram - 1
current_context = context
depth = 0
```

2. Проверка наличия контекста в модели:

На каждом шаге выполняется проверка: встречался ли текущий контекст в обучающих данных. Если да — можно вычислять вероятности слов, следующих за этим контекстом.

```
if current_context in self.ngrams:
    probabilities = dict()
```

3. Вычисление вероятностей слов для текущего контекста:

Для каждого слова из словаря, за исключением служебных токенов, вычисляется вероятность по выбранной стратегии сглаживания:

```
for word in self.vocab:
    if word not in [self.title_token, self.title_token, self.end_token]:
        probabilities[word] = self.get_probability(current_context, word)
```

4. Нормировка распределения:

После расчёта суммируются все вероятности. Если сумма положительна, выполняется нормировка, чтобы получить корректное вероятностное распределение, сумма которого равна 1.

```
total_prob = sum(probabilities.values())
if total_prob > 0:
    return {word: prob / total_prob for word, prob in probabilities.items()}
```

5. При отсутствии данных — уменьшение контекста (backoff):

Если ни один след за данным контекстом не зафиксирован, модель постепенно сокращает контекст, удаляя первый токен.

Процесс продолжается, пока:

- а) не будет найдено подходящее состояние;
- б) или не будет достигнута максимальная глубина отката;
- в) или контекст не станет однословным.

```

if len(current_context) > 1:
    current_context = current_context[1:]
else:
    break
depth += 1

```

6. Фоллбек: равномерное распределение по словарю:

Если ни один контекст (ни меньшего, ни полного порядка) не встречался в корпусе, применяется равномерное распределение по всем доступным словам словаря, за исключением служебных токенов.

```

words = [w for w in self.vocab if w not in [self.start_token, self.title_token,
↪ self.end_token]]
return {word: 1.0 / len(words) for word in words}

```

Метод `generate_title` выполняет автоматическое порождение заголовка на основе входного текста.

Он использует вероятностную модель n-грамм, backoff-алгоритм и выбранный метод сглаживания.

Процесс генерации включает несколько последовательных этапов.

1. Токенизация входного текста и нормализация токенов:

На первом шаге входная строка разбивается на токены. Каждый токен проверяется на принадлежность словарю модели: если токен отсутствует в словаре, он заменяется специальным маркером `<unk>`.

После текста добавляется маркер `<title>`, обозначающий начало заголовка.

```

tokens = tokenize(input_text)
tokens = [token if token in self.vocab else self.unknown_token for token in
↪ tokens]
tokens += [self.title_token]

```

2. Формирование исходного контекста:

Для старта генерации используется последние $n-1$ токенов исходной последовательности, куда входит маркер `<title>`.

Это обеспечивает корректную условность предсказываемого слова: $P(w_t | \text{context})$.

3. Итеративное предсказание следующего слова:

Генерация выполняется максимум в течение `max_words` шагов.

На каждом шаге рассчитывается вероятностное распределение следующих слов с помощью `backoff`-механизма.

```
word_probs = self.generate_with_backoff(context)
if not word_probs:
    break
```

4. Нормировка распределения:

`Backoff` возвращает ненормированное распределение.

Сначала вычисляется сумма вероятностей, затем каждое значение нормируется так, чтобы сумма равнялась 1.

```
word_probs = {word: prob for word, prob in word_probs.items()}
total = sum(word_probs.values())
word_probs = {word: prob / total for word, prob in word_probs.items()}
```

5. Стохастический выбор следующего слова:

Следующее слово выбирается случайно, пропорционально вероятностям.

Используется стандартная выборка с весами.

```
words = list(word_probs.keys())
probs = list(word_probs.values())
next_word = random.choices(words, weights=probs)[0]
```

Если модель сгенерировала маркеры `</s>` или `<title>`, процесс завершится — это признаки конца последовательности.

```
if next_word in {self.end_token, self.title_token}:
    break
```

6. Обновление контекста для следующего шага:

Добавленное слово включается в контекст, который смещается на один шаг вправо.

Таким образом, в каждый момент времени используется актуальная $n-1$ -грамма.

```
result.append(next_word)
context = (*context[1:], next_word) if len(context) > 0 else (next_word,)
```

7. Формирование итогового заголовка:

Полученные слова объединяются в строку. Если модель не смогла сгенерировать ни одного слова, возвращается fallback-значение «Без названия».

```
title = " ".join(result)
return title if title else "Без названия"
```

Для оценки качества работы модели, основанной на n-граммах, использовалась только одна метрика — METEOR, поскольку она наиболее подходит для оценки качества коротких текстов и учитывает как точные совпадения слов, так и семантическую близость.

Метод `evaluate_meteor` вычисляет среднее значение METEOR-метрики для заданного тестового набора.

Эта метрика позволяет количественно оценить качество генерации заголовков, сопоставляя с эталонными (reference) заголовками.

```
def evaluate_meteor(self, test_pairs: List[Tuple[str, str]]) -> float:
    """Вычисление средней METEOR-оценки для тестового набора (text,
    ↪ reference_title)."""
    scores = []
    for text, true_title in tqdm(test_pairs, desc="Оценка METEOR"):
        generated = self.generate_title(text)
        score = meteor_score([tokenize(true_title)], tokenize(generated))
        scores.append(score)
    return sum(scores) / len(scores) if scores else 0.0
```

Реализованный класс `TitleNgramModel` содержит методы `save` и `load`, которые позволяют сериализовать модель на диск и восстановить её из файла.

Для этого используется стандартный модуль Python — `pickle`.

```
def save(self, path: str):
    with open(path, "wb") as f:
        pickle.dump(self, f)

    @staticmethod
    def load(path: str):
        with open(path, "rb") as f:
            return pickle.load(f)
```

Для обучения n-граммной модели использовался заранее подготовленный набор данных, содержащий пары (текст + заголовок). Обучающая выборка была загружена из CSV-файла, после чего модель была обучена и сохранена для последующего использования.

Модель была создана с использованием триграмм ($n = 3$). Такой размер контекста позволяет учитывать два предыдущих слова при предсказании следующего.

После запуска процесса обучения был сформирован полный набор n-грамм и произведена оценка качества генерации на валидационном наборе.

В ходе обучения были получены следующие результаты:

1. Было обработано 821612 обучающих примеров.
2. По итогам обучения модель сформировала 11126709 уникальных контекстов.

После завершения обучения была проведена оценка качества генерации заголовков с использованием метрики METEOR на валидационном наборе объемом 5963 примера. В результате средняя METEOR оценка составила 0.

Это показывает, что сгенерированные моделью заголовки практически не совпадают с референсными. Это ожидаемо для простой статистической модели.

Для демонстрации работы модели были выполнены запросы на генерацию названий для нескольких произвольных текстов. Как видно из примеров, модель формирует последовательности слов, которые не связаны по смыслу с содержанием текста. Это подтверждает ограниченность n-граммного подхода в задачах семантической генерации.

Пример 1.

Исходный текст (фрагмент): «У меня большая семья из шести человек: я, мама, папа, старшая сестра, бабушка и дедушка. . . »

Сгенерированное название: «свиста ниппур апокалиптические каллиграфическими кастрированный благословение гвардейскую»

Пример 2.

Исходный текст (фрагмент): «Я с детства хотел завести собаку, но родители мне не разрешали. . . »

Сгенерированное название: «местечковая постройнела просыпали унижениями расчерченные направляютьтолкают многотысячными»

Пример 3.

Исходный текст (фрагмент): «Когда я окончил университет, то начал ходить по собеседованиям в разные компании. . . »

Сгенерированное название: «запястий эмбриологии залишилась крючьев сочиняющие айви зацветающих»

Сгенерированные заголовки почти полностью состоят из редких или случайных слов, часто морфологически несовместимых. У них отсутствуют: смысловые связи, тематическая релевантность, грамматическая структура.

Поведение модели объясняется тем, что n-граммная модель:

1. Работает только на поверхностных статистических закономерностях;
2. Не понимает смысла текста;
3. Формирует заголовки по принципу вероятностного продолжения последовательностей, встретившихся в корпусе.

3.2 Feedforward Neural Network Language Model (FNNLM)

3.2.1 Подготовка обучающей выборки и формирование словаря

Процесс подготовки данных является фундаментом для обучения нейросетевой языковой модели, так как именно на этом этапе осуществляется переход от неструктурированного текста к тензорным представлениям. Реализация препроцессинга сосредоточена в модуле `dataset.py` и включает механизмы балансировки, токенизации и формирования контекстных окон.

Для обеспечения корректной работы модели на начальном этапе выполняется нормализация текстовых данных и выравнивание распределения классов. Функция `get_balanced_sample` предназначена для предотвращения доминирования определенных источников или типов заголовков, что критично для предотвращения переобучения на статистических аномалиях выборки.

```
def get_balanced_sample(df, limit=100000):
    title_counts = df['title'].value_counts()
    max_per_title = max(1, limit // len(title_counts))
    sampled_dfs = []
    for title in title_counts.index:
```

```

title_df = df[df['title'] == title].head(max_per_title)
sampled_dfs.append(title_df)
result_df = pd.concat(sampled_dfs).sample(frac=1).reset_index(drop=True)
return result_df.head(limit)

```

Дополнительно применяется метод `first_n_sentences`, который ограничивает входной контекст первыми двумя предложениями текста.

Трансформация слов в числовые индексы требует создания словаря, учитывающего частотные характеристики корпуса. В классе `FNNLMDataset` реализована фильтрация редких лексем через параметр `MIN_WORD_FREQ`, что позволяет исключить опечатки и уникальные имена собственные, перегружающие веса выходного слоя.

```

word_counts = Counter(words_corpus)
filtered_words = sorted([w for w, freq in word_counts.items() if freq >=
    ↪ MIN_WORD_FREQ])

self.vocab = {word: i + 3 for i, word in enumerate(filtered_words)}
self.vocab['<PAD>'] = 0
self.vocab['<UNK>'] = 1
self.vocab['<END>'] = 2

```

Особое значение имеют зарезервированные токены: токен `<PAD>` используется для дополнения последовательностей до фиксированной длины, `<UNK>` служит для представления слов, не вошедших в основной словарь, а `<END>` является индикатором завершения генерации заголовка.

Центральным элементом подготовки данных является разбиение текстов на обучающие пары, состоящие из контекста фиксированной длины и целевого слова. Параметр `context_size` определяет размер рецептивного поля модели. В случае, если длина входной последовательности меньше заданного окна, применяется левостороннее дополнение токенами `<PAD>`.

```

if len(text_tokens) < context_size:
    text_tokens = ['<PAD>'] * (context_size - len(text_tokens)) + text_tokens

current_context = text_tokens
for target in title_tokens:
    input_ids = [self.vocab.get(w, self.vocab['<UNK>']) for w in current_context]
    target_id = self.vocab.get(target, self.vocab['<UNK>'])
    self.data.append((torch.tensor(input_ids, dtype=torch.long), target_id))

```

```
current_context = current_context[1:] + [target if target in self.vocab else
↪ '<UNK>']
```

Алгоритм итеративно проходит по токенам заголовка, на каждом шаге сдвигая окно на одну позицию вправо. Предсказанное на предыдущем шаге целевое слово становится частью нового контекста для генерации следующего элемента последовательности. Нейронная сеть обучается предсказывать условную вероятность следующего токена $P(w_t | w_{t-n}, \dots, w_{t-1})$, что соответствует теоретическим принципам N-граммных моделей, реализованных на базе нейронных сетей.

3.2.2 Архитектура нейросетевой модели FNNLM

Реализованная модель Feed-forward Neural Network Language Model (FNNLM) представляет собой многослойный перцептрон, оптимизированный для задачи предсказания следующего токена на основе фиксированного контекста.

Процесс обработки данных в методе forward начинается с трансформации входных индексов токенов в плотные векторные представления. Слой nn.Embedding сопоставляет каждому слову из словаря вектор размерности embedding_dim. В отличие от классических подходов, использующих конкатенацию векторов контекста, в данной работе применена агрегация путем усреднения:

```
embeds = self.embeddings(x)
pooled = torch.mean(embeds, dim=1)
pooled = self.dropout(pooled)
```

Использование операции torch.mean позволяет получить инвариантное к длине последовательности представление контекста, что снижает количество обучаемых параметров в последующих полносвязных слоях и способствует лучшей обобщающей способности модели. Для регуляризации на данном этапе применяется Dropout, предотвращающий коадаптацию нейронов.

Скрытый слой модели выполняет нелинейное преобразование агрегированного вектора контекста. Он состоит из линейного преобразования linear1

и функции активации гиперболического тангенса ($\tanh$), которая обеспечивает диапазон значений $[-1, 1]$ и помогает стабилизировать процесс обучения:

```
hidden = torch.tanh(self.linear1(pooled))  
hidden = self.dropout(hidden)
```

Финальный расчет логитов для классификации по всему словарю реализован через комбинацию выхода скрытого слоя и прямой связи от агрегированных эмбедингов. Математически это можно выразить формулой:

$$y = W_2 \cdot h + W_d \cdot p + b$$

где h — выход скрытого слоя, p — усредненный вектор контекста, а W_d — матрица весов слоя прямой связи (`self.direct`).

```
output = self.linear2(hidden) + self.direct(pooled)
```

Включение слоя `self.direct` является важным архитектурным решением. Данный механизм позволяет градиентам при обратном распространении течь напрямую к слою эмбедингов, минуя нелинейную функцию активации скрытого слоя. Это ускоряет сходимость модели и позволяет сети легче выучивать простые линейные зависимости между контекстными словами и целевым токеном, оставляя скрытому слою задачу моделирования более сложных семантических взаимодействий. Результатом работы функции является вектор логитов размерности `vocab_size`, который в дальнейшем используется для вычисления функции потерь или генерации текста.

3.2.3 Конфигурация и процедура обучения модели

Процесс обучения нейросетевой модели генерации заголовков регламентируется набором гиперпараметров, определенных в модуле `config.py`, и логикой итеративного обновления весов, реализованной в `utils.py`. Использование фиксированных конфигураций обеспечивает воспроизводимость экспериментов и стабильность сходимости градиентных методов.

Для достижения оптимального баланса между скоростью обучения и качеством аппроксимации выбраны следующие параметры: размер батча (BATCH_SIZE) составляет 32 при общем количестве эпох (EPOCHS), ограниченном 30. Процесс минимизации функции потерь осуществляется с использованием оптимизатора Adam при начальной скорости обучения (LEARNING_RATE) 10^{-4} . Для предотвращения переобучения, помимо слоя Dropout, применяется регуляризация весов (weight_decay) со значением 10^{-5} . Вычисления производятся на доступном графическом ускорителе (CUDA или MPS) либо на центральном процессоре, что определяется автоматически в зависимости от аппаратной конфигурации.

Центральным компонентом обучения является функция потерь CrossEntropyLoss. В данной работе её конфигурация включает два важных аспекта, специфичных для языкового моделирования:

```
criterion = nn.CrossEntropyLoss(ignore_index=pad_id, label_smoothing=0.1)
optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE,
    ↪ weight_decay=WEIGHT_DECAY)
```

Параметр ignore_index позволяет исключить влияние токенов заполнения ($< PAD >$) на вычисление градиента, фокусируя модель исключительно на значимых лексемах. Применение сглаживания меток (label_smoothing=0.1) предотвращает чрезмерную уверенность модели в предсказаниях (overconfidence), что способствует лучшей генерализации и помогает сети избегать локальных минимумов при работе с естественным языком.

Для оценки качества модели на каждом этапе обучения используется валидационная выборка. Основной метрикой выступает перплексия (Perplexity, PPL), которая математически интерпретируется как экспонента от средней функции потерь на кросс-энтропию:

$$PPL = e^{Loss_{avg}}$$

В коде расчет реализован следующим образом:

```
avg_val_loss = val_loss / val_samples
val_ppl = math.exp(avg_val_loss)
```

Данная метрика показывает, насколько уверенно модель предсказывает следующий токен: чем ниже значение PPL, тем лучше модель усвоила структуру и вероятностное распределение слов в обучающем корпусе.

С целью предотвращения переобучения и экономии вычислительных ресурсов в процедуру обучения интегрирован алгоритм ранней остановки. Система отслеживает динамику `val_loss` и сохраняет веса модели только в том случае, если текущий результат превосходит лучший достигнутый ранее.

```
if avg_val_loss < best_val_loss:
    best_val_loss = avg_val_loss
    counter = 0
    torch.save(model.state_dict(), save_path)
else:
    counter += 1
    if counter >= PATIENCE:
        print(f"Ранняя остановка на эпохе {epoch + 1}")
        break
```

Если значение потерь на валидации не улучшается в течение заданного количества эпох (`PATIENCE = 3`), процесс обучения прерывается. По завершении цикла обучения производится автоматическая загрузка наилучшей сохраненной версии модели (`best_fnnlm_model.pth`), что гарантирует использование наиболее эффективного состояния сети для дальнейшей генерации заголовков.

3.2.4 Алгоритм генерации текста и стратегии инференса

Завершающим этапом реализации системы является алгоритм инференса, трансформирующий вероятностные предсказания модели в связный текст заголовка. В модуле `utils.py` этот процесс инкапсулирован в функции `generate_title`, которая реализует авторегрессионный метод генерации с применением стохастических стратегий семплирования.

Процесс начинается с подготовки начального контекста. Входной текст подвергается нормализации и приводится к фиксированному размеру `CONTEXT_SIZE`.

Если исходный текст избыточен, извлекаются последние токены; если недостаточен — применяется левосторонний паддинг токенами $\langle PAD \rangle$. Это гарантирует соответствие входного вектора размерности, ожидаемой слоем эмбеддингов модели.

```
clean_input = re.sub(r'[\^\w\s]', ' ', input_text.lower()).split()
if len(clean_input) > CONTEXT_SIZE:
    current_context = clean_input[-CONTEXT_SIZE:]
else:
    current_context = ['<PAD>'] * (CONTEXT_SIZE - len(clean_input)) +
        ↪ clean_input
```

Основной цикл генерации выполняется итеративно до достижения лимита `max_words` или появления токена остановки $\langle END \rangle$. На каждой итерации модель вычисляет логиты для следующего слова, которые затем подвергаются модификации для управления качеством и вариативностью текста. Важным элементом является масштабирование логитов температурой (T):

$$logits_{scaled} = \frac{logits}{T}$$

Параметр `temperature` ($T = 0.7$) позволяет корректировать «уверенность» модели: низкие значения делают распределение более острым, отдавая предпочтение наиболее вероятным словам, в то время как высокие значения повышают креативность генерации за счет выравнивания вероятностей.

Для исключения маловероятных и потенциально некорректных токенов применяется стратегия `Top-k` семплирования. Из распределения выбираются только k наиболее вероятных кандидатов (параметр `top_k=5`), после чего вероятности пересчитываются через функцию `softmax` только для этого подмножества.

```
with torch.no_grad():
    logits = model(input_ids)
    logits /= temperature
    top_logits, top_indices = torch.topk(logits, top_k)
    probs = F.softmax(top_logits, dim=-1)
    idx_in_top = torch.multinomial(probs, num_samples=1).item()
    pred_id = top_indices[0, idx_in_top].item()
    pred_word = rev_vocab[pred_id]
```

Авторегрессионная природа алгоритма проявляется в обновлении контекста после каждой итерации: предсказанное слово добавляется в конец текущего окна, а самый старый токен удаляется. Таким образом, каждое новое слово генерируется с учетом как исходного текста статьи, так и уже сформированной части заголовка. Использование функции `torch.multinomial` в комбинации с Top-k фильтрацией позволяет модели генерировать разнообразные и грамматически корректные последовательности, избегая заикливания и тривиальных предсказаний.

3.2.5 Результаты и оценка качества генерации

В ходе эксперимента была проведена количественная оценка обученной модели FNNLM и качественный анализ сгенерированных ею заголовков. В отличие от статистических n-граммных моделей, нейросетевой подход на базе полносвязных слоев теоретически должен обеспечивать лучшую обобщающую способность за счет использования распределенных представлений слов (эмбеддингов).

Процесс обучения фиксировался с помощью функции потерь (Cross-Entropy Loss) на обучающей и валидационной выборках. Итоговые показатели представлены в таблице 1.

Таблица 1 – Метрики обучения модели FNNLM

Метрика	Значение
Функция потерь на обучении (Train Loss)	4.891
Функция потерь на валидации (Val Loss)	5.204
Перплексия на валидации (Val PPL)	182.0

Значение валидационной перплексии на уровне 182.0 указывает на то, что модель при предсказании каждого следующего слова в среднем выбирает из 182 равновероятных вариантов. Относительно небольшой разрыв между *Train Loss* и *Val Loss* свидетельствует об отсутствии критического переобучения, однако высокие абсолютные значения потерь указывают на сложность задачи генерации заголовков для данной архитектуры.

Для демонстрации работы модели были выбраны те же тестовые фрагменты текстов, что и для n-граммной модели. Результаты генерации представлены ниже.

Пример 1.

Исходный текст: «У меня большая семья из шести человек: я, мама, папа, старшая сестра, бабушка и дедушка. Мы живем все вместе с собакой Бимом и кошкой Муркой в большом доме в деревне...»

Результат FNNLM: (пустая строка)

Пример 2.

Исходный текст: «Я с детства хотел завести собаку, но родители мне не разрешали. Пока я был ребёнком, у меня жил хомяк Хома. Хома был очень маленький и пушистый...»

Результат FNNLM: (пустая строка)

Пример 3.

Исходный текст: «Когда я окончил университет, то начал ходить по собеседованиям в разные компании. Я учился на экономиста и хотел работать в финансовой компании...»

Результат FNNLM: (пустая строка)

Наблюдаемое поведение модели, при котором на выходе генерируется пустая строка, является классическим примером дегенерации нейросетевой языковой модели. Это обусловлено тем, что токен завершения последовательности ($< END >$) получил наибольшую вероятность на первом же шаге генерации.

Сравнительный анализ FNNLM с ранее рассмотренной n-граммной моделью позволяет сделать следующие выводы:

1. **Проблема редких слов:** В то время как n-граммная модель генерировала случайный набор редких и несвязных слов («запастий эмбриологии залишилась»), FNNLM ведет себя более консервативно. Из-за механизма сглаживания меток (*label smoothing*) и фильтрации словаря модель стремится минимизировать ошибку, отдавая предпочтение наиболее вероятным в

корпусе паттернам.

2. **Доминирование маркера конца:** В условиях высокой неопределенности (высокая перплексия) модель предсказывает токен $\langle END \rangle$ как наиболее безопасный вариант, который статистически часто встречается после коротких контекстов в обучающем датасете.
3. **Ограниченность архитектуры:** Полносвязная сеть (Feed-Forward) не способна эффективно моделировать длинные зависимости и семантическую структуру текста. Усреднение эмбеддингов контекста (mean pooling) приводит к потере информации о порядке слов, что делает невозможным построение логически обоснованного заголовка для сложного текста.

Таким образом, переход от простых n -грамм к базовой нейросетевой модели FNNLM изменил характер ошибок: модель перешла от генерации «семантического шума» к отказу от генерации из-за низкой уверенности в предсказании. Это подтверждает необходимость использования более сложных рекуррентных или трансформерных архитектур для решения задачи суммаризации текста.

3.3 Рекуррентная нейронная сеть (RNN)

3.3.1 Подготовка последовательностей и токенизация

Для перехода от сырого текстового представления лекций к тензорным структурам, пригодным для обработки рекуррентной нейронной сетью, реализован специализированный конвейер препроцессинга. В отличие от модели FNNLM, работающей с фиксированным «окном» контекста, архитектура RNN способна обрабатывать последовательности произвольной длины, что накладывает специфические требования на этап подготовки данных.

Логика обработки сосредоточена в модуле `dataset.py` и базируется на динамическом формировании словаря и выравнивании последовательностей.

Формирование словаря и фильтрация шума. Центральным компонентом является класс `Vocab`, инкапсулирующий логику построения отображения токенов в целочисленные индексы. На этапе инициализации выполняется то-

кенизация всего корпуса текстов с приведением к нижнему регистру. Для повышения обобщающей способности модели и сокращения размерности выходного слоя применяется механизм фильтрации редких лексем через параметр `min_freq`.

```
counter = Counter(tokens)
for token, freq in counter.items():
    if freq >= min_freq and token not in self.stoi:
        self.stoi[token] = len(self.itos)
        self.itos.append(token)
```

Слова, встречающиеся в обучающей выборке менее 5 раз, исключаются из словаря. Это позволяет минимизировать влияние «шума» (опечаток и уникальных имен), предотвращая переобучение сети на статистически незначимых данных.

Для управления процессом рекуррентной генерации в словарь интегрированы сервисные токены:

- `<pad>` — используется для дополнения последовательностей до одинаковой длины в рамках одного батча;
- `<sos>` (Start of Sequence) — маркер начала генерации, подаваемый на вход декодеру;
- `<eos>` (End of Sequence) — индикатор завершения предложения, позволяющий модели сигнализировать об окончании заголовка;
- `<unk>` — заменяет слова, не вошедшие в словарь из-за низкой частотности.

Класс `TitleDataset` осуществляет трансформацию строковых данных в тензоры типа `torch.long`. В методе `__getitem__` происходит формирование целевой последовательности заголовка, которая в обязательном порядке обрамляется управляющими токенами:

```
title_tensor = torch.tensor(
    [self.vocab.stoi[config.SOS_TOKEN]] +
    title_encoded +
    [self.vocab.stoi[config.EOS_TOKEN]],
    dtype=torch.long
)
```

Наличие маркеров начала и конца позволяет модели в процессе обучения выстраивать зависимости между скрытым состоянием текста лекции и семантическими границами заголовка.

Одной из ключевых особенностей работы с RNN является вариативность длины входных данных. В отличие от FNNLM, где контекст жестко ограничивался, здесь сохраняется полная структура предложений. Для эффективной параллельной обработки данные объединяются в батчи с помощью функции `collate_fn`.

Поскольку тензоры в батче должны иметь одинаковую размерность, используется метод `pad_sequence`, который дополняет последовательности токеном `<pad>` до длины самого длинного элемента в текущей итерации.

```
def collate_fn(batch, pad_idx):  
    lecs, titles = zip(*batch)  
    lecs_padded = pad_sequence(lecs, batch_first=True, padding_value=pad_idx)  
    titles_padded = pad_sequence(titles, batch_first=True, padding_value=pad_idx)  
    return lecs_padded, titles_padded
```

Параметр `batch_first=True` гарантирует, что первая размерность тензора соответствует размеру батча, что необходимо для корректной передачи данных в слой эмбедингов и рекуррентный блок `nn.RNN`.

3.3.2 Архитектура нейросетевой модели TitleRNN

Реализованная архитектура представляет собой многослойную рекуррентную нейронную сеть, построенную по принципу передачи контекстного вектора (скрытого состояния) от последовательности-источника к целевой последовательности. Модель оптимизирована для извлечения семантического вектора из текста лекции и его последующей дешифровки в связный заголовок.

Процесс обработки начинается с трансформации входных индексов токенов в плотные векторы фиксированной размерности `emb_dim`. Слой `nn.Embedding` позволяет сопоставить каждому слову из словаря обучаемый вектор, отражающий его семантическое значение. Для повышения устойчивости модели к переобучению к эмбедингам применяется `Dropout`.

```
self.embedding = nn.Embedding(vocab_size, emb_dim)
self.dropout = nn.Dropout(dropout)
```

Основным вычислительным модулем является слой `nn.RNN`. В отличие от однослойных моделей, использование параметра `n_layers` позволяет выстраивать иерархию скрытых представлений: нижние слои фокусируются на синтаксических структурах, а верхние — на более абстрактных концептах текста.

Ключевой особенностью реализации является роль скрытого состояния (h) как «памяти» сети. В методе `forward` этот механизм разделен на два этапа:

1. **Кодирование контекста:** входной текст пропускается через RNN. Полученное финальное скрытое состояние h аккумулирует в себе информацию о содержании всей лекции.
2. **Инициализация генерации:** вектор h используется в качестве начального скрытого состояния для той же RNN при обработке токенов заголовка.

Модель не просто предсказывает следующее слово, а делает это на основе «сжатого» смысла всей предыдущей последовательности.

```
# Извлечение контекста из текста
_, h = self.rnn(embedded_text)

# Использование контекста h для генерации заголовка
output, _ = self.rnn(embedded_title, h)
```

После рекуррентной обработки векторы скрытых состояний для каждого шага заголовка передаются в полносвязный слой `self.fc_out`. Он выполняет проекцию вектора из пространства скрытых признаков (`hid_dim`) обратно в размерность словаря (`vocab_size`), формируя логиты для классификации.

```
self.fc_out = nn.Linear(hid_dim, vocab_size)
```

Важную роль играет параметр `dropout`, интегрированный как в структуру самого блока RNN (между слоями), так и в качестве отдельного слоя после эмбедингов. Это позволяет случайным образом «выключать» часть нейронов

в процессе обучения, что предотвращает их чрезмерную коадаптацию и заставляет сеть выучивать более устойчивые признаки, снижая риск переобучения на специфических оборотах обучающей выборки.

3.3.3 Процесс обучения и механизм Teacher Forcing

Эффективность обучения рекуррентной модели в задаче генерации последовательностей напрямую зависит от стратегии подачи целевых данных и выбора критерия оптимизации. Процесс обучения базируется на методе принудительного обучения (Teacher Forcing) и адаптивном градиентном спуске.

В процессе обучения на каждой итерации модель получает два тензора: текст и соответствующий ему заголовок. Однако для корректного предсказания слов в режиме авторегрессии данные заголовка подаются со сдвигом:

```
# Передача в модель: текст и заголовок без последнего токена  
output = model(text, titles[:, :-1])
```

```
# Вычисление потерь: предсказания сравниваются с заголовком без первого токена  
loss = criterion(output.reshape(-1, output_dim), titles[:, 1:].reshape(-1))
```

Логика этого разделения заключается в следующем: чтобы предсказать первое слово заголовка, модели нужен токен `<sos>`. Чтобы предсказать второе — первые два токена, и так далее. Таким образом, вход модели (`titles[:, :-1]`) всегда на один шаг «отстает» от целевых меток (`titles[:, 1:]`), которые она должна сгенерировать.

Концепция Teacher Forcing заключается в использовании эталонных данных из обучающей выборки в качестве входных сигналов на каждом шаге рекуррентной сети, вместо использования собственных предсказаний модели с предыдущего шага.

В начале обучения предсказания сети неизбежно ошибочны. Если бы модель использовала свой собственный выход как вход для следующего шага (как это происходит при применении), одна ошибка в начале предложения привела бы к накоплению искажений по всей цепочке, делая обучение крайне неста-

бильным. Использование Teacher Forcing позволяет «корректировать» модель на каждом шаге, направляя её по правильной траектории градиентного спуска.

Для задачи выбрана кросс-энтропия (CrossEntropyLoss). Особенностью реализации является использование параметра ignore_index:

```
criterion = nn.CrossEntropyLoss(ignore_index=pad_idx)
```

Это позволяет исключить влияние токенов заполнения (<pad>) на расчет градиентов. Поскольку эти токены не несут семантической нагрузки и добавлены лишь для выравнивания длин в батче, модель не должна штрафоваться за их «предсказание», что делает метрику потерь более точной.

Для обновления весов используется алгоритм Adam с начальной скоростью обучения 10^{-3} . Выбор обусловлен способностью метода адаптировать шаг обучения для каждого параметра индивидуально, что критично для RNN из-за риска затухания или взрыва градиентов.

Для предотвращения переобучения и оптимизации времени вычислений внедрен механизм ранней остановки, управляемый параметрами best_loss и PATIENT.

```
if val_loss < best_loss:
    best_loss = val_loss
    epoch_no_improve = 0
    torch.save(...) # Сохранение лучшего состояния
else:
    epoch_no_improve += 1

if epoch_no_improve == config.PATIENT:
    # Остановка, если валидация не улучшается заданное число эпох
    break
```

Система отслеживает значение потерь на валидационной выборке. Если в течение PATIENT эпох не наблюдается улучшения val_loss, процесс прерывается. Это гарантирует, что итоговая модель сохранит наилучшую обобщающую способность до того, как начнется подгонка под специфику обучающего набора.

3.3.4 Алгоритм авторегрессионной генерации и стратегия инференса

Завершающим этапом реализации системы является алгоритм инференса, инкапсулированный в функции `generate_title`. В отличие от этапа обучения, где используется метод принудительного обучения, на этапе предсказания модель работает в авторегрессионном режиме, когда каждый последующий токен генерируется на основе ранее предсказанных данных.

Процесс начинается с трансформации исходного текста лекции в семантический вектор. Входная строка токенизируется и кодируется с помощью словаря, после чего пропускается через слой эмбедингов и рекуррентный блок модели. На данном этапе основной целью является получение финального скрытого состояния h , которое служит компактным представлением всего входного контекста.

```
# Получение контекстного вектора h из текста  
_, h = model.rnn(model.embedding(src_tensor))
```

Вектор h аккумулирует информацию, необходимую для генерации заголовка, и передается в качестве начального состояния для первого шага декодирования.

Процесс построения заголовка происходит итеративно. В качестве начального входного значения используется служебный токен начала последовательности `<sos>`. На каждой итерации цикла выполняются следующие действия:

1. Модель принимает текущий токен (на первом шаге — `<sos>`) и актуальное скрытое состояние h .
2. Рекуррентный слой вычисляет новое скрытое состояние и вектор логитов.
3. С помощью операции `argmax` выбирается индекс токена с максимальной вероятностью.
4. Выбранный токен преобразуется обратно в слово через `vocab.itos` и добавляется в результирующий список.
5. Индекс предсказанного слова становится входным значением для следующей итерации цикла.

```

# Итеративный процесс генерации
for _ in range(max_len):
    input_tensor = torch.LongTensor([curr_idx]).to(device)
    output, h = model.rnn(model.embedding(input_tensor), h)

    # Выбор наиболее вероятного следующего слова
    next_word_idx = output.argmax(2).item()
    ...
    curr_idx = next_word_idx

```

Такой подход позволяет модели сохранять связность текста, так как каждое новое слово выбирается с учетом всей предыстории генерации, хранящейся в обновляемом векторе h .

Работа алгоритма прекращается при наступлении одного из двух событий:

- **Маркер завершения:** Если модель предсказывает токен `<eos>`, это сигнализирует о логическом завершении заголовка, и цикл прерывается.
- **Лимит длины:** Для предотвращения заикливания и генерации избыточно длинных последовательностей установлено ограничение `max_len`.

Итоговая последовательность токенов объединяется в строку, формируя готовый заголовок к лекции. Данный алгоритм обеспечивает баланс между точностью передачи смысла исходного текста и грамматической корректностью выходной последовательности.

3.3.5 Результаты и оценка качества генерации

Основной целью этапа был анализ того, насколько последовательная обработка данных и сохранение скрытого состояния позволяют улучшить качество генерации заголовков по сравнению с полносвязной моделью (FNNLM).

Итоговые показатели функции потерь на обучающей и валидационной выборках представлены в таблице 2.

Таблица 2 – Метрики обучения модели RNN

Метрика	Значение
Функция потерь на обучении (Train Loss)	2.8660
Функция потерь на валидации (Val Loss)	10.5666

Анализ метрик указывает на наличие выраженного переобучения: значение *Train Loss* значительно ниже, чем у модели FNNLM, что говорит о способности RNN успешно аппроксимировать зависимости внутри обучающего набора. Однако высокий показатель *Val Loss* свидетельствует о плохой обобщающей способности — модель «запоминает» последовательности из тренировочного датасета, но не может корректно применить выученные паттерны к новым текстам.

Переход от архитектуры FNNLM к RNN продемонстрировал качественное изменение в поведении модели:

1. **Учет порядка слов:** в отличие от FNNLM, которая усредняла эмбединги и полностью теряла информацию о структуре предложения, RNN обрабатывает токены последовательно. Это позволяет вектору скрытого состояния h сохранять синтаксические связи.
2. **Преодоление дегенерации генерации:** модель FNNLM во всех тестовых примерах выдавала пустую строку, так как маркер конца последовательности ($\langle \text{eos} \rangle$) получал максимальную вероятность сразу. RNN, благодаря рекуррентной связи, «удерживает» контекст дольше и генерирует цепочки слов.
3. **Проблема «галлюцинаций»:** несмотря на то, что RNN начала генерировать текст, его семантическая связность остается низкой. Модель склонна к генерации «словесного салата», используя частотные слова в случайном порядке.

Ниже приведены результаты работы модели на тестовых фрагментах текстов, которые использовались для тестирования предыдущих моделей:

Пример 1.

Исходный текст: «У меня большая семья из шести человек: я, мама, папа, старшая сестра, бабушка и дедушка. Мы живем все вместе с собакой Бимом и кошкой Муркой в большом доме в деревне...»

Результат RNN: «где мыслимые джека захватывающий детективный сло-

ва обещала интриги точнее под слова сегодня точнее римским где»

Пример 2.

Исходный текст: «Я с детства хотел завести собаку, но родители мне не разрешали. Пока я был ребёнком, у меня жил хомяк Хома. Хома был очень маленький и пушистый...»

Результат RNN: «где один первый сегодня сегодня точнее где от объёме бы произведение обещала от обман под»

Пример 3.

Исходный текст: «Когда я окончил университет, то начал ходить по собеседованиям в разные компании. Я учился на экономиста и хотел работать в финансовой компании...»

Результат RNN: «где при дело сегодня от обман под где роман обычном дно не объёме мыслимые точнее»

Несмотря на теоретическое преимущество в виде способности моделировать длинные зависимости, классическая RNN сталкивается с проблемой затухающих градиентов и высокой сложности пространства поиска в задаче суммаризации.

Результаты показывают, что модель перешла от «молчания» (как в FNNLM) к генерации избыточного набора токенов, которые грамматически не согласованы между собой и не отражают суть текста. Это подтверждает, что простая рекуррентность недостаточна для формирования заголовков, что указывает на необходимость использования более продвинутых механизмов, таких как блоки памяти (LSTM).

3.4 Реализация модели на базе архитектуры LSTM

3.4.1 Подготовка данных и структурирование последовательностей

Для реализации модели на базе архитектуры LSTM был разработан конвейер обработки данных. Основная идея реализации — переход от отдельной обработки текста и заголовка к формату «Text-to-Title» в рамках одной последо-

вательности. Это позволяет модели использовать текст лекции как долгосрочный контекст для генерации.

Для снижения размерности и удаления шума используется параметр `min_freq=10`. Также в словарь добавлены специальные токены, такие же как в RNN.

```
class TitleVocab:
    def __init__(self, df, min_freq=10):
        self.itos = {0: "<PAD>", 1: "<SOS>", 2: "<EOS>", 3: "<UNK>", 4: "<SEP>"}
        self.stoi = {v: k for k, v in self.itos.items()}

        words = []
        for col in ['text', 'title']:
            for text in df[col].dropna():
                words.extend(str(text).lower().split())

        counts = Counter(words)
        for word, freq in counts.items():
            if freq >= min_freq and word not in self.stoi:
                idx = len(self.itos)
                self.stoi[word] = idx
                self.itos[idx] = word
```

В классе `LSTMTitleDataset` реализована логика склейки: текст ограничивается (первые 80 токенов) и соединяется с заголовком через разделитель. Итоговая структура имеет вид:

$$[SOS] + \text{текст} + [SEP] + \text{заголовок} + [EOS]$$

Модель учится предсказывать заголовок, имея текст в качестве «левого» контекста.

```
full_seq = [vocab.stoi["<SOS>"]] + text_tokens + [vocab.stoi["<SEP>"]] + \
            title_tokens + [vocab.stoi["<EOS>"]]
```

Ключевая особенность реализации — использование тензора маски (`mask`). Поскольку нам не нужно, чтобы модель обучалась восстанавливать исходный текст, мы зануляем функцию потерь для всех токенов, идущих до разделителя `<SEP>`.

```
# Находим индекс разделителя
```

```
sep_idx = tokens.index(self.vocab.stoi["<SEP>"])

# Создаем маску: 0 для текста, 1 для заголовка
mask = [0] * len(input_seq)
for i in range(sep_idx, len(mask)):
    mask[i] = 1
```

Маска представляет собой вектор той же длины, что и входная последовательность, где значения 1 соответствуют токенам заголовка, а 0 — всему остальному. В процессе обучения это позволяет реализовать «взвешенную» кросс-энтропию:

$$\mathcal{L}_{final} = \frac{\sum (\mathcal{L}_{raw} \cdot Mask)}{\sum Mask}$$

Благодаря маске, градиенты вычисляются только на основе предсказаний слов заголовка. Это заставляет LSTM фокусировать все вычислительные ресурсы на генерации, используя текст исключительно как информационный фундамент.

3.4.2 Архитектура нейросетевой модели LSTMTitleGen

За счет использования системы «ворот», LSTM способна сохранять информацию в долгосрочной памяти, что критично для задачи генерации заголовка, где важно удерживать контекст начала лекции до самого конца последовательности.

Входные индексы токенов преобразуются в плотные векторы через слой `nn.Embedding`. Основным вычислительным блоком является двунаправленная LSTM (`bidirectional=True`). В отличие от стандартной RNN, такая конфигурация обрабатывает последовательность одновременно в прямом и обратном направлениях. Это позволяет каждому состоянию учитывать не только предшествующий, но и последующий контекст, формируя более глубокое семантическое представление текста лекции.

```
self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
```

```

self.lstm = nn.LSTM(
    embed_dim,
    hidden_dim,
    num_layers=num_layers,
    batch_first=True,
    dropout=dropout if num_layers > 1 else 0,
    bidirectional=True
)

```

Для повышения стабильности обучения и борьбы с переобучением, которое было выявлено на этапе тестирования простой RNN, в архитектуру внедрены два механизма:

1. **Layer Normalization** (`nn.LayerNorm`): Нормализует выходы LSTM, что ускоряет сходимость и делает модель менее чувствительной к масштабу весов.
2. **Dropout**: Случайное зануление нейронов предотвращает чрезмерную подгонку под специфические обороты обучающей выборки.

```

self.ln = nn.LayerNorm(hidden_dim * 2)
self.dropout = nn.Dropout(dropout)

```

Поскольку модель является двунаправленной, размерность скрытого состояния на выходе удваивается (`hidden_dim * 2`). Полносвязный линейный слой (`nn.Linear`) выполняет финальную проекцию этого вектора обратно в размерность словаря (`vocab_size`), формируя логиты для предсказания следующего токена.

```

self.fc = nn.Linear(hidden_dim * 2, vocab_size)

def forward(self, x, hidden=None):
    x = self.embedding(x)
    out, hidden = self.lstm(x, hidden)

    # Применение нормализации и регуляризации
    out = self.ln(out)
    out = self.dropout(out)

    logits = self.fc(out)
    return logits, hidden

```


Итоговая архитектура сочетает в себе способность улавливать долгосрочные зависимости с механизмами стабилизации, что позволяет генерировать более грамматически верные и контекстуально обусловленные заголовки.

3.4.3 Процесс обучения

Процесс начинается с инициализации загрузчиков данных `DataLoader`, которые обеспечивают перемешивание обучающей выборки (`shuffle=True`) и параллельную подачу батчей. На каждой итерации модель получает тензор входных индексов, целевой тензор (сдвинутый на один шаг вперед) и тензор маски.

Стандартная функция кросс-энтропии вычисляет ошибку для всех токенов последовательности. В реализации текст служит лишь контекстом, и его предсказание не является целью обучения. Для этого используется параметр `reduction='none'`, который возвращает вектор потерь для каждого токена отдельно, без усреднения.

Математически это выражается формулой взвешенной средней ошибки:

$$\mathcal{L}_{masked} = \frac{\sum_{i=1}^N (L_i \cdot m_i)}{\sum_{i=1}^N m_i + \epsilon}$$

где L_i — значение `CrossEntropy` для i -го токена, $m_i \in \{0, 1\}$ — значение маски, а $\epsilon = 10^{-9}$ — малая величина для предотвращения деления на ноль.

```
# Вычисление сырых потерь без усреднения
raw_loss = criterion(logits.view(-1, len(vocab)), targets.view(-1))

# Применение маски: зануление потерь на тексте и падингах
masked_loss = raw_loss * masks.view(-1)

# Финальное усреднение только по релевантным токенам (заголовку)
loss = masked_loss.sum() / (masks.sum() + 1e-9)
```

Такой подход гарантирует, что градиенты обновляют веса модели только на основе того, насколько успешно она генерирует заголовок, игнорируя «заучивание» входного текста.

Для обновления весов используется оптимизатор Adam (`torch.optim.Adam`). Выбор обусловлен его способностью адаптировать скорость обучения для каждого параметра индивидуально.

Одной из главных проблем рекуррентных сетей является риск быстрого переобучения. Для его нейтрализации внедрен алгоритм Early Stopping с использованием параметра PATIENCE.

```
if avg_val_loss < best_loss:
    best_loss = avg_val_loss
    epoch_no_improve = 0
    # Сохранение лучшего состояния весов и словаря
    torch.save({'model_state': model.state_dict(), 'vocab': vocab},
               config.MODEL_SAVE_PATH)
else:
    epoch_no_improve += 1

if epoch_no_improve >= config.PATIENCE:
    # Прерывание обучения при отсутствии улучшений на валидации
    break
```

Модель отслеживает значение функции потерь на валидационной выборке. Если в течение `config.PATIENCE` эпох `val_loss` не демонстрирует снижения, процесс обучения прерывается. Это позволяет сохранить веса модели в точке их наилучшей обобщающей способности, не допуская деградации качества на новых данных.

3.4.4 Стохастическая генерация

Завершающим этапом реализации системы является алгоритм инференса. На этапе предсказания модель переходит из режима параллельной обработки батчей в авторегрессионный режим, где каждое следующее слово выбирается на основе накопленного контекста и предыдущих предсказаний.

Процесс начинается с формирования стартового вектора. Модель получает на вход последовательность, состоящую из токена начала (`<SOS>`), токенизированного текста и разделителя (`<SEP>`).

```
input_ids = [vocab.stoi["<SOS>"]] + tokens + [vocab.stoi["<SEP>"]]
input_tensor = torch.tensor([input_ids]).to(device)
```

В цикле генерации на каждой итерации модель анализирует всю текущую цепочку токенов. После получения логитов для последнего элемента, предсказанный индекс добавляется в конец `input_tensor` с помощью `torch.cat`. Так, база знаний модели о собственном ответе растет с каждым шагом, что позволяет сохранять грамматическую связность.

Вместо использования жадного поиска, который выбирает слово с максимальной вероятностью через `argmax`, здесь применен метод температурного сэмплирования. Это позволяет избежать «роботизированных», предсказуемых и заикленных заголовков.

Математически это реализуется через модификацию функции Softmax:

$$P(x_i) = \frac{\exp(L_i/T)}{\sum_j \exp(L_j/T)}$$

где L_i — логиты на выходе модели, а T — температура.

```
# Деление на температуру 0.4 делает распределение более «острым»
probs = torch.softmax(last_token_logits / 0.4, dim=-1)

# Стохастический выбор токена на основе распределения вероятностей
next_token = torch.multinomial(probs, num_samples=1).item()
```

Температура 0.4 является компромиссной: она достаточно низка, чтобы модель не генерировала случайный набор слов, но при этом оставляет место для вариативности, что критично для творческой задачи суммаризации. Использование `torch.multinomial` гарантирует, что модель будет выбирать слова пропорционально их вероятностям, делая текст более «живым».

Для предотвращения бесконечной генерации и заикливания предусмотрены два механизма выхода из цикла:

- **Семантический стоп-сигнал:** если модель предсказывает токен `<EOS>`, это означает, что заголовок считается завершенным.
- **Жесткое ограничение:** параметр `max_gen_len` ограничивает длину заголовка (20 токенами), чтобы результат оставался лаконичным.

```

if next_token == vocab.stoi["<EOS>"]:
    break

generated.append(next_token)

```

После выхода из цикла список индексов передается в метод `decode`, который отфильтровывает технические токены и возвращает финальную строку — готовый заголовок.

3.4.5 Результаты и оценка качества генерации

Завершающим этапом исследования является оценка эффективности обученной LSTM-модели.

В таблице 3 представлены итоговые метрики модели LSTM после завершения процесса обучения.

Таблица 3 – Метрики обучения и валидации LSTM

Метрика	Значение
Функция потерь на обучении (Train Loss)	0.0317
Функция потерь на валидации (Val Loss)	0.0131
Перплексия на валидации (Val PPL)	1.01

Анализ метрик показывает высокую степень сходимости модели. Значение перплексии ($PPL \approx 1.01$) свидетельствует о том, что модель крайне уверена в своих предсказаниях на валидационной выборке. Примечательно, что *Val Loss* оказался ниже *Train Loss* — это обусловлено использованием механизмов регуляризации (Dropout, LayerNorm) в процессе обучения и эффективной работой маскирования, которая сфокусировала обучение исключительно на коротких последовательностях заголовков.

Сравнение результатов генерации демонстрирует качественный скачок при переходе к LSTM:

- **Тематическая привязка:** В отличие от базовой RNN, которая генерировала «словесный салат» из случайных предлогов и существительных, LSTM начала выделять устойчивые паттерны (например, структуры «Топ 10», «искусственный интеллект»).

внедрение механизма внимания и использования seq2seq архитектуры.

3.5 Seq2seq

3.5.1 Подготовка данных и токенизация

Эффективность модели Sequence-to-Sequence (Seq2Seq) напрямую зависит от предварительной обработки текста и того, как именно слова преобразуются в числовые векторы.

В данной реализации вместо стандартного разбиения по пробелам (`.split()`) использована библиотека `tiktoken` с кодировкой `cl100k_base`. Это решение обосновано использованием алгоритма Byte Pair Encoding (BPE).

Преимущества перед простым разбиением:

- **Работа с редкими словами:** В отличие от word-based токенизаторов, которые заменяют любое незнакомое слово на токен `<unk>`, BPE разбивает редкие слова на субъединицы. Это позволяет модели «собирать» смысл незнакомых слов из знакомых частей.
- **Эффективность словаря:** Использование BPE позволяет сохранять умеренный размер словаря при полном покрытии языка, что снижает нагрузку на вычислительные ресурсы.
- **Обработка морфологии:** Токенизатор корректно обрабатывает регистр и знаки препинания, что критично для генерации читаемых заголовков.

```
encoding = tiktoken.get_encoding("cl100k_base")

def simple_tokenize(text):
    """Разбивает текст на токены с использованием BPE cl100k_base"""
    tokens_ids = encoding.encode(text.lower())
    return [encoding.decode([t]) for t in tokens_ids]
```

Класс `Vocab` отвечает за сопоставление уникальных токенов с их индексами. При инициализации в словарь принудительно добавляются четыре служебных токена, тех же, которые использовались ранее в других архитектурах.

Для ограничения размера модели используется параметр `MAX_VOCAB_SIZE`. В словарь попадают только наиболее частотные токены,

что отсекает шум и ускоряет сходимость.

Класс `TitleDataset` реализует преобразование сырого текста в тензоры. Ключевым методом является `text_to_indices`, который выполняет токенизацию, обрезку по максимальной длине (`max_len`) и добавление спецтокенов.

Важной особенностью реализации является выборочное использование маркеров `<sos>` и `<eos>`:

1. **Для исходного текста (source):** Эти токены не добавляются. Энкодер считает текст как единый контекст, и для его понимания модели достаточно самой последовательности токенов.
2. **Для заголовка (target):** Токены добавляются обязательно. `<sos>` необходим декодеру как «пусковой механизм» для начала генерации первого слова, а `<eos>` служит сигналом того, что генерация завершена и модель должна остановиться.

```
def text_to_indices(self, text, max_len, add_sos_eos=True):
    tokens = simple_tokenize(str(text))
    if len(tokens) > max_len:
        tokens = tokens[:max_len]

    indices = [self.vocab.word2index.get(t, 1) for t in tokens]

    if add_sos_eos:
        # Добавляем [2] (<sos>) в начало и [3] (<eos>) в конец
        indices = [2] + indices + [3]

    return torch.tensor(indices, dtype=torch.long)
```

Поскольку тексты и заголовки имеют разную длину, их нельзя напрямую объединить в прямоугольный тензор для вычислений на GPU. Для решения этой проблемы используется функция `collate_fn`, которая применяется к каждому батчу в `DataLoader`.

Функция использует `pad_sequence`, которая дополняет все последовательности в батче нулевыми значениями (токен `<pad>`) до длины самого длинного элемента. Параметр `batch_first=True` устанавливает размерность тензоров как $(Batch \times Seq \times Length)$, что является стандартом для обработки в современных нейросетях. Это позволяет минимизировать пустые вычисления.

```
def collate_fn(batch):
    """Динамическое выравнивание батча по максимальной длине"""
    src_batch, trg_batch = zip(*batch)

    # Дополнение паддингами (value=0)
    src_padded = pad_sequence(src_batch, padding_value=0, batch_first=True)
    trg_padded = pad_sequence(trg_batch, padding_value=0, batch_first=True)

    return src_padded, trg_padded
```

3.5.2 Компоненты нейросетевой модели

Архитектура энкодера и «сжатие» контекста

Энкодер отвечает за построение информативного представления входного текста. Реализован двунаправленный энкодер на базе двунаправленной LSTM.

```
self.lstm = nn.LSTM(emb_dim, hidden_dim, batch_first=True, bidirectional=True)
```

Использование двунаправленной архитектуры позволяет каждому токenu «видеть» контекст как слева, так и справа, что значительно улучшает семантическое понимание текста лекции. Однако декодер является однонаправленным, поэтому возникает задача согласования размерностей.

Скрытые состояния и состояния ячейки последнего временного шага для прямого и обратного проходов конкатенируются. Чтобы привести результирующий вектор размерности $\text{hidden_dim} * 2$ обратно к hidden_dim , используются полносвязные слои с активацией $\tanh$.

```
hidden = torch.tanh(self.fc_hidden(torch.cat((hidden[-2,:], hidden[-1,:]), dim=1)))
cell = torch.tanh(self.fc_cell(torch.cat((cell[-2,:], cell[-1,:]), dim=1)))
```

Полученные векторы служат начальным состоянием для декодера, передавая ему сжатую информацию о всей входной последовательности.

Механизм внимания позволяет декодеру на каждом шаге генерации «подсматривать» на конкретные части входного текста, а не полагаться только на один вектор контекста. Было реализовано аддитивное внимание (Бадану):

1. На вход подается текущее состояние декодера и все выходы энкодера.

2. Вычисляется «энергия» взаимодействия через обучаемый линейный слой:

$$E = \tanh(W \cdot [h_{dec}; h_{enc_all}])$$

3. С помощью функции Softmax вычисляются веса внимания, сумма которых равна 1. Они определяют важность каждого слова входа для генерации текущего слова выхода.

```
energy = torch.tanh(self.attn(torch.cat((hidden_repeated, encoder_outputs), dim=2)))
attention_weights = F.softmax(self.v.energy).squeeze(2), dim=1)
```

Итогом работы модуля является контекстный вектор — взвешенная сумма выходов энкодера, которая передается в декодер.

Декодер генерирует заголовок по одному токenu за шаг. Его архитектура спроектирована так, чтобы максимально эффективно использовать данные от механизма внимания.

На каждом шаге в LSTM подается не только эмбединг текущего токена, но и контекстный вектор, полученный из модуля внимания. Это гарантирует, что на каждом этапе рекуррентный слой «помнит», на какую часть исходного текста он сейчас сфокусирован.

```
rnn_input = torch.cat((embedded, context), dim=2)
output, (hidden, cell) = self.lstm(rnn_input, (hidden, cell))
```

Для предсказания вероятностей следующего слова используется полно-связный слой `fc_out`. В него подается максимально полный набор данных:

1. Текущий выход LSTM;
2. Контекстный вектор от внимания;
3. Эмбединг текущего токена.

Такая конкатенация (`hidden_dim * 3 + emb_dim`) реализует принцип «глубокого проброса связей», позволяя модели учитывать как долгосрочную память, так и мгновенный контекст внимания и текущее слово.

```
prediction = self.fc_out(torch.cat((output, context, embedded), dim=2).squeeze(1))
```

3.5.3 Класс Seq2Seq

Класс Seq2Seq является связующим звеном, координирующим работу энкодера и декодера. Его основная задача — передать контекст из входной последовательности в процесс генерации выходной последовательности.

В отличие от стандартных слоев нейросетей, Seq2Seq работает итеративно. Сначала энкодер обрабатывает всю входную последовательность целиком, возвращая финальные скрытые состояния и выходы каждого шага. Затем управление переходит к циклу, который выполняется `trg_len` раз:

- На каждом шаге t декодер принимает текущий токен, предыдущие состояния и контекст энкодера.
- Результат предсказания сохраняется в тензор `outputs`.
- Выбирается следующий токен для подачи на вход декодеру на шаге $t+1$.

Одной из главных сложностей обучения RNN является зависимость каждого шага от правильности предыдущего. Если на старте обучения модель ошибется в первом слове, вся дальнейшая генерация потеряет смысл, что замедлит сходимость. Для решения этой проблемы использован механизм Teacher Forcing.

```
teacher_force = random.random() < teacher_forcing_ratio
top1 = output.argmax(1)
input_tok = trg[:, t] if teacher_force else top1
```

Суть метода заключается в том, что с вероятностью `teacher_forcing_ratio` модель получает на вход не свое собственное предыдущее предсказание, а «правильное» слово из обучающей выборки.

- **При высоком ratio:** Модель обучается быстрее и стабильнее, так как всегда опирается на корректный контекст.
- **При низком ratio:** Модель приучается исправлять собственные ошибки и становится более автономной, что важно для режима инференса, где правильных ответов нет.

Установлено значение 0.5, что обеспечивает оптимальный баланс между скоростью обучения и устойчивостью модели.

3.5.4 Конфигурация и оптимизация процесса обучения

Процесс обучения реализован в функции `train_epoch`, которая включает в себя несколько критически важных механизмов стабилизации.

Для вычисления ошибки используется `CrossEntropyLoss`. Поскольку все последовательности в батче дополнены токенами `<pad>` до максимальной длины, расчет потерь на этих токенах искажил бы градиенты (модель обучалась бы «предсказывать пустоту»). Параметр `ignore_index=PAD_IDX` заставляет функцию потерь игнорировать элементы с индексом паддинга.

```
# Подготовка тензоров для CrossEntropy (выравнивание в один вектор)
outputs = outputs[:, 1:].reshape(-1, out_dim)
trg_flat = trg[:, 1:].reshape(-1)
loss = criterion(outputs, trg_flat)
```

В качестве оптимизатора выбран `AdamW` — модификация классического `Adam` с исправленным механизмом затухания весов. Это позволяет эффективнее бороться с переобучением. Для динамической настройки скорости обучения применен `ReduceLROnPlateau`. Он отслеживает `val_loss` и, если ошибка не уменьшается в течение нескольких эпох, снижает LR вдвое, позволяя модели более точно «доучиться» в локальном минимуме.

Рекуррентные сети подвержены проблеме «взрывающихся градиентов», когда в процессе обратного распространения ошибки значения производных становятся экстремально большими, что приводит к расходимости модели. Метод `clip_grad_norm_` ограничивает норму градиента значением `config.CLIP`, предотвращая резкие скачки весов.

```
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), clip)
optimizer.step()
```

3.5.5 Алгоритм инференса

После завершения обучения модель переходит в режим инференса, где основной задачей является генерация наиболее вероятной последовательности слов для заголовка. В данной реализации был осуществлен переход от простейшего жадного поиска к более продвинутому алгоритму — Beam Search.

Жадный поиск на каждом шаге выбирает токен с максимальной вероятностью. Однако такой подход имеет существенный недостаток: локально оптимальный выбор на шаге t может привести к крайне маловероятной последовательности на шагах $t + 1, t + 2$ и далее. Если модель совершает небольшую ошибку в начале предложения, она не может «вернуться назад» и исправить её. Beam Search решает эту проблему, рассматривая несколько наиболее вероятных гипотез параллельно.

Основным параметром алгоритма является ширина луча. Вместо одной последовательности модель хранит K наиболее перспективных вариантов.

1. **Логарифмические вероятности:** чтобы избежать проблемы исчезающих градиентов и численной нестабильности при перемножении малых вероятностей, алгоритм работает с логарифмами вероятностей:

$$\log P(w_1, \dots, w_t) = \sum_{i=1}^t \log P(w_i | w_1, \dots, w_{i-1})$$

Суммирование логарифмов позволяет эффективно сравнивать «стоимость» разных путей.

2. **Хранение гипотез:** в коде каждая гипотеза представлена кортежем, содержащим текущий суммарный скор, список накопленных токенов и скрытые состояния LSTM, уникальные для каждой ветви.

```
# Инициализация первого луча (score = 0.0, только токен <sos>)  
beams = [(0.0, [sos_idx], hidden, cell)]
```

```
for _ in range(max_len):  
    new_beams = []  
    for score, tokens, h, c in beams:
```

```

# Если последовательность уже завершена, сохраняем её как есть
if tokens[-1] == eos_idx:
    new_beams.append((score, tokens, h, c))
    continue

# Получаем предсказания декодера для последнего токена луча
output, new_h, new_c = model.decoder(torch.tensor([tokens[-1]]), h, c,
    ↪ encoder_outputs)
log_probs = torch.log_softmax(output, dim=1)

# Берем K лучших продолжений для данного луча
top_v, top_i = torch.topk(log_probs, beam_width)
for i in range(beam_width):
    next_score = score + top_v[0, i].item()
    next_tokens = tokens + [top_i[0, i].item()]
    new_beams.append((next_score, next_tokens, new_h, new_c))

# Сортируем все возможные продолжения и оставляем только топ-K
beams = sorted(new_beams, key=lambda x: x[0], reverse=True)[:beam_width]

```

Работа цикла генерации прекращается в двух случаях:

- Достижение максимальной длины `max_len` (защита от бесконечной генерации).
- Семантическая завершенность: если все K гипотез в луче закончились токеном `<eos>`.

После завершения цикла выбирается гипотеза с наивысшим суммарным логарифмом вероятности. На финальном этапе происходит преобразование индексов обратно в текст с помощью словаря.

Важным моментом является фильтрация технических токенов: `<sos>` игнорируется в начале, генерация строки останавливается при встрече первого `<eos>`, а токены `<unk>` отображаются как соответствующие заглушки. В конце список слов объединяется в финальную строку заголовка.

```

best_tokens = beams[0][1]
result_words = []
for idx in best_tokens:
    if idx == eos_idx: break
    if idx != sos_idx:
        result_words.append(vocab.index2word.get(idx, "<unk>"))
return "".join(result_words)

```

3.5.6 Результаты и сравнительный анализ

Завершающим этапом оценки реализованной архитектуры Seq2Seq с механизмом внимания является анализ метрик обучения и качественная проверка результатов генерации на тестовых данных.

В процессе обучения производился мониторинг функции потерь на обучающей и валидационной выборках. Основным критерием качества выступало значение `val_loss`. Для предотвращения переобучения и сохранения наиболее обобщающей способности модели был реализован механизм сохранения «лучшего» состояния (checkpoint): запись весов в файл происходила только в том случае, если текущее значение потерь на валидации было ниже всех зафиксированных ранее.

В таблице 4 представлены итоговые значения функции потерь для реализованной Seq2Seq модели.

Таблица 4 – Метрики обучения архитектуры Seq2Seq с вниманием

Метрика	Значение
Функция потерь на обучении (Train Loss)	5.182
Функция потерь на валидации (Val Loss)	7.099

Ниже приведены результаты работы модели на тех же фрагментах текста, что использовались для тестирования прошлых архитектур.

Пример 1.

Исходный текст: «У меня большая семья из шести человек: я, мама, папа, старшая сестра, бабушка и дедушка. Мы живем все вместе с собакой Бимом и кошкой Муркой в большом доме в деревне...»

Результат Seq2Seq: «роман»

Пример 2.

Исходный текст: «Я с детства хотел завести собаку, но родители мне не разрешали. Пока я был ребёнком, у меня жил хомяк Хома. Хома был очень маленький и пушистый...»

Результат Seq2Seq: «пория»

Пример 3.

Исходный текст: «Когда я окончил университет, то начал ходить по собеседованиям в разные компании. Я учился на экономиста и хотел работать в финансовой компании...»

Результат Seq2Seq: «мысли жизнь»

При сравнении полученных результатов с предыдущей реализацией на базе простой LSTM можно выделить несколько ключевых изменений в поведении модели:

1. **Устранение заикливания:** предыдущая модель страдала от бесконечных повторов одного и того же токена (например, «шакьямуни шакьямуни...» или «10 10 10...»). Архитектура Seq2Seq с механизмом внимания полностью решила проблему заикливания. Благодаря разделению энкодера и декодера, а также использованию Beam Search, модель генерирует лаконичные и завершённые ответы.
2. **Фокусировка на заголовке:** механизм внимания позволил модели выделять наиболее важные части текста. В то время как обычная LSTM генерировала случайные сущности, не связанные с контекстом, Seq2Seq пытается подобрать обобщающее слово (субъективное «роман» или «мысли жизнь»), что свидетельствует о понимании структуры задачи суммаризации.
3. **Проблема абстрактности:** несмотря на прогресс в структуре, модель всё ещё склонна к слишком общим заголовкам. Это обусловлено тем, что при высоком значении потерь ($\text{Loss} \approx 7$) модель выбирает наиболее безопасные и часто встречающиеся в корпусе слова-обобщения вместо конкретных деталей из текста.

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Techjury. How Much Data Is Created Every Day in 2024? [Электронный ресурс онлайн]. — [Б. м. : б. и.], 2024. — Режим доступа: <https://techjury.net/blog/how-much-data-is-created-every-day/>.
- 2 Шамигов, Ф.Ф. Автоматическая генерация новостных заголовков при помощи нейронной сети RuGPT-3 (влияние обучающего датасета на результативность модели) [Текст] / Шамигов, Ф.Ф. и Резанова, З.И. // Искусственный интеллект и цифровые коммуникации. — 2025. — Т. 4, № 1(13). — С. 62–70. — Поступила в редакцию: 06.11.2024; Принята к печати: 21.01.2025. Режим доступа: <https://elibrary.ru/hfslfk>.
- 3 Boczkowski, Pablo J. News comes across when I'm in a moment of leisure: Understanding the practices of incidental news consumption on social media [Текст] / Boczkowski, Pablo J., Mitchelstein, Eugenia и Matassi, María // New Media & Society. — 2018. — Т. 20, № 10. — С. 3523–3539.
- 4 Боковиков, С. А. Извлечение данных о погоде с помощью парсинга веб-страниц в Python [Текст] // Современные научные исследования и инновации. — 2024. — № 1 (153). — Студент 2 курса, факультет экономико-математический.
- 5 Jurafsky, Daniel. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition [Текст] / Jurafsky, Daniel и Martin, James H. — 2nd изд. — Upper Saddle River : Prentice Hall, 2024.
- 6 Вашкевич, Е.К. ТОКЕНЕЗАЦИЯ В NLP [Электронный ресурс онлайн]. — [Б. м. : б. и.], 2020. — Режим доступа: https://libeldoc.bsuir.by/bitstream/123456789/40257/1/Vashkevich_Tokenizatsiya.pdf.
- 7 Астапов, Р.Л. Автоматизированная предобработка текста для определения эмоциональной окраски текста [Текст] / Астапов, Р.Л. и Мухамадеева, Р.М. //

Актуальные научные исследования в современном мире. — 2021. — № 5-2 (73). — С. 19–23.

- 8 BLEU: a Method for Automatic Evaluation of Machine Translation [Текст] / Papineni, Kishore, Roukos, Salim, Ward, Todd и Zhu, Wei-Jing. — [Б. м.] : Association for Computational Linguistics, 2002. — С. 311–318.
- 9 Jurafsky, Daniel. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models [Текст] / Jurafsky, Daniel и Martin, James H. — 3rd изд. — [Б. м. : б. и.], 2025. — Режим доступа: <https://web.stanford.edu/~jurafsky/slp3/>.
- 10 Lin, Chin-Yew. ROUGE: A Package for Automatic Evaluation of Summaries [Text] // Text Summarization Branches Out. — 2004. — P. 74–81.
- 11 Биджиева, С.Х. Анализ метрик оценки качества генеративных языковых моделей [Текст] / Биджиева, С.Х., Байрамукова, М.И. и Гривева, К.М. // Тенденции развития науки и образования. — 2024. — № 116-19. — С. 15–17.
- 12 Banerjee, Satanjeev. METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments [Текст] / Banerjee, Satanjeev и Lavie, Alon. — 2005. — Июнь. — С. 65–72. — Режим доступа: <https://www.aclweb.org/anthology/W05-0909>.
- 13 Weizenbaum, Joseph. ELIZA – A Computer Program for the Study of Natural Language Communication between Man and Machine [Текст] // Communications of the ACM. — 1966. — Т. 9, № 1. — С. 36–45.
- 14 Colby, Kenneth. Simulation of Belief Systems [Текст] // Computer Models of Thought and Language. — 1972. — С. 251–286.
- 15 Winograd, Terry. Understanding Natural Language [Текст]. — New York : Academic Press, 1972.

- 16 Jurafsky, Daniel. Speech and Language Processing [Текст] / Jurafsky, Daniel и Martin, James H. — 1st изд. — [Б. м.] : Prentice Hall, 2000.
- 17 Katz, Slava M. Estimation of probabilities from sparse data for the language model component of a speech recognizer [Текст]. — 1987. — № TR-13-87.
- 18 Franz, Andreas. N-gram-based language models [Текст] / Franz, Andreas и Brants, Thorsten. — 2006. — С. 1234–1237.
- 19 Aiden, Erez. The Digital Humanities and the Future of the Book [Текст] / Aiden, Erez и Michel, Jean-Baptiste. — [Б. м. : б. и.], 2011. — С. 45–52.
- 20 Infini-gram: Scaling Unbounded n-gram Language Models to a Trillion Tokens [Текст] / Liu, Jiacheng, Min, Sewon, Zettlemoyer, Luke, Choi, Yejin и Hajishirzi, Hannaneh // arXiv preprint arXiv:2401.17377. — 2024. — jan. — License: CC BY 4.0. Режим доступа: <https://arxiv.org/abs/2401.17377>.
- 21 A Neural Probabilistic Language Model [Text] / Bengio, Yoshua, Ducharme, Réjean, Vincent, Pascal, and Jauvin, Christian // Journal of Machine Learning Research. — 2003. — Vol. 3. — P. 1137–1155.
- 22 Olah, Christopher. Understanding LSTM Networks [Electronic resource online]. — [S. l. : s. n.], 2015. — Access mode: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>.
- 23 Москаленко, А.А. Разработка приложения веб-скрапинга с возможностями обхода блокировок [Текст] / Москаленко, А.А., Лапониная, О.Р. и Сухомлин, В.А. // Современные информационные технологии и ИТ-образование. — 2019. — Т. 15, № 2. — С. 413–420.
- 24 Shannon, Claude E. A Mathematical Theory of Communication [Текст] // Bell System Technical Journal. — 1948. — Т. 27, № 3. — С. 379–423.
- 25 Manning, Christopher D. Foundations of Statistical Natural Language Processing [Текст] / Manning, Christopher D. и Schütze, Hinrich. — Cambridge, MA : MIT Press, 1999.

- 26 Jurafsky, Daniel. Speech and Language Processing [Текст] / Jurafsky, Daniel и Martin, James H. — Upper Saddle River, NJ : Prentice Hall, 2000.
- 27 Towards Explainable Evaluation Metrics for Natural Language Generation [Текст] / Leiter, Christoph, Lertvittayakumjorn, Piyawat, Fomicheva, Marina, Zhao, Wei, Gao, Yang и Eger, Steffen. — 2022. — 03.
- 28 Zhang, Zhen. Language Model Perplexity Predicts Scientific Surprise and Transformative Impact [Text] / Zhang, Zhen and Evans, James // arXiv preprint. — 2025. — arXiv:2509.05591.
- 29 Li, Ming. What is Wrong with Perplexity for Long-context Language Modeling? [Text] / Li, Ming and Zhang, Wei // arXiv preprint. — 2024. — arXiv:2410.23771.

ПРИЛОЖЕНИЕ А

Парсинг

```
1 import time
2 import json
3 import re
4 from typing import Optional, Dict
5 from datetime import datetime, timedelta
6 from urllib.parse import urljoin
7 from tqdm import tqdm
8
9 import requests
10 from bs4 import BeautifulSoup
11 from fake_useragent import UserAgent
12
13
14 class ProzaRuParser:
15     """Парсер для сайта proza.ru"""
16
17     def __init__(self, base_url: str = "https://proza.ru/texts/list.html", delay: float =
18         ↪ 1.5):
19         self.base_url = base_url
20         self.headers = {"User-Agent": UserAgent().random}
21         self.timeout = 10
22         self.delay = delay
23         self.output_file = "../data/temp_data/json/data_proza_ru.json"
24
25         with open(self.output_file, 'w', encoding='utf-8') as f:
26             json.dump({}, f, ensure_ascii=False, indent=4)
27
28     def _get_page(self, url: str) -> Optional[BeautifulSoup]:
29         """Загружает страницу и возвращает BeautifulSoup-объект."""
30         try:
31             print(f"Загружается: {url}")
32             time.sleep(self.delay)
33             response = requests.get(url, headers=self.headers, timeout=self.timeout)
34             response.raise_for_status()
35             return BeautifulSoup(response.content, "html.parser")
36         except requests.exceptions.RequestException as e:
37             print(f"Ошибка при загрузке {url}: {e}")
38             return None
39
40     def get_all_forms(self) -> Dict[str, str]:
41         """Получает названия и ссылки на разделы с малыми формами."""
42         soup = self._get_page(self.base_url)
43         if not soup:
44             return {}
45
46         works_block = soup.find('ul', attrs={'type': 'square', 'style':
47             ↪ 'color:#404040'})
48         all_forms = works_block.find_all('ul', attrs={'type': 'square'})
49         data_all_forms = {}
```

```

48     for form in all_forms:
49         category = form.find_all( 'a ' )
50         for link in category:
51             title = link.text.strip()
52             full_link = "https://www.proza.ru" + link[ 'href ' ]
53             data_all_forms[title] = full_link
54
55     return data_all_forms
56
57 def get_text(self, url: str) -> str:
58     soup = self._get_page(url)
59     if not soup:
60         return ""
61
62     text = soup.find( 'div ', attrs={ 'class ': 'text ' })
63     if not text:
64         return ""
65
66     return self.clean_text(str(text))
67
68 def clean_text(self, html: str) -> str:
69     soup = BeautifulSoup(html, 'html.parser ')
70     for element in soup([ 'iframe ', 'img ', 'script ', 'style ',
71                          'div.video-blk ', 'div.video-block ',
72                          'div.ads ', 'div.advertisement ']):
73         element.decompose()
74
75     for div in soup.find_all( 'div ', class_=lambda x: x and 'hidden ' in x):
76         div.decompose()
77
78     clean_text = soup.get_text(separator= '\n ', strip=True)
79     lines = [line.strip() for line in clean_text.split( '\n ' ) if line.strip()]
80     return '\n '.join(lines)
81
82 def get_works(self, url: str, category_title: str) -> Dict[str, Dict[str, str]]:
83     """
84
85     :param url:
86     :param category_title:
87     :return:
88     """
89     soup = self._get_page(url)
90     if not soup:
91         return {}
92
93     works_block = soup.find_all( 'ul ', attrs={ 'type ': 'square ', 'style ':
94                                             ↪ 'color:#404040 ' })
95     data_small_works = {}
96
97     for works_list in works_block:
98         for work in works_list.find_all( 'li ' ):
99             work_data = work.find( 'a ' )
100             if not work_data:

```

```

100         continue
101     try:
102         author = work.find( 'a ', attrs={ 'class ': 'poemlink '}).text.strip()
103         title = work_data.text.strip()
104         link = "https://www.proza.ru" + work_data[ 'href ']
105         text = self.get_text(link)
106
107         data_small_works[author] = {
108             'link ': link,
109             'title ': title,
110             'text ': text
111         }
112         self._update_output_file(category_title, title, data_small_works[title])
113         time.sleep(self.delay)
114     except Exception as e:
115         print(f"Ошибка при обработке произведения: {e}")
116         continue
117
118     return data_small_works
119
120 def parse_by_dates(self, start_date: str, end_date: str, category_title: str, topic:
↪ str) -> Dict[str, dict]:
121     """
122     Парсит материалы за указанный период в обратном порядке
123     :param category_title:
124     :param start_date: Дата начала в формате 'YYYY-MM-DD '
125     :param end_date: Дата окончания в формате 'YYYY-MM-DD '
126     :param topic: ID темы
127     :return: Словарь с данными
128     """
129     current_date = datetime.strptime(start_date, "%Y-%m-%d")
130     end_date = datetime.strptime(end_date, "%Y-%m-%d")
131     result = {}
132
133     while current_date >= end_date:
134         date_str = current_date.strftime("%Y-%m-%d")
135         print(f"\nОбработка даты: {date_str}")
136
137         day = current_date.strftime("%d")
138         month = current_date.strftime("%m")
139         year = current_date.strftime("%Y")
140
141         url = f"{self.base_url}?day={day}&month={month}&year={year}&topic={
↪ topic}"
142         data_day = self.get_works(url, category_title)
143         result.update(data_day)
144
145         current_date -= timedelta(days=1)
146
147     return result
148
149 def _update_output_file(self, category: str, title: str, work_data: dict):
150     """Обновляет JSON-файл, добавляя новое произведение"""

```

```

151     try:
152         with open(self.output_file, 'r ', encoding='utf-8 ') as f:
153             existing_data = json.load(f)
154
155         if category not in existing_data:
156             existing_data[category] = {}
157         existing_data[category][title] = work_data
158
159         with open(self.output_file, 'w ', encoding='utf-8 ') as f:
160             json.dump(existing_data, f, ensure_ascii=False, indent=4)
161
162     except Exception as e:
163         print(f" Ошибка при обновлении файла: {e}")
164
165     def get_all_work(self) -> Dict[str, Dict[str, Dict[str, str]]]:
166         """Получает все малые формы и произведения внутри них."""
167         all_data = {}
168         all_forms = self.get_all_forms()
169
170         for all_form_title, all_form_link in all_forms.items():
171             print(f"\nОбработка категории: {all_form_title}")
172             works = self.get_works(all_form_link, all_form_title)
173             topic = re.search(r 'topic=(\d+) ', all_form_link).group(1)
174             works_by_data = self.parse_by_dates("2025-07-18", "2025-07-11",
175                 ↪ all_form_title, topic)
176             merged_works = {**works, **works_by_data}
177             all_data[all_form_title] = merged_works
178             time.sleep(self.delay)
179
180         return all_data
181
182     def save_to_json(self, data: dict, filename: str = "data/data_proza_ru.json"):
183         """Сохраняет данные в JSON-файл."""
184         try:
185             with open(filename, 'w ', encoding='utf-8 ') as f:
186                 json.dump(data, f, ensure_ascii=False, indent=4)
187                 print(f"\n Данные сохранены в {filename}")
188         except Exception as e:
189             print(f" Ошибка при сохранении JSON: {e}")
190
191     parser = ProzaRuParser()
192     parser.get_all_work()

```

```

1  import random
2  import time
3  import json
4  from typing import Optional, Dict
5
6  from tqdm import tqdm
7
8  import requests
9  from bs4 import BeautifulSoup

```



```

10 from fake_useragent import UserAgent
11
12 link = "https://briefly.ru/cultures/"
13
14
15 class ParsingBriefly():
16     """Парсинг сайта Брифли с краткими пересказами произведений"""
17
18     def __init__(self, url: str):
19         self.briefly_url = "https://briefly.ru"
20         self.base_url = url
21         self.headers = {"User-Agent": UserAgent().random}
22         self.timeout = 5
23         self.filename = "../data/temp_data/json/briefly.json"
24
25     def _get_page(self, url: str) -> Optional[BeautifulSoup]:
26         """Загружает страницу и возвращает BeautifulSoup-объект."""
27         try:
28             self.human_delay()
29             response = requests.get(url, headers=self.headers, timeout=self.timeout)
30             response.raise_for_status()
31             return BeautifulSoup(response.content, "html.parser")
32         except requests.exceptions.RequestException as e:
33             print(f"Ошибка при загрузке {url}: {e}")
34             return None
35
36     def get_authors(self, url: str, name_culture: str = None):
37         """Загружает всех авторов и их произведения"""
38         soup = self._get_page(url)
39         all_data = {}
40         if soup is None:
41             return None
42
43         alphabet_index = soup.find("div", class_="index alphabet")
44         if alphabet_index is None:
45             return None
46
47         authors = alphabet_index.find_all("a", class_="author")
48         for author in tqdm(authors, postfix=name_culture):
49             link = self.briefly_url + author.get("href")
50             author_data = self.get_works(link)
51             if author_data: # защита
52                 all_data.update(author_data)
53
54         return all_data
55
56     def get_works(self, url: str):
57         """Загружает произведения автора"""
58         soup = self._get_page(url)
59         if soup is None:
60             return {}
61
62         works_data = {}

```

```

63     try:
64         section = soup.find("section", class_="works_index")
65
66     if section is None:
67         section = soup.find("section", class_="author_works")
68         if section is None:
69             return works_data
70
71         works = section.find_all("div", class_="w-featured")
72         for work in works:
73             title_el = work.find("div", class_="w-title")
74             link_el = work.find("a")
75             if not title_el or not link_el:
76                 continue
77
78             title = title_el.text.strip()
79             link = self.briefly_url + link_el.get("href")
80             if any(word in title.lower() for word in ["глава", "том", "действие"]):
81                 continue
82
83             text = self.get_text(link, category="published")
84             works_data[title] = text
85             self.human_delay(base=1.5, var=1.0)
86         else:
87             full_retelling = section.find_all("li", class_="published")
88             small_retelling = section.find_all("li", class_="pending")
89
90             for work in small_retelling:
91                 work = work.find("a", class_="title")
92                 if not work:
93                     continue
94                 title = work.text.strip()
95                 link = work.get("href")
96                 if any(word in title.lower() for word in ["глава", "том", "действие"]):
97                     continue
98                 text = self.get_text(link, category="pending")
99                 works_data[title] = text
100                 self.human_delay(base=1.5, var=1.0)
101
102             for work in full_retelling:
103                 work = work.find("a", class_="title")
104                 if not work:
105                     continue
106                 title = work.text.strip()
107                 link = self.briefly_url + work.get("href")
108                 if any(word in title.lower() for word in ["глава", "том", "действие"]):
109                     continue
110                 text = self.get_text(link, category="published")
111                 works_data[title] = text
112                 self.human_delay(base=1.5, var=1.0)
113
114         return works_data
115     except Exception as e:

```

```

116         print(f"Ошибка при парсинге {url}: {e}")
117         return {}
118
119     def get_text(self, url: str, category: str):
120         """Получает текст произведения"""
121         soup = self._get_page(url)
122         if soup is None:
123             return "Описание отсутствует"
124
125         try:
126             if category == "pending":
127                 element = soup.find("div", class_="microsummary__content")
128             else:
129                 element = soup.find("p", class_="microsummary__content")
130
131             if element:
132                 text = element.get_text(strip=True)
133             else:
134                 main_div = soup.find("div", id="text")
135                 if main_div:
136                     for ad in main_div.find_all("div", class_="honey"):
137                         ad.decompose()
138                     paragraphs = [p.get_text(" ", strip=True) for p in
139                                   ↪ main_div.find_all("p")]
140                     text = " ".join(paragraphs)
141                 else:
142                     text = ""
143             return text if text else "Описание отсутствует"
144         except Exception as e:
145             print(f"Ошибка в get_text {url}: {e}")
146             return "Описание отсутствует"
147
148     def get_all_data(self, url: str):
149         all_data = self._load_existing_data()
150         soup = self._get_page(url)
151         try:
152             cultures_cards = soup.find_all("a", class_="visited-hidden")
153             for culture in cultures_cards[7:]:
154                 name = culture.get_text(strip=True)
155                 link = self.briefly_url + culture.get("href")
156                 data_culture = self.get_authors(link, name)
157                 if data_culture:
158                     all_data.update(data_culture)
159
160                 self.save_to_json(all_data)
161                 self.human_delay(base=2, var=1.5)
162         except Exception as e:
163             print(e)
164
165     def save_to_json(self, data: Dict, filename: str = None) -> None:
166         """Сохраняет данные в JSON-файл."""
167         filename = filename or self.filename
168         with open(filename, "w", encoding="utf-8") as f:

```

```

168         json.dump(data, f, ensure_ascii=False, indent=4)
169
170     def _load_existing_data(self) -> Dict:
171         """Загружает уже сохраненные данные, чтобы дописывать новые."""
172         try:
173             with open(self.filename, "r", encoding="utf-8") as f:
174                 return json.load(f)
175         except (FileNotFoundError, json.JSONDecodeError):
176             return {}
177
178     @staticmethod
179     def human_delay(base: float = 1.5, var: float = 1.0, long_pause_prob: float = 0.05):
180         """Делает более человеческие задержки"""
181         delay = random.uniform(base, base + var)
182         time.sleep(delay)
183
184         if random.random() < long_pause_prob:
185             long_delay = random.uniform(5, 15)
186             time.sleep(long_delay)
187
188
189     parser = ParsingBriefly(link)
190     parser.get_all_data(link)

```



```

1  import time
2  import json
3  from typing import Optional, Dict
4  from urllib.parse import urljoin
5  from tqdm import tqdm
6
7  import requests
8  from bs4 import BeautifulSoup
9  from fake_useragent import UserAgent
10
11
12  class LitPrichalParser:
13      """Парсер сайта ЛитПричал"""
14
15      def __init__(self, base_url: str = "https://www.litprichal.ru"):
16          self.base_url = base_url
17          self.headers = {"User-Agent": UserAgent().random}
18          self.timeout = 10
19
20      def _get_page(self, url: str) -> Optional[BeautifulSoup]:
21          """Загружает страницу и возвращает BeautifulSoup-объект."""
22          try:
23              time.sleep(1)
24              response = requests.get(url, headers=self.headers, timeout=self.timeout)
25              response.raise_for_status()
26              return BeautifulSoup(response.content, "html.parser")
27          except requests.exceptions.RequestException as e:
28              print(f"Ошибка при загрузке {url}: {e}")
29              return None

```

```

30
31 def get_genres(self) -> Dict[str, Dict[str, str]]:
32     """Парсит список жанров с главной страницы."""
33     url = f"{self.base_url}/prose.php"
34     soup = self._get_page(url)
35     if not soup:
36         return {}
37
38     genres = {}
39     genre_blocks = soup.find_all("div", class_="col-sm-6 col-md-4")
40
41     print("Парсинг жанров:")
42     for block in tqdm(genre_blocks, desc="Жанры"):
43         for genre_link in block.find_all("a"):
44             name = genre_link.text.strip()
45             link = urljoin(self.base_url, genre_link.get("href"))
46             genres[name] = {"link": link}
47
48     return genres
49
50 def _get_page_count(self, genre_url: str) -> int:
51     """Определяет количество страниц в жанре."""
52     soup = self._get_page(genre_url)
53     if not soup:
54         return 1
55
56     pagination = soup.find("ul", class_="pagination")
57     if not pagination:
58         return 1
59
60     pages = pagination.find_all("li")
61     if not pages:
62         return 1
63
64     try:
65         last_page = int(pages[-1].find("a").get("href")
66                             .strip("/").split("/")[1]
67                             .replace("p", ""))
68         return last_page
69     except (ValueError, IndexError):
70         return 1
71
72 def get_books(self, genre_url: str) -> Dict[str, Dict[str, str]]:
73     """Парсит книги из указанного жанра с учетом пагинации.
74
75     Args:
76         genre_url: URL страницы жанра
77     """
78     total_pages = self._get_page_count(genre_url)
79
80     books_data = {}
81
82     print(f"\nПарсинг книг в жанре {genre_url} (всего страниц: {total_pages}):")

```

```

83
84     for page in range(1, total_pages + 1):
85         page_url = f"{genre_url}/{f'p{str(page)} '}" if page > 1 else genre_url
86         soup = self._get_page(page_url)
87         if not soup:
88             continue
89
90         books = soup.find_all("div", class_="col-md-6 x2")
91
92         for book in tqdm(books, desc=f"Страница {page}/{total_pages}"):
93             try:
94                 title = book.find("a", class_="bigList").text.strip()
95                 link = self.base_url + book.find("a", class_="bigList").get("href")
96                 author = book.find("a", class_="forum").text.strip()
97
98                 if author not in books_data:
99                     text = self.get_text(link)
100                     books_data[author] = {
101                         "link": link,
102                         "title": title,
103                         "text": text,
104                         "genre_url": genre_url
105                     }
106
107             except Exception as e:
108                 print(f"\nОшибка при парсинге книги: {e}")
109                 continue
110
111         time.sleep(1.5)
112
113     return books_data
114
115 def get_text(self, url: str) -> str:
116     """Возвращает текст"""
117     time.sleep(0.5)
118     soup = self._get_page(url)
119     if not soup:
120         return ""
121
122     text_blocks = soup.find_all("div", class_="col-md-12 x2")
123     return self.clean_text(str(text_blocks[1]))
124
125 def clean_text(self, html: str) -> str:
126     """Очищает HTML от ненужных элементов и возвращает чистый текст"""
127     soup = BeautifulSoup(html, 'html.parser')
128
129     for element in soup([ 'iframe ', 'img ', 'script ', 'style ',
130                          'div.video-blk ', 'div.video-block ',
131                          'div.ads ', 'div.advertisement ']):
132         element.decompose()
133
134     for div in soup.find_all( 'div ', class_=lambda x: x and 'hidden ' in x):
135         div.decompose()

```

```

136     clean_text = soup.get_text(separator='\\n ', strip=True)
137     lines = []
138     for line in clean_text.split('\\n '):
139         line = line.strip()
140         if line:
141             lines.append(line)
142
143     final_text = '\\n '.join(lines)
144
145     return final_text
146
147     def save_to_json(self, data: Dict, filename: str =
↵ "data/парсинг/data_litprichal.json") -> None:
148         """Сохраняет данные в JSON-файл."""
149         with open(filename, "w", encoding="utf-8") as f:
150             json.dump(data, f, ensure_ascii=False, indent=4)
151         print(f '\\nДанные сохранены в {filename} ')
152
153     def parse_all_in_genre(self) -> Dict[str, Dict]:
154         """Парсит все жанры и книги в них."""
155         genres = self.get_genres()
156         result = {}
157         print("\\nПарсинг книг по всем жанрам:")
158         for genre_name, genre_data in tqdm(genres.items(), desc="Общий прогресс"):
159             books = self.get_books(genre_data["link"])
160             result[genre_name] = books
161             time.sleep(10)
162         return genres
163
164
165     parser = LitPrichalParser()
166     books_data = parser.parse_all_in_genre()
167     parser.save_to_json(books_data)

```



```

1  import time
2  import json
3  from typing import Optional, Dict
4  from tqdm import tqdm
5
6  import requests
7  from bs4 import BeautifulSoup
8  from fake_useragent import UserAgent
9
10 link = "https://www.litres.ru/genre/klassicheskaya-literatura-5028/"
11
12
13 class LitresParser:
14     """Парсер сайта ЛитРес"""
15
16     def __init__(self, url: str):
17         self.litres_url = "https://www.litres.ru"
18         self.base_url = url
19         self.headers = {"User-Agent": UserAgent().random}

```

```

20     self.timeout = 10
21     self.filename = "/data/temp_data/litres.json"
22
23     def _get_page(self, url: str) -> Optional[BeautifulSoup]:
24         """Загружает страницу и возвращает BeautifulSoup-объект."""
25         try:
26             time.sleep(1.5)
27             response = requests.get(url, headers=self.headers, timeout=self.timeout)
28             response.raise_for_status()
29             return BeautifulSoup(response.content, "html.parser")
30         except requests.exceptions.RequestException as e:
31             print(f"Ошибка при загрузке {url}: {e}")
32             return None
33
34     def get_books(self, url: str, pages: int = 800):
35         """Парсит книги с учетом пагинации и сохраняет после каждой страницы."""
36         books_data = self._load_existing_data()
37
38         for page in tqdm(range(1, pages + 1)):
39             page_url = f"{url}?page={page}"
40             soup = self._get_page(page_url)
41             if not soup:
42                 continue
43
44             books = soup.find_all(
45                 "div", class_="Art-module__3wrtfG__content
46                 ↳ Art-module__3wrtfG__content_full"
47             )
48             time.sleep(1)
49
50             for book in books:
51                 try:
52                     info = book.find("a", class_="ArtInfo-module__Y-DtKG__title")
53                     title = info.text.strip()
54                     link = self.litres_url + info.get("href")
55
56                     if title in books_data:
57                         continue
58
59                     text = self.get_text(link)
60                     books_data[title] = text
61
62                     time.sleep(0.5)
63
64                 except Exception as e:
65                     print(f"\nОшибка при парсинге книги: {e}")
66                     continue
67
68             self.save_to_json(books_data)
69             time.sleep(2)
70
71     return books_data

```



```

72 def get_text(self, url: str):
73     """Получает текст - аннотацию"""
74     soup = self._get_page(url)
75     if not soup:
76         return None
77
78     block = soup.find("div",
79         ↪ class_="BookDetailsAbout-module__p8ABVW__truncate")
80     if not block:
81         return None
82
83     truncated = block.find("div",
84         ↪ class_="Truncate-module__FwxwPG__truncated")
85     return truncated.text if truncated else None
86
87 def save_to_json(self, data: Dict, filename: str = None) -> None:
88     """Сохраняет данные в JSON-файл."""
89     filename = filename or self.filename
90     with open(filename, "w", encoding="utf-8") as f:
91         json.dump(data, f, ensure_ascii=False, indent=4)
92
93 def _load_existing_data(self) -> Dict:
94     """Загружает уже сохраненные данные, чтобы дописывать новые."""
95     try:
96         with open(self.filename, "r", encoding="utf-8") as f:
97             return json.load(f)
98     except (FileNotFoundError, json.JSONDecodeError):
99         return {}
100
101 parser = LitresParser(link)
102 books = parser.get_books(link)

```

ПРИЛОЖЕНИЕ Б

Работа с данными

```
1  #%% md
2  # # Импорт библиотек
3  #%%
4  import json
5  import re
6
7  import pandas as pd
8
9  import matplotlib.pyplot as plt
10 from pandas import DataFrame
11
12 from wordcloud import WordCloud
13
14 #%% md
15 # # Объединим все данные в один датасет
16 #%%
17 def open_json(input_file: str) -> dict:
18     with open(input_file) as json_file:
19         data = json.load(json_file)
20     return data
21 #%%
22 def json_to_csv_lp(json_data: dict) -> DataFrame:
23     rows = []
24     for category, works in json_data.items():
25         for author, work_data in works.items():
26             rows.append({
27                 "title": work_data.get("title", ""),
28                 "text": work_data.get("text", "")
29             })
30
31     df = pd.DataFrame(rows)
32     return df
33 #%%
34 def json_to_csv(json_data: dict) -> DataFrame:
35     rows = []
36     for title, text in json_data.items():
37         rows.append({
38             "title": title,
39             "text": text
40         })
41
42     df = pd.DataFrame(rows)
43     return df
44 #%% md
45 # clean_data
46 #%%
47 briefly = open_json("../data/temp_data/json/briefly.json")
48 litprichal = open_json("../data/temp_data/json/data_litprichal.json")
49 proza_ru = open_json("../data/temp_data/json/data_proza_ru.json")
```

```

50 litres = open_json("../data/temp_data/json/litres.json")
51 #%%
52 briefly = json_to_csv(briefly)
53 litprichal = json_to_csv_lp(litprichal)
54 proza_ru = json_to_csv_lp(proza_ru)
55 litres = json_to_csv(litres)
56 #%%
57 data = pd.concat([briefly, litprichal, proza_ru, litres], ignore_index=True)
58 #%%
59 data
60 #%%
61 len(briefly) + len(litprichal) + len(proza_ru) + len(litres) - len(data)
62 #%% md
63 # Никакие данные не потерялись
64 #%% md
65 # # Работа с данными
66 #%%
67 data = data.apply(lambda col: col.str.lower().str.strip())
68 #%% md
69 # ## Почистим данные от всего лишнего
70 #%% md
71 # ### Удалим лишние символы
72 #%% md
73 # Удалим пропущенные значения
74 #%%
75 data.isna().sum()
76 #%%
77 data = data.dropna().reset_index(drop=True)
78 #%%
79 def clean_text(text):
80     """Более аккуратная очистка текста для генерации заголовков"""
81     if pd.isna(text):
82         return ""
83     text = str(text).lower()
84     text = re.sub(r"http\S+", "", text)
85     text = re.sub(r"<[^\>]+\>", "", text)
86     text = re.sub(r"^[^\w\s,!\?-\]", " ", text)
87     text = re.sub(r"\s+", " ", text).strip()
88     return text
89 #%%
90 data.text = data.text.apply(clean_text)
91 #%% md
92 # ## Уберем из названий нумерацию (том, эпизод и т.п.)
93 #%%
94 def clean_title_completely(title):
95     """
96     Полная очистка названия от всех видов нумерации
97     """
98     if pd.isna(title):
99         return title
100
101     cleaned = str(title).lower().strip()
102

```

```

103 complex_patterns = [
104     r '(?s*\d+\s+(?:эпизод|серия|глава|часть)\s+\d+\s+(?:том|книга|т\.)\s*\d_
    ↪ *s*\.\.?)? ',
105     r '(?s*\d+\s+(?:том|книга|т\.)\s+\d+\s+(?:эпизод|серия|глава|часть)\s*\d_
    ↪ *s*\.\.?)? ',
106     r '\b\d+\s+\d+\s+(?:том|часть|книга|эпизод|серия)\b ',
107     r '\b(?:том|часть|книга|эпизод|серия)\s+\d+\s+\d+\b ',
108     r '\b\d+[-\.,]\s*\d+\s+(?:том|часть|книга) ',
109     r '\b(?:том|часть|книга)\s+\d+[-\.,]\s*\d+\b ',
110 ]
111
112 for pattern in complex_patterns:
113     cleaned = re.sub(pattern, ' ', cleaned, flags=re.IGNORECASE)
114
115 basic_patterns = [
116     r '\s*(?:том|часть|книга|т\.|vol\.|?эпизод|серия|глава|выпуск)\s*[ivxldm0-9]+ ',
117     r '\s*[ivxldm0-9]+\s*(?:том|часть|книга|т\.|vol\.|?эпизод|серия|глава|выпуск) ',
118     r '\s*\d+[-\.,]\s*(?:том|часть|книга|глава) ',
119     r '\s*(?:том|часть|книга|глава)[-\.,]?s*\d+ ',
120 ]
121 for pattern in basic_patterns:
122     cleaned = re.sub(pattern, ' ', cleaned, flags=re.IGNORECASE)
123
124 number_words = [
125     "первая", "вторая", "третья", "четвертая", "пятая", "шестая", "седьмая",
126     "восьмая", "девятая", "десятая", "одиннадцатая", "двенадцатая",
    ↪ "тринадцатая",
127     "четырнадцатая", "пятнадцатая", "шестнадцатая", "семнадцатая",
    ↪ "восемнадцатая",
128     "девятнадцатая", "двадцатая"
129 ]
130 text_num_pattern = r '\b(?:глава|часть|том|эпизод|серия|книга|выпуск)\s+(?: '
    ↪ + "|".join(number_words) + r ')\b '
131 cleaned = re.sub(text_num_pattern, ' ', cleaned, flags=re.IGNORECASE)
132
133 cleaned = re.sub(
134     r '\b\d+[-]?(?:я|й|е|ой|ая|ое|ые|ых)?\s+(?:глава|часть|том|книга|серия|эпизо_
    ↪ д|выпуск)\b ',
135     ' ', cleaned, flags=re.IGNORECASE
136 )
137 cleaned = re.sub(
138     r '\b(?:глава|часть|том|книга|серия|эпизод|выпуск)\s+\d+[-]?(?:я|й|е|ая|ое|ы_
    ↪ е|ых)?\b ',
139     ' ', cleaned, flags=re.IGNORECASE
140 )
141
142 cleaned = re.sub(r '№ ', ' ', cleaned)
143
144 cleaned = re.sub(r '\s*([^\s]*\d+[\s]*)', ' ', cleaned)
145 cleaned = re.sub(r '\s+\d+\s*\.?$', ' ', cleaned)
146 cleaned = re.sub(r '^ \d+\s+', ' ', cleaned)
147
148 cleaned = re.sub(r '[^\w\s,!?-]', ' ', cleaned)

```

```

149     cleaned = re.sub(r'\s+', ' ', cleaned)
150     cleaned = cleaned.strip(' ,.- ')
151
152     return cleaned
153     #%%
154     data["cleaned_title"] = data.title.apply(clean_title_completely)
155     #%%
156     changed_count = (data.title != data.cleaned_title).sum()
157     print(f"Изменено названий: {changed_count} из {len(data)}")
158     #%%
159     data.describe()
160     #%% md
161     # почему то есть полностью пустые тексты
162     #%%
163     data.isna().sum()
164     #%%
165     data = data[data.text != ""].reset_index(drop=True)
166     data = data[data.title != ""].reset_index(drop=True)
167     #%%
168     data.describe()
169     #%% md
170     # есть пустые тексты с заглушкой «описание отсутствует», удалим их
171     #%%
172     data = data[data.text != "описание отсутствует"].reset_index(drop=True)
173     #%%
174     data.describe()
175     #%%
176     data.cleaned_title.value_counts().head(20)
177     #%% md
178     # Почистим строки, в которых есть не русские буквы
179     #%%
180     def keep_only_russian(text):
181         if pd.isna(text):
182             return ""
183         text = re.sub(r"[^А-Яа-яЁё0-9\s.,!?-]", "", text)
184         text = re.sub(r'\s+', " ", text).strip()
185         return text
186
187     data.text = data.text.apply(keep_only_russian)
188     data.cleaned_title = data.cleaned_title.apply(keep_only_russian)
189
190     data = data[(data.text != "") & (data.cleaned_title != "")].reset_index(drop=True)
191
192     #%% md
193     # Заметим, что в названиях часто встречаются пустые названия или состоящие
194     ↪ только из символов.
195     #%%
196     data = data[data.cleaned_title.str.strip() != ""]
197     noise_titles = ["****"]
198     data = data[~data.cleaned_title.isin(noise_titles)].reset_index(drop=True)
199
200     print(data.cleaned_title.value_counts().head(20))
201     print(f"Всего примеров после очистки: {len(data)}")

```

```

201 #%% md
202 # ## Почистим строки, в которых нет текста и удалим лидирующую пунктуацию
203 #%%
204 def is_meaningful(text):
205     return bool(re.search(r"[А-Яа-яА-Za-z0-9]", text))
206
207 def clean_leading_punct(text):
208     return re.sub(r"^[^\wА-Яа-я0-9]+", "", text).strip()
209 #%%
210 data = data[data.text.apply(is_meaningful)].reset_index(drop=True)
211 data.text = data.text.str.replace(r"^[^\w\s,.\!?-]", " ", regex=True)
212 data.text = data.text.str.replace(r"\s+", " ", regex=True).str.strip()
213 data.describe()
214 #%% md
215 # ## Посмотрим на дубликаты
216 #%%
217 data.duplicated().sum()
218 #%%
219 data.text.duplicated().sum(), data.title.duplicated().sum()
220 #%% md
221 # Заметим, что есть дубликаты в названиях, но это нормально, так как парсились
    ↳ разные сайты, на которых могли быть одни и те же произведения. Такое
    ↳ можно оставить.
222 #
223 # Но есть дубликаты в тексте, причем большинство из них такие, что текст
    ↳ одинаковый, а названия разные. Такое нужно удалять, это создаст лишний
    ↳ шум для модели.
224 #%%
225 data = data[~data.text.duplicated(keep=False)].reset_index(drop=True)
226 #%%
227 data.duplicated().sum()
228 #%%
229 data.describe()
230 #%%
231 data.drop(columns=["title"], inplace=True)
232 #%%
233 data.rename(columns={"cleaned_title": "title"}, inplace=True)
234 #%%
235 data.to_csv("../data/temp_data/all_data_cleaned.csv", index=False)
236 #%% md
237 # ## Анализ длины слов в названиях
238 #%%
239 title_counts = data.title.value_counts()
240 title_counts.head(10)
241 #%%
242 data["title_len"] = data.title.apply(lambda x: len(str(x).split()))
243
244 length_counts = data.groupby('title_len').size()
245
246 plt.figure(figsize=(12,6))
247 length_counts.plot(kind='bar')
248 plt.xlabel("Длина названия (слов)")
249 plt.ylabel("Количество названий")

```

```

250 plt.title("Распределение длин названий в датасете без аугментации")
251 plt.show()
252 #%% md
253 # Заметно, что есть названия с большим количеством слов. Это внесет в модель
    ↪ шум, поэтому стоит удалить их.
254 #%%
255 data.describe(include="O")
256 #%%
257 data = data[data.title_len <= 10].reset_index(drop=True)
258 #%%
259 data.describe(include="O")
260 #%%
261 data['text_len'] = data.text.str.split().apply(len)
262 #%%
263 print("Всего примеров:", len(data))
264 print("Уникальные заголовки:", data.title.nunique())
265 print("Средняя длина текста:", data.text_len.mean())
266 print("Средняя длина заголовка:", data.title_len.mean())
267 #%%
268 data.head()
269 #%%
270 data.drop(columns=["title_len", "text_len"], inplace=True)
271 #%%
272 data.to_csv("../data/all_data.csv", index=False)
273 #%% md
274 # # Посмотрим на облако слов
275 #%%
276 all_titles = " ".join(data.title.astype(str))
277
278 wordcloud = WordCloud(
279     width=1200, height=600,
280     background_color="white",
281     max_words=400,
282     colormap="viridis",
283     collocations=False,
284 ).generate(all_titles)
285 #%%
286 plt.figure(figsize=(12, 6))
287 plt.imshow(wordcloud, interpolation='bilinear')
288 plt.axis("off")
289 plt.title("Облако слов по заголовкам", fontsize=16)
290 plt.show()
291 #%% md
292 # Заметно, что самые популярные слова — предлоги, союзы и т.п., что логично. Их
    ↪ можно убрать, но оставим, чтобы модель училась генерировать названия,
    ↪ приближенные к реальности.

1 # %% md
2 # # Импорт библиотек
3 # %%
4 import re
5
6 import pandas as pd

```

```

7 import nltk
8 import plotly.graph_objects as go
9 from sklearn.model_selection import train_test_split
10 import numpy as np
11
12 from tqdm import tqdm
13
14 nltk.download("punkt")
15
16 from nltk.tokenize import sent_tokenize
17
18 # %%
19 data = pd.read_csv("../data/all_data.csv")
20 # %%
21 data.describe()
22 # %% md
23 # # Разделение данных
24 # %%
25 train_df, val_df = train_test_split(data, test_size=0.2, random_state=42)
26 # %%
27 len(train_df), len(val_df)
28 # %%
29 val_df.describe()
30
31
32 # %% md
33 # Заметим, что в валидацию попали несколько текстов с одинаковыми названиями.
34   ↪ Это не очень хорошо, так как может исказить результаты оценки модели.
35 #
36 # Переделаем разделение.
37 # %%
38 def split_with_controlled_test_size(data, target_test_size=0.2, random_state=42):
39     np.random.seed(random_state)
40
41     title_groups = data.groupby('title').apply(lambda x: x.index.tolist()).to_dict()
42
43     unique_titles = list(title_groups.keys())
44     np.random.shuffle(unique_titles)
45
46     train_indices = []
47     test_indices = []
48
49     target_test_count = int(len(data) * target_test_size)
50
51     for title in unique_titles:
52         indices = title_groups[title]
53
54         if len(test_indices) < target_test_count:
55             test_idx = np.random.choice(indices, 1)[0]
56             test_indices.append(test_idx)
57             train_indices.extend([idx for idx in indices if idx != test_idx])
58         else:
59             train_indices.extend(indices)

```



```

59
60     return data.iloc[train_indices], data.iloc[test_indices]
61
62
63 train_df, val_df = split_with_controlled_test_size(data)
64 # %%
65 print(f"Общий размер данных: {len(data)}")
66 print(f"Тренировочная выборка: {len(train_df)} записей ({len(train_df) / len(data)
    ↪ * 100:.1f}%)")
67 print(f"Валидационная выборка: {len(val_df)} записей ({len(val_df) / len(data) *
    ↪ 100:.1f}%)")
68 # %%
69 val_df.describe()
70 # %% md
71 # Теперь в валидации нет повторяющихся заголовков.
72 # %%
73 val_df.to_csv("../data/training_data/val_df.csv", index=False)
74 # %% md
75 # val_df.to_csv("../data/val_df.csv", index=False) # Аугментация данных
76 # %% md
77 # Аугментировать будем только train набор, чтобы не произошла утечка.
78 # %%
79 train_df.reset_index(drop=True, inplace=True)
80
81
82 # %%
83 def split_text_into_chunks(text, sentences_per_chunk=3):
84     sentences = sent_tokenize(text, language="russian")
85     chunks = []
86     for i in range(0, len(sentences), sentences_per_chunk):
87         chunk = " ".join(sentences[i:i + sentences_per_chunk])
88         chunks.append(chunk)
89     return chunks
90
91
92 # %%
93 augmented_rows = []
94 for _, row in tqdm(train_df.iterrows(), total=len(train_df)):
95     chunks = split_text_into_chunks(row.text, sentences_per_chunk=3)
96     for chunk in chunks:
97         augmented_rows.append({"text": chunk, "title": row.title})
98 # %%
99 augmented_dataset = pd.DataFrame(augmented_rows)
100 # %%
101 augmented_dataset.describe()
102
103
104 # %% md
105 # ## Почистим строки, в которых нет текста и удалим лидирующую пунктуацию
106 # %%
107 def is_meaningful(text):
108     return bool(re.search(r"[А-Яа-яА-Za-z0-9]", text))
109

```

```

110
111 def clean_leading_punct(text):
112     return re.sub(r"^[^\wА-Яа-я0-9]+", "", text).strip()
113
114
115 # %%
116 augmented_dataset = augmented_dataset[augmented_dataset.text.apply(is_meaning_
    ↪ ful)].reset_index(drop=True)
117 augmented_dataset.text = augmented_dataset.text.str.replace(r"^[^\w\s,!?-]", " ",
    ↪ regex=True)
118 augmented_dataset.text = augmented_dataset.text.str.replace(r"\s+", " ",
    ↪ regex=True).str.strip()
119 augmented_dataset.describe()
120 # %% md
121 # Удалим строки, с небольшим количеством данных
122 # %%
123 augmented_dataset = augmented_dataset[augmented_dataset.text.str.strip().str.len()
    ↪ > 10]
124 # %%
125 augmented_dataset.describe()
126 # %% md
127 # Заметим, что у нас очень много текстов с одинаковыми названиями. Это может
    ↪ плохо повлиять на модель, если она будет видеть одни и те же названия.
    ↪ Оставим только по 500 каждого
128 # %%
129 max_per_title = 500
130 augmented_dataset = augmented_dataset.groupby("title").head(max_per_title).rese
    ↪ t_index(drop=True)
131
132 print(augmented_dataset.title.value_counts().head(10))
133 # %%
134 augmented_dataset.describe()
135 # %% md
136 # Появились одинаковые тексты
137 # %%
138 augmented_dataset.duplicated().sum()
139 # %% md
140 # Есть полные дубликаты текст + название. Такое удалим.
141 # %%
142 augmented_dataset.drop_duplicates(inplace=True)
143 # %%
144 augmented_dataset.reset_index(drop=True, inplace=True)
145 # %%
146 augmented_dataset.describe()
147 # %% md
148 # Все еще остались одинаковые тексты, но теперь у них разные названия. Удалим
    ↪ и их.
149 # %%
150 augmented_dataset = augmented_dataset[~augmented_dataset.text.duplicated(keep_
    ↪ =False)].reset_index(drop=True)
151 # %%
152 augmented_dataset.describe()

```

```

153 # %%
154 augmented_dataset
155 # %%
156 augmented_dataset.to_csv("../data/training_data/train_df.csv", index=False)
157 # %% md
158 # ## Посмотрим на распределение названий по длине после разделения
159 # %%
160 train_df = augmented_dataset.copy()
161 # %%
162 dup_titles = (
163     augmented_dataset.groupby("text")["title"]
164     .nunique()
165     .reset_index()
166     .query("title > 1")
167 )
168
169 print(f"Текстов с одинаковыми содержаниями, но разными названиями:
    ↪ {len(dup_titles)}")
170 # %%
171 data["title_len"] = data.title.apply(lambda x: len(str(x).split()))
172 length_counts = data.groupby('title_len').size()
173 train_df["title_len"] = train_df.title.apply(lambda x: len(str(x).split()))
174 length_counts_aug = train_df.groupby('title_len').size()
175 val_df["title_len"] = val_df.title.apply(lambda x: len(str(x).split()))
176 length_counts_aug_val = val_df.groupby('title_len').size()
177 # %%
178 fig = go.Figure()
179
180 fig.add_trace(go.Bar(
181     x=length_counts.index,
182     y=length_counts.values,
183     name="Без аугментации",
184     marker_color="blue"
185 ))
186
187 fig.add_trace(go.Bar(
188     x=length_counts_aug.index,
189     y=length_counts_aug.values,
190     name="train после аугментации",
191     marker_color="orange"
192 ))
193
194 fig.add_trace(go.Bar(
195     x=length_counts_aug_val.index,
196     y=length_counts_aug_val.values,
197     name="test после аугментации",
198     marker_color="green"
199 ))
200
201 fig.update_layout(
202     title="Сравнение распределения длин названий до и после аугментации",
203     xaxis_title="Длина названия (слов)",
204     yaxis_title="Количество названий",

```

```
205     barmode="group",
206     bargap=0.2,
207     bargroupgap=0.1,
208     width=1000,
209     height=500
210 )
211
212 fig.show()
```

ПРИЛОЖЕНИЕ В

Модель на основе n-gram

```
1 import random
2 import re
3 from collections import defaultdict, Counter
4 from typing import List, Tuple
5 from nltk.translate.meteor_score import meteor_score
6 import pickle
7
8 import pandas as pd
9 from tqdm import tqdm
10
11
12 def tokenize(text: str) -> List[str]:
13     """Функция самой простой предобработки текста, основанной на разбиении на
14     ↪ токены по пробелам."""
15     text = text.lower()
16     text = re.sub(r"[^a-za-яё0-9\s]", "", text)
17     return text.split()
18
19 class TitleNgramModel:
20     """Модель n-грамм, обучающаяся по парам (текст ↪ название)."""
21
22     def __init__(self, n_gram: int = 4, smoothing: str | None = 'laplace', alpha: float
23     ↪ = 1.):
24         self.n_gram = n_gram
25         self.ngrams = defaultdict(Counter)
26         self.context_counts = defaultdict(int)
27         self.vocab = set()
28         self.start_token = "<s>"
29         self.title_token = "<title>"
30         self.end_token = "</s>"
31         self.unknown_token = "<unk>"
32         self.smoothing = smoothing
33         self.alpha = alpha
34         self.lower_order_models = {}
35
36     def create_ngrams(self, tokens: List[str]) -> list[tuple[str, ...]]:
37         """Создание n-грамм с учётом начала и конца."""
38         tokens = [self.start_token] + tokens + [self.end_token]
39         return [tuple(tokens[i:i + self.n_gram]) for i in range(len(tokens) - self.n_gram +
40         ↪ 1)]
41
42     def train(self, pairs: List[Tuple[str, str]]) -> None:
43         """Обучение на списке пар (text, title)"""
44         all_tokens = []
45         for text, title in pairs:
46             text_tokens = tokenize(text)
47             title_tokens = tokenize(title)
48             combined = text_tokens + [self.title_token] + title_tokens
```

```

47         all_tokens.extend(combined)
48
49     token_counts = Counter(all_tokens)
50     self.vocab = set(token for token, count in token_counts.items() if count > 1)
51     self.vocab.update([self.start_token, self.title_token, self.end_token,
52         ↪ self.unknown_token])
53
54     if self.n_gram > 1:
55         for n in range(1, self.n_gram):
56             self.lower_order_models[n] = TitleNgramModel(n, self.smoothing, self.alpha)
57             self.lower_order_models[n].train(pairs)
58
59     for text, title in pairs:
60         text_tokens = tokenize(text)
61         title_tokens = tokenize(title)
62
63         text_tokens = [token if token in self.vocab else self.unknown_token for token
64             ↪ in text_tokens]
65         title_tokens = [token if token in self.vocab else self.unknown_token for token
66             ↪ in title_tokens]
67
68         combined = text_tokens + [self.title_token] + title_tokens
69         ngrams_list = self.create_ngrams(combined)
70
71         for ngram in ngrams_list:
72             context = ngram[:-1]
73             next_word = ngram[-1]
74             self.ngrams[context][next_word] += 1
75             self.context_counts[context] += 1
76
77 def train_from_csv(self, csv_path: str, text_col: str = "text", title_col: str =
78     ↪ "title",
79     limit: int | None = None):
80     """Обучение модели по CSV-файлу"""
81     df = pd.read_csv(csv_path)
82     if limit:
83         df = df.head(limit)
84
85     pairs = []
86     for _, row in tqdm(df.iterrows(), total=len(df), desc="Обучение модели"):
87         text = getattr(row, text_col, None)
88         title = getattr(row, title_col, None)
89         if text and title:
90             pairs.append((text, title))
91
92     self.train(pairs)
93
94     print(f"Обучение завершено. Количество уникальных контекстов:
95         ↪ {len(self.ngrams)}")
96
97 def get_simple_probability(self, context: tuple, word: str) -> float:
98     """Вычисляет вероятность по формуле  $P(w_t | context) = \frac{C(context + word)}{C(context)}$ """
99     ↪ / C(context)"""

```

```

94     context_counts = self.ngrams.get(context, {})
95     total = sum(context_counts.values())
96     return context_counts[word] / total if total else 0.0
97
98     def get_laplace_probability(self, context: tuple, word: str) -> float:
99         """Сглаживание Лапласа (Add-one)."""
100         context_counts = self.ngrams.get(context, {})
101         total = sum(context_counts.values())
102         vocab_size = len(self.vocab)
103
104         count_word = context_counts.get(word, 0)
105         return (count_word + self.alpha) / (total + self.alpha * vocab_size)
106
107     def get_linear_interpolation_probability(self, context: tuple, word: str) -> float:
108         """Линейная интерполяция с моделями меньшего порядка."""
109         if self.n_gram == 1 or not self.lower_order_models:
110             return self.get_laplace_probability(context, word)
111
112         lambda_current = 0.6
113         lambda_backoff = 0.4
114
115         current_prob = self.get_laplace_probability(context, word)
116
117         backoff_context = context[1:] if len(context) > 1 else tuple()
118         backoff_model = self.lower_order_models[self.n_gram - 1]
119         backoff_prob = backoff_model.get_probability(backoff_context, word)
120
121         return lambda_current * current_prob + lambda_backoff * backoff_prob
122
123     def get_probability(self, context: tuple, word: str) -> float:
124         """Вычисляет вероятность с выбранным методом сглаживания."""
125
126         if word not in self.vocab:
127             word = self.unknown_token
128
129         if self.smoothing == 'laplace':
130             return self.get_laplace_probability(context, word)
131         elif self.smoothing == 'linear':
132             return self.get_linear_interpolation_probability(context, word)
133         elif self.smoothing is None:
134             return self.get_simple_probability(context, word)
135         else:
136             return self.get_laplace_probability(context, word)
137
138     def generate_with_backoff(self, context: tuple, max_depth: None | int = None):
139         """Генерация с backoff: если контекст не найден, используем контекст
140         ↪ короче."""
141
142         if max_depth is None:
143             max_depth = self.n_gram - 1
144
145         current_context = context
146         depth = 0

```

```

146
147 while depth <= max_depth:
148     if current_context in self.ngrams:
149         probabilities = dict()
150
151         for word in self.vocab:
152             if word not in [self.title_token, self.title_token, self.end_token]:
153                 probabilities[word] = self.get_probability(current_context, word)
154
155         total_prob = sum(probabilities.values())
156         if total_prob > 0:
157             return {word: prob / total_prob for word, prob in probabilities.items()}
158
159         if len(current_context) > 1:
160             current_context = current_context[1:]
161         else:
162             break
163         depth += 1
164
165     words = [w for w in self.vocab if w not in [self.start_token, self.title_token,
166         ↪ self.end_token]]
167     return {word: 1.0 / len(words) for word in words}
168
169 def generate_title(self, input_text: str, max_words: int = 7) -> str:
170     """Генерация названия на основе входного текста с использованием backoff и
171     ↪ сглаживания."""
172     tokens = tokenize(input_text)
173     tokens = [token if token in self.vocab else self.unknown_token for token in tokens]
174     tokens += [self.title_token]
175     context = tuple(tokens[-(self.n_gram - 1):])
176     result = []
177
178     for _ in range(max_words):
179         word_probs = self.generate_with_backoff(context)
180
181         if not word_probs:
182             break
183
184         word_probs = {word: prob for word, prob in word_probs.items()}
185         total = sum(word_probs.values())
186         word_probs = {word: prob / total for word, prob in word_probs.items()}
187         words = list(word_probs.keys())
188         probs = list(word_probs.values())
189
190         next_word = random.choices(words, weights=probs)[0]
191
192         if next_word in {self.end_token, self.title_token}:
193             break
194
195         result.append(next_word)
196         context = (*context[1:], next_word) if len(context) > 0 else (next_word,)
197
198     title = " ".join(result)

```



```

197         return title if title else "Без названия"
198
199     def evaluate_meteor(self, test_pairs: List[Tuple[str, str]]) -> float:
200         """Вычисление средней МЕТЕОР-оценки для тестового набора (text,
201             ↳ reference_title)."""
202         scores = []
203         for text, true_title in tqdm(test_pairs, desc="Оценка МЕТЕОР"):
204             generated = self.generate_title(text)
205             score = meteor_score([tokenize(true_title)], tokenize(generated))
206             scores.append(score)
207         return sum(scores) / len(scores) if scores else 0.0
208
209     def save(self, path: str):
210         with open(path, "wb") as f:
211             pickle.dump(self, f)
212
213     @staticmethod
214     def load(path: str):
215         with open(path, "rb") as f:
216             return pickle.load(f)
217
218
219 1 import pandas as pd
220 2 from n_gram_model import TitleNgramModel
221 3
222 4 train_df = "../data/training_data/train_df.csv"
223 5 val_df = pd.read_csv("../data/training_data/val_df.csv")
224 6
225 7 model = TitleNgramModel(n_gram=3)
226 8 model.train_from_csv(train_df)
227 9 model.save(path="../models/history/title_ngram_model.pkl")
228 10
229 11 val_pairs = list(zip(val_df.text, val_df.title))
230 12 meteor = model.evaluate_meteor(val_pairs)
231 13 print(f"\nСредняя МЕТЕОР-оценка на валидационном наборе: {meteor:.4f}")
232
233
234 1 from n_gram_model import TitleNgramModel
235 2
236 3 model = TitleNgramModel.load("../models/history/title_ngram_model.pkl")
237 4
238 5 while True:
239 6     text = input("> Введите текст или exit для выхода")
240 7     if text.lower() == "exit":
241 8         break
242 9     print(f"Сгенерированное название: {model.generate_title(text)}")

```

ПРИЛОЖЕНИЕ Г

Feedforward Neural Network Language Model

```
1 import pandas as pd
2 from torch.utils.data import Dataset
3 from collections import Counter
4 import re
5 from my_models.history.FNNLM.config import *
6
7
8 def first_n_sentences(text, n=2):
9     text = str(text)
10    sentences = re.split(r'[!?.]+', text)
11    sentences = [s.strip() for s in sentences if s.strip()]
12    return ' '.join(sentences[:n])
13
14
15 def get_balanced_sample(df, limit=100000):
16     title_counts = df['title'].value_counts()
17
18     max_per_title = max(1, limit // len(title_counts))
19
20     sampled_dfs = []
21     for title in title_counts.index:
22         title_df = df[df['title'] == title].head(max_per_title)
23         sampled_dfs.append(title_df)
24
25     result_df = pd.concat(sampled_dfs).sample(frac=1).reset_index(drop=True)
26     return result_df.head(limit)
27
28
29 def clean_text(text):
30     return re.sub(r'[^\w\s]', ' ', str(text).lower()).split()
31
32
33 class FNNLMDataset(Dataset):
34     def __init__(self, csv_file, context_size, vocab=None, first_sentences: int =
35         ↪ None):
36         df = pd.read_csv(csv_file).head(LIMIT)
37         df = get_balanced_sample(df, LIMIT)
38
39         self.data = []
40         if first_sentences is not None:
41             df['text'] = df['text'].apply(lambda x: first_n_sentences(x, first_sentences))
42
43         if vocab is None:
44             # Создаём словарь для train
45             words_corpus = []
46             for _, row in df.iterrows():
47                 words_corpus.extend(clean_text(row['text']))
48                 words_corpus.extend(clean_text(row['title']))
```

```

49     word_counts = Counter(words_corpus)
50     filtered_words = sorted([w for w, freq in word_counts.items() if freq >=
    ↪     MIN_WORD_FREQ])
51
52     self.vocab = {word: i + 3 for i, word in enumerate(filtered_words)}
53     self.vocab['<PAD>'] = 0
54     self.vocab['<UNK>'] = 1
55     self.vocab['<END>'] = 2
56     else:
57         self.vocab = vocab
58
59     self.rev_vocab = {i: w for w, i in self.vocab.items()}
60
61     for _, row in df.iterrows():
62         text_tokens = clean_text(row['text'][:context_size])
63         title_tokens = clean_text(row['title']) + ['<END>']
64
65         if len(text_tokens) < context_size:
66             text_tokens = ['<PAD>'] * (context_size - len(text_tokens)) +
    ↪             text_tokens
67
68         current_context = text_tokens
69         for target in title_tokens:
70             input_ids = []
71             for w in current_context:
72                 if w in self.vocab:
73                     input_ids.append(self.vocab[w])
74                 else:
75                     input_ids.append(self.vocab['<UNK>'])
76
77             if target in self.vocab:
78                 target_id = self.vocab[target]
79             else:
80                 target_id = self.vocab['<UNK>']
81
82             self.data.append((torch.tensor(input_ids, dtype=torch.long), target_id))
83
84             current_context = current_context[1:] + [target if target in self.vocab else
    ↪             '<UNK>']
85
86     if vocab is not None:
87         all_words = set()
88         for _, row in df.iterrows():
89             all_words.update(clean_text(row['text']))
90             all_words.update(clean_text(row['title']))
91
92     def __len__(self):
93         return len(self.data)
94
95     def __getitem__(self, idx):
96         return self.data[idx]

```

```

1 import torch

```

```

2 import torch.nn as nn
3
4
5 class FNNLM(nn.Module):
6     def __init__(self, vocab_size, embedding_dim, hidden_dim, dropout=0.3):
7         super(FNNLM, self).__init__()
8         self.embeddings = nn.Embedding(vocab_size, embedding_dim)
9         self.dropout = nn.Dropout(dropout)
10
11         self.linear1 = nn.Linear(embedding_dim, hidden_dim)
12         self.linear2 = nn.Linear(hidden_dim, vocab_size)
13
14         self.direct = nn.Linear(embedding_dim, vocab_size)
15
16     def forward(self, x):
17         embeds = self.embeddings(x)
18
19         pooled = torch.mean(embeds, dim=1)
20         pooled = self.dropout(pooled)
21
22         hidden = torch.tanh(self.linear1(pooled))
23         hidden = self.dropout(hidden)
24
25         output = self.linear2(hidden) + self.direct(pooled)
26         return output

```



```

1 import math
2 import os
3 import pickle
4
5 from torch import nn, optim
6 from torch.utils.data import DataLoader
7 import torch.nn.functional as F
8 import re
9
10 from tqdm import tqdm
11
12 from my_models.history.FNNLM.config import *
13 from my_models.history.FNNLM.data.dataset import FNNLMDataset
14 from my_models.history.FNNLM.model.model import FNNLM
15
16
17 def train_model():
18     train_dataset = FNNLMDataset(TRAIN_DATASET, CONTEXT_SIZE)
19     val_dataset = FNNLMDataset(VAL_DATASET, CONTEXT_SIZE,
20     ↪ vocab=train_dataset.vocab, first_sentences=2)
21
22     train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE,
23     ↪ shuffle=True)
24     val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
25
26     model = FNNLM(len(train_dataset.vocab), EMBEDDING_DIM,
27     ↪ CONTEXT_SIZE, HIDDEN_DIM, DROPOUT).to(DEVICE)

```

```

25 pad_id = train_dataset.vocab["<PAD>"]
26 criterion = nn.CrossEntropyLoss(ignore_index=pad_id, label_smoothing=0.1)
27 optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE,
    ↪ weight_decay=WEIGHT_DECAY)
28
29 counter = 0
30 best_val_loss = float('inf')
31
32 model.train()
33 pbar_epochs = tqdm(range(EPOCHS), desc="Обучение FNNLM")
34
35 for epoch in pbar_epochs:
36     model.train()
37     train_loss = 0
38
39     for inputs, targets in train_loader:
40         inputs, targets = inputs.to(DEVICE), targets.to(DEVICE)
41         optimizer.zero_grad()
42         outputs = model(inputs)
43         loss = criterion(outputs, targets)
44         loss.backward()
45         optimizer.step()
46         train_loss += loss.item()
47
48     avg_train_loss = train_loss / len(train_loader)
49
50     # --- ВАЛИДАЦИЯ ---
51     model.eval()
52     val_loss = 0
53     val_samples = 0
54     with torch.no_grad():
55         for inputs, targets in val_loader:
56             inputs, targets = inputs.to(DEVICE), targets.to(DEVICE)
57             outputs = model(inputs)
58             loss = criterion(outputs, targets)
59             val_loss += loss.item() * inputs.size(0)
60             val_samples += inputs.size(0)
61
62     avg_val_loss = val_loss / val_samples
63     val_ppl = math.exp(avg_val_loss)
64
65     pbar_epochs.set_postfix({
66         "tr_loss": f"{avg_train_loss:.3f}",
67         "val_loss": f"{avg_val_loss:.3f}",
68         "val_PPL": f"{val_ppl:.1f}"
69     })
70
71     # --- ЛОГИКА РАННЕЙ ОСТАНОВКИ И СОХРАНЕНИЯ ---
72     if avg_val_loss < best_val_loss:
73         best_val_loss = avg_val_loss
74         counter = 0
75         torch.save(model.state_dict(), save_path)
76     else:

```

```

77         counter += 1
78         if counter >= PATIENCE:
79             print(f"\nРанняя остановка на эпохе {epoch + 1}. Лучший Val Loss:
              ↪ {best_val_loss:.4f}")
80             break
81
82     dataset_info = {
83         'vocab': train_dataset.vocab,
84         'rev_vocab': train_dataset.rev_vocab,
85         'vocab_size': len(train_dataset.vocab)
86     }
87     with open('../././models/fnnlm/dataset_info.pkl', 'wb') as f:
88         pickle.dump(dataset_info, f)
89
90     if os.path.exists(save_path):
91         model.load_state_dict(torch.load(save_path))
92
93     return model, train_dataset
94
95
96 def generate_title(model, input_text, vocab, rev_vocab, max_words=12,
97 ↪ temperature=0.7, top_k=5):
98     device = next(model.parameters()).device
99     model.eval()
100     clean_input = re.sub(r'[^\\w\\s]', '', input_text.lower()).split()
101     if len(clean_input) > CONTEXT_SIZE:
102         current_context = clean_input[-CONTEXT_SIZE:]
103     else:
104         current_context = ['<PAD>'] * (CONTEXT_SIZE - len(clean_input)) +
105 ↪ clean_input
106
107     result_title = []
108
109     for _ in range(max_words):
110         input_ids = torch.tensor([[vocab.get(w, vocab['<UNK>']) for w in
111 ↪ current_context]]).to(device)
112
113         with torch.no_grad():
114             logits = model(input_ids)
115
116             logits /= temperature
117
118             top_logits, top_indices = torch.topk(logits, top_k)
119
120             probs = F.softmax(top_logits, dim=-1)
121
122             idx_in_top = torch.multinomial(probs, num_samples=1).item()
123             pred_id = top_indices[0, idx_in_top].item()
124
125             pred_word = rev_vocab[pred_id]
126
127     if pred_word == '<END>':

```

```

126         break
127
128     if pred_word not in ['<PAD>', '<UNK>']:
129         result_title.append(pred_word)
130
131     current_context = current_context[1:] + [pred_word]
132
133     return " ".join(result_title)

1 import torch
2
3 CONTEXT_SIZE = 8
4 EMBEDDING_DIM = 64
5 HIDDEN_DIM = 128
6 BATCH_SIZE = 32
7 EPOCHS = 30
8 MIN_WORD_FREQ = 25
9
10 LEARNING_RATE = 0.0001
11 DROPOUT = 0.3
12 WEIGHT_DECAY = 1e-5
13
14 TRAIN_DATASET = "../././data/training_data/train_df.csv"
15 VAL_DATASET = "../././data/training_data/val_df.csv"
16
17 LIMIT = 100000
18
19 PATIENCE = 3
20 COUNTER = 0
21 BEST_VAL_LOSS = float('inf')
22 save_path = "../././models/fnnlm/best_fnnlm_model.pth"
23
24 if torch.backends.mps.is_available():
25     DEVICE = torch.device("mps")
26 elif torch.cuda.is_available():
27     DEVICE = torch.device("cuda")
28 else:
29     DEVICE = torch.device("cpu")
30
31 print(f"Используемое устройство: {DEVICE}")

1 from my_models.history.FNNLM.model.utils import train_model, generate_title
2
3 model, dataset = train_model()
4
5 print("\n--- Тестирование генерации ---")
6 sample_text = "в данной работе мы исследуем методы машинного обучения для  

↪ классификации"
7 predicted = generate_title(model, sample_text, dataset.vocab, dataset.rev_vocab)
8
9 print(f"Входной текст: {sample_text}")
10 print(f"Сгенерированное название: {predicted}")

```

```

1 import torch
2 import re
3 import pickle
4 from my_models.history.FNNLM.model.model import FNNLM
5 from my_models.history.FNNLM.config import *
6 from my_models.history.FNNLM.model.utils import generate_title
7
8
9 def load_model_with_dataset():
10     """Загрузка модели с информацией о датасете"""
11
12     try:
13         dataset_path = "../..../models/fnnlm/dataset_info.pkl"
14         with open(dataset_path, 'rb') as f:
15             dataset_info = pickle.load(f)
16
17         vocab = dataset_info['vocab']
18         rev_vocab = dataset_info['rev_vocab']
19         vocab_size = dataset_info['vocab_size']
20
21         print(f"Информация о датасете загружена!")
22         print(f"Размер словаря: {vocab_size}")
23
24         model = FNNLM(vocab_size=vocab_size,
25             ↪ embedding_dim=EMBEDDING_DIM,
26             ↪ hidden_dim=HIDDEN_DIM).to(DEVICE)
27
28         model.load_state_dict(torch.load(save_path, map_location=DEVICE))
29         model.eval()
30
31         print("Модель успешно загружена!")
32         return model, vocab, rev_vocab
33
34     except FileNotFoundError:
35         print("Файл с информацией о датасете не найден!")
36         print("Убедитесь, что вы сохранили dataset_info.pkl при обучении")
37         return None, None, None
38     except Exception as e:
39         print(f"Ошибка загрузки: {str(e)}")
40         return None, None, None
41
42
43 def interactive_generator():
44     model, vocab, rev_vocab = load_model_with_dataset()
45     if model is None: return
46
47     print("\nГотово! Введите текст. Для выхода — 'quit'.")
48
49     while True:
50         user_text = input(">>> ").strip()
51         if user_text.lower() == 'quit': break
52         if len(user_text) < 5: continue

```



```

52
53     try:
54         title = generate_title(model, user_text, vocab, rev_vocab)
55         if title.strip() == "":
56             print(f"Модель ничего не сгенерировала\n")
57             continue
58         print(f"Заголовок: {title}\n")
59     except Exception as e:
60         print(f"Ошибка при генерации: {e}")
61
62
63 if __name__ == "__main__":
64     interactive_generator()

```

ПРИЛОЖЕНИЕ Д

Рекуррентные нейронные сети

```
1 import torch
2 from torch.utils.data import Dataset
3 from torch.nn.utils.rnn import pad_sequence
4 import re
5 from collections import Counter
6 import my_models.history.RNN.config as config
7
8
9 class Vocab:
10     def __init__(self, texts, min_freq=5):
11         self.itos = [config.PAD_TOKEN, config.SOS_TOKEN, config.EOS_TOKEN,
12             ↪ config.UNK_TOKEN]
13         self.stoi = {token: i for i, token in enumerate(self.itos)}
14
15         tokens = []
16         for text in texts:
17             tokens.extend(self.tokenize(str(text)))
18
19         counter = Counter(tokens)
20         for token, freq in counter.items():
21             if freq >= min_freq and token not in self.stoi:
22                 self.stoi[token] = len(self.itos)
23                 self.itos.append(token)
24
25     @staticmethod
26     def tokenize(text):
27         return re.findall(r'\w+', text.lower())
28
29     def encode(self, text):
30         return [self.stoi.get(token, self.stoi[config.UNK_TOKEN]) for token in
31             ↪ self.tokenize(str(text))]
32
33     def __len__(self):
34         return len(self.itos)
35
36 class TitleDataset(Dataset):
37     def __init__(self, df, vocab):
38         self.text = df['text'].astype(str).values
39         self.titles = df['title'].astype(str).values
40         self.vocab = vocab
41
42     def __len__(self):
43         return len(self.text)
44
45     def __getitem__(self, idx):
46         text = self.text[idx]
47         title = self.titles[idx]
```

```

48     lec_tensor = torch.tensor(self.vocab.encode(text), dtype=torch.long)
49
50     title_encoded = self.vocab.encode(title)
51     title_tensor = torch.tensor(
52         [self.vocab.stoi[config.SOS_TOKEN]] +
53         title_encoded +
54         [self.vocab.stoi[config.EOS_TOKEN]],
55         dtype=torch.long
56     )
57
58     return lec_tensor, title_tensor
59
60
61 def collate_fn(batch, pad_idx):
62     lecs, titles = zip(*batch)
63     lecs_padded = pad_sequence(lecs, batch_first=True, padding_value=pad_idx)
64     titles_padded = pad_sequence(titles, batch_first=True, padding_value=pad_idx)
65     return lecs_padded, titles_padded

```



```

1  import torch
2  import torch.nn as nn
3
4
5  class TitleRNN(nn.Module):
6      def __init__(self, vocab_size, emb_dim, hid_dim, n_layers, dropout):
7          super().__init__()
8          self.embedding = nn.Embedding(vocab_size, emb_dim)
9          self.rnn = nn.RNN(emb_dim, hid_dim, n_layers, dropout=dropout,
10                           ↪ batch_first=True)
11          self.fc_out = nn.Linear(hid_dim, vocab_size)
12          self.dropout = nn.Dropout(dropout)
13
14      def forward(self, text, title):
15          embedded_text = self.dropout(self.embedding(text))
16          _, h = self.rnn(embedded_text)
17
18          embedded_title = self.dropout(self.embedding(title))
19
20          output, _ = self.rnn(embedded_title, h)
21
22          return self.fc_out(output)

```



```

1  import torch
2  import my_models.history.RNN.config as config
3
4
5  def train_epoch(model, loader, optimizer, criterion, device):
6      model.train()
7      epoch_loss = 0
8
9      for text, titles in loader:
10         text, titles = text.to(device), titles.to(device)
11

```

```

12     optimizer.zero_grad()
13     output = model(text, titles[:, :-1])
14
15     output_dim = output.shape[-1]
16     loss = criterion(output.reshape(-1, output_dim), titles[:, 1:].reshape(-1))
17
18     loss.backward()
19     optimizer.step()
20     epoch_loss += loss.item()
21
22     return epoch_loss / len(loader)
23
24
25 def evaluate(model, loader, criterion, device):
26     model.eval()
27     epoch_loss = 0
28     with torch.no_grad():
29         for text, titles in loader:
30             text, titles = text.to(device), titles.to(device)
31             output = model(text, titles[:, :-1])
32             output_dim = output.shape[-1]
33             loss = criterion(output.reshape(-1, output_dim), titles[:, 1:].reshape(-1))
34             epoch_loss += loss.item()
35     return epoch_loss / len(loader)
36
37
38 def generate_title(model, text, vocab, device, max_len=15):
39     model.eval()
40     with torch.no_grad():
41         tokens = vocab.encode(text)
42         if not tokens: return "..."
43
44         src_tensor = torch.LongTensor(tokens).unsqueeze(0).to(device)
45
46         _, h = model.rnn(model.embedding(src_tensor))
47
48         curr_idx = vocab.stoi[config.SOS_TOKEN]
49         result = []
50
51         for _ in range(max_len):
52             input_tensor = torch.LongTensor([curr_idx]).to(device)
53             output, h = model.rnn(model.embedding(input_tensor), h)
54
55             next_word_idx = output.argmax(2).item()
56             if next_word_idx == vocab.stoi[config.EOS_TOKEN]:
57                 break
58
59             if next_word_idx < len(vocab.itos):
60                 result.append(vocab.itos[next_word_idx])
61
62             curr_idx = next_word_idx
63
64     return " ".join(result)

```

```

1  import torch
2
3  TRAIN_DATA_PATH = ".././../data/training_data/train_df.csv"
4  VAL_DATA_PATH = ".././../data/training_data/val_df.csv"
5  MODEL_SAVE_PATH = ".././../models/rnn/rnn_model.pt"
6
7  LIMIT = 100_000
8
9  if torch.backends.mps.is_available():
10     DEVICE = torch.device("mps")
11  elif torch.cuda.is_available():
12     DEVICE = torch.device("cuda")
13  else:
14     DEVICE = torch.device("cpu")
15
16  EMBED_DIM = 128
17  HID_DIM = 256
18  N_LAYERS = 2
19  DROPOUT = 0.3
20  LR = 1e-3
21  BATCH_SIZE = 16
22  EPOCHS = 15
23  PATIENT = 2
24
25  PAD_TOKEN = "<pad>"
26  SOS_TOKEN = "<sos>"
27  EOS_TOKEN = "<eos>"
28  UNK_TOKEN = "<unk>"

1  import pandas as pd
2  import torch
3  import torch.nn as nn
4  from torch.utils.data import DataLoader
5  from tqdm import tqdm
6  from functools import partial
7
8  from data.dataset import TitleDataset, Vocab, collate_fn
9  from model.model import TitleRNN
10 from model.utils import train_epoch, evaluate
11 import config
12 import os
13 os.environ['PYTORCH_MPS_HIGH_WATERMARK_RATIO'] = '0.0'
14
15 train_df = pd.read_csv(config.TRAIN_DATA_PATH).head(config.LIMIT)
16 val_df = pd.read_csv(config.VAL_DATA_PATH).head(config.LIMIT)
17
18 all_text = (train_df['text'].astype(str).tolist() +
19            train_df['title'].astype(str).tolist() +
20            val_df['text'].astype(str).tolist() +
21            val_df['title'].astype(str).tolist())
22 vocab = Vocab(all_text)
23
24 train_dataset = TitleDataset(train_df, vocab)

```

```

25 val_dataset = TitleDataset(val_df, vocab)
26
27 pad_idx = vocab.stoi[config.PAD_TOKEN]
28 collate_p = partial(collate_fn, pad_idx=pad_idx)
29
30 train_loader = DataLoader(train_dataset, batch_size=config.BATCH_SIZE,
31                           shuffle=True, collate_fn=collate_p)
32 val_loader = DataLoader(val_dataset, batch_size=config.BATCH_SIZE,
33                         shuffle=False, collate_fn=collate_p)
34
35 model = TitleRNN(
36     vocab_size=len(vocab.itos),
37     emb_dim=config.EMBED_DIM,
38     hid_dim=config.HID_DIM,
39     n_layers=config.N_LAYERS,
40     dropout=config.DROPOUT
41 ).to(config.DEVICE)
42
43 optimizer = torch.optim.Adam(model.parameters(), lr=config.LR)
44 criterion = nn.CrossEntropyLoss(ignore_index=pad_idx)
45 print(f"Размер словаря: {len(vocab)} | Train образцов: {len(train_dataset)} | Val
    ↪ образцов: {len(val_dataset)}")
46 best_loss = float('inf')
47 epoch_no_improve = 0
48 epoch_range = tqdm(range(config.EPOCHS), desc="Training progress")
49 for epoch in epoch_range:
50     train_loss = train_epoch(model, train_loader, optimizer, criterion, config.DEVICE)
51
52     val_loss = evaluate(model, val_loader, criterion, config.DEVICE)
53
54     epoch_range.set_postfix({
55         'train_loss': f'{train_loss:.4f}',
56         'val_loss': f'{val_loss:.4f}'
57     })
58
59     if torch.backends.mps.is_available():
60         torch.mps.empty_cache()
61
62     if val_loss < best_loss:
63         best_loss = val_loss
64         epoch_no_improve = 0
65         torch.save({'model_state': model.state_dict(), 'vocab': vocab},
    ↪ config.MODEL_SAVE_PATH)
66     else:
67         epoch_no_improve += 1
68     if epoch_no_improve == config.PATIENT:
69         print(f"Ранняя остановка на {epoch + 1} эпохе | Best Loss: {best_loss:.4f}")
70         break

1 import torch
2 import config
3 from model.model import TitleRNN
4 from model.utils import generate_title

```

```

5
6
7 def run_inference():
8     try:
9         checkpoint = torch.load(config.MODEL_SAVE_PATH,
10             ↪ map_location=config.DEVICE, weights_only=False)
11
12         vocab = checkpoint['vocab']
13         model_state = checkpoint['model_state']
14
15         model = TitleRNN(
16             vocab_size=len(vocab.itos),
17             emb_dim=config.EMBED_DIM,
18             hid_dim=config.HID_DIM,
19             n_layers=config.N_LAYERS,
20             dropout=config.DROPOUT
21         ).to(config.DEVICE)
22
23         model.load_state_dict(model_state)
24         model.eval()
25
26         print(f"--- Модель загружена (Vocab size: {len(vocab.itos)}) ---")
27     except FileNotFoundError:
28         print(f"Файл {config.MODEL_SAVE_PATH} не найден!")
29         return
30     except Exception as e:
31         print(f"Ошибка при загрузке: {e}")
32         return
33
34     print("\nВведите текст лекции для генерации заголовка (выход: 'quit')")
35     while True:
36         try:
37             text = input("\nТекст >>> ").strip()
38
39             if text.lower() in ['quit', 'exit', 'выход']:
40                 break
41
42             if not text:
43                 continue
44
45             res = generate_title(model, text, vocab, config.DEVICE)
46
47             print(f"Заголовок: {res}")
48         except KeyboardInterrupt:
49             break
50         except Exception as e:
51             print(f"Ошибка: {e}")
52
53
54 if __name__ == "__main__":
55     run_inference()

```

ПРИЛОЖЕНИЕ Е

LSTM

```
1 import torch
2 from torch.utils.data import Dataset
3 from collections import Counter
4
5
6 class TitleVocab:
7     def __init__(self, df, min_freq=10):
8         self.itos = {0: "<PAD>", 1: "<SOS>", 2: "<EOS>", 3: "<UNK>", 4: "<SEP>"}
9         self.stoi = {v: k for k, v in self.itos.items()}
10
11         words = []
12         for col in ['text', 'title']:
13             for text in df[col].dropna():
14                 words.extend(str(text).lower().split())
15
16         counts = Counter(words)
17         for word, freq in counts.items():
18             if freq >= min_freq and word not in self.stoi:
19                 idx = len(self.itos)
20                 self.stoi[word] = idx
21                 self.itos[idx] = word
22
23     def __len__(self):
24         return len(self.itos)
25
26     def encode(self, text):
27         return [self.stoi.get(w, self.stoi["<UNK>"]) for w in str(text).lower().split()]
28
29     def decode(self, indices):
30         return " ".join([self.itos.get(i.item() if torch.is_tensor(i) else i, "<UNK>")
31                           for i in indices if i > 4])
32
33
34 class LSTMTitleDataset(Dataset):
35     def __init__(self, df, vocab, max_len=100):
36         self.vocab = vocab
37         self.max_len = max_len
38         self.samples = []
39
40         for _, row in df.dropna(subset=['text', 'title']).iterrows():
41             text_tokens = self.vocab.encode(str(row['text']))[:80]
42             title_tokens = self.vocab.encode(row['title'])
43
44             full_seq = [vocab.stoi["<SOS>"]] + text_tokens + [vocab.stoi["<SEP>"]] +
45                 ↪ title_tokens + [
46                     vocab.stoi["<EOS>"]]
47
48             if len(full_seq) > 5:
49                 self.samples.append(full_seq)
```



```

49
50     def __len__(self):
51         return len(self.samples)
52
53     def __getitem__(self, idx):
54         tokens = self.samples[idx]
55         sep_idx = tokens.index(self.vocab.stoi["<SEP>"])
56
57         input_seq = tokens[:1]
58         target_seq = tokens[1:]
59
60         mask = [0] * len(input_seq)
61         for i in range(sep_idx, len(mask)):
62             mask[i] = 1
63
64         input_seq = input_seq[:self.max_len] + [0] * max(0, self.max_len -
        ↪ len(input_seq))
65         target_seq = target_seq[:self.max_len] + [0] * max(0, self.max_len -
        ↪ len(target_seq))
66         mask = mask[:self.max_len] + [0] * max(0, self.max_len - len(mask))
67
68         return torch.tensor(input_seq), torch.tensor(target_seq), torch.tensor(mask,
        ↪ dtype=torch.float)

1  import torch.nn as nn
2
3
4  class LSTMTITLEGen(nn.Module):
5      def __init__(self, vocab_size, embed_dim, hidden_dim, num_layers, dropout):
6          super().__init__()
7          self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
8
9          self.lstm = nn.LSTM(
10             embed_dim,
11             hidden_dim,
12             num_layers=num_layers,
13             batch_first=True,
14             dropout=dropout if num_layers > 1 else 0,
15             bidirectional=True
16         )
17
18         self.fc = nn.Linear(hidden_dim * 2, vocab_size)
19
20         self.dropout = nn.Dropout(dropout)
21         self.ln = nn.LayerNorm(hidden_dim * 2)
22
23     def forward(self, x, hidden=None):
24         x = self.embedding(x)
25
26         out, hidden = self.lstm(x, hidden)
27
28         out = self.ln(out)
29         out = self.dropout(out)

```

```

30
31     logits = self.fc(out)
32     return logits, hidden

1  import torch
2
3
4  def generate_title(model, vocab, device, text, max_gen_len=20):
5      model.eval()
6      tokens = vocab.encode(text)
7      input_ids = [vocab.stoi["<SOS>"]] + tokens + [vocab.stoi["<SEP>"]]
8      input_tensor = torch.tensor([input_ids]).to(device)
9
10     generated = []
11
12     with torch.no_grad():
13         for _ in range(max_gen_len):
14             logits, _ = model(input_tensor)
15
16             last_token_logits = logits[0, -1, :]
17
18             probs = torch.softmax(last_token_logits / 0.4, dim=-1)
19
20             next_token = torch.multinomial(probs, num_samples=1).item()
21
22             if next_token == vocab.stoi["<EOS>"]:
23                 break
24
25             generated.append(next_token)
26
27             next_tensor = torch.tensor([[next_token]]).to(device)
28             input_tensor = torch.cat([input_tensor, next_tensor], dim=1)
29
30     return vocab.decode(generated)

1  import torch
2
3  TRAIN_DATA_PATH = ".././../data/training_data/train_df.csv"
4  VAL_DATA_PATH = ".././../data/training_data/val_df.csv"
5  MODEL_SAVE_PATH = ".././../models/lstm/lstm_model.pt"
6
7  EMBED_DIM = 64
8  HIDDEN_DIM = 128
9  NUM_LAYERS = 2
10 DROPOUT = 0.3
11 BATCH_SIZE = 32
12 LR = 1e-3
13 EPOCHS = 20
14 PATIENCE = 3
15 MAX_LEN = 150
16
17 LIMIT = 100000
18

```

```

19 if torch.backends.mps.is_available():
20     DEVICE = torch.device("mps")
21 elif torch.cuda.is_available():
22     DEVICE = torch.device("cuda")
23 else:
24     DEVICE = torch.device("cpu")

1 import math
2
3 import pandas as pd
4 import torch
5 import torch.nn as nn
6 from torch.utils.data import DataLoader
7 from tqdm import tqdm
8
9 from data.dataset import TitleVocab, LSTMTitleDataset
10 from model.model import LSTMTitleGen
11 import config
12
13 train_full = pd.read_csv(config.TRAIN_DATA_PATH)
14 train_full = train_full.sample(frac=1, random_state=42).reset_index(drop=True)
15 train_df = train_full.head(config.LIMIT).dropna(subset=['text', 'title'])
16
17 val_full = pd.read_csv(config.VAL_DATA_PATH)
18 val_full = val_full.sample(frac=1, random_state=42).reset_index(drop=True)
19 val_df = val_full.head(config.LIMIT).dropna(subset=['text', 'title'])
20
21 vocab = TitleVocab(train_df)
22
23 train_dataset = LSTMTitleDataset(train_df, vocab, config.MAX_LEN)
24 val_dataset = LSTMTitleDataset(val_df, vocab, config.MAX_LEN)
25
26 train_loader = DataLoader(train_dataset, batch_size=config.BATCH_SIZE,
    ↪ shuffle=True)
27 val_loader = DataLoader(val_dataset, batch_size=config.BATCH_SIZE,
    ↪ shuffle=False)
28
29 model = LSTMTitleGen(
30     vocab_size=len(vocab),
31     embed_dim=config.EMBED_DIM,
32     hidden_dim=config.HIDDEN_DIM,
33     num_layers=config.NUM_LAYERS,
34     dropout=config.DROPOUT
35 ).to(config.DEVICE)
36
37 criterion = nn.CrossEntropyLoss(ignore_index=0, reduction='none')
38 optimizer = torch.optim.Adam(model.parameters(), lr=config.LR)
39
40 epoch_no_improve = 0
41 best_loss = float('inf')
42
43 print(f"Размер словаря: {len(vocab)} | Train образцов: {len(train_dataset)} | Val
    ↪ образцов: {len(val_dataset)}")

```

```

44
45 pbar = tqdm(range(config.EPOCHS), desc="Training Progress")
46 for epoch in pbar:
47     model.train()
48     train_loss = 0
49     for inputs, targets, masks in train_loader:
50         inputs, targets, masks = inputs.to(config.DEVICE), targets.to(config.DEVICE),
           ↪ masks.to(config.DEVICE)
51         optimizer.zero_grad()
52         logits, _ = model(inputs)
53         raw_loss = criterion(logits.view(-1, len(vocab)), targets.view(-1))
54
55         masked_loss = raw_loss * masks.view(-1)
56
57         loss = masked_loss.sum() / (masks.sum() + 1e-9)
58         loss.backward()
59         optimizer.step()
60         train_loss += loss.item()
61
62     avg_train_loss = train_loss / len(train_loader)
63
64     model.eval()
65     val_loss = 0
66     with torch.no_grad():
67         for inputs, targets, masks in val_loader:
68             inputs, targets, masks = inputs.to(config.DEVICE),
           ↪ targets.to(config.DEVICE), masks.to(config.DEVICE)
69
70             logits, _ = model(inputs)
71
72             raw_loss = criterion(logits.view(-1, len(vocab)), targets.view(-1))
73             masked_loss = raw_loss * masks.view(-1)
74             loss = masked_loss.sum() / (masks.sum() + 1e-9)
75
76             val_loss += loss.item()
77
78     avg_val_loss = val_loss / len(val_loader)
79     val_ppl = math.exp(avg_val_loss) if avg_val_loss < 20 else float('inf')
80
81     pbar.set_postfix({
82         'train loss': f"{avg_train_loss:.4f}",
83         'val loss': f"{avg_val_loss:.4f}",
84         'val ppl': f"{val_ppl:.2f}"
85     })
86
87     if avg_val_loss < best_loss:
88         best_loss = avg_val_loss
89         epoch_no_improve = 0
90         torch.save({'model_state': model.state_dict(), 'vocab': vocab},
           ↪ config.MODEL_SAVE_PATH)
91     else:
92         epoch_no_improve += 1
93

```

```

94     if epoch_no_improve >= config.PATIENCE:
95         print(f"Ранняя остановка на {epoch + 1} эпохе | Best Loss: {best_loss:.4f}")
96         break

1  import torch
2  import config
3  from model.model import LSTMTitleGen
4  from model.utils import generate_title
5
6
7  def load_model_and_vocab():
8      try:
9          checkpoint = torch.load(config.MODEL_SAVE_PATH,
10             ↪ map_location=config.DEVICE, weights_only=False)
11          vocab = checkpoint['vocab']
12
13          print(f"Размер словаря: {len(vocab)}")
14
15          model = LSTMTitleGen(
16              vocab_size=len(vocab),
17              embed_dim=config.EMBED_DIM,
18              hidden_dim=config.HIDDEN_DIM,
19              num_layers=config.NUM_LAYERS,
20              dropout=config.DROPOUT
21          ).to(config.DEVICE)
22
23          model.load_state_dict(checkpoint['model_state'])
24          model.eval()
25
26          return model, vocab
27      except FileNotFoundError:
28          print(f"Файл {config.MODEL_SAVE_PATH} не найден!")
29          return None, None
30      except Exception as e:
31          print(f"Ошибка при загрузке: {e}")
32          return None, None
33
34
35  model, vocab = load_model_and_vocab()
36
37  print("\nГотово! Введите текст. Для выхода — 'quit'.")
38
39  while True:
40      try:
41          user_text = input("\nВведите текст >>> ").strip()
42
43          if user_text.lower() in ['quit', 'exit', 'выход']:
44              print("Выход...")
45              break
46
47          if len(user_text) < 5:
48              print("Слишком короткий текст, попробуйте еще раз.")

```

```
49         continue
50
51     title = generate_title(model, vocab, config.DEVICE, user_text)
52
53     if not title or title.strip() == "":
54         print("Модель ничего не сгенерировала.")
55     else:
56         print(f"Заголовок: {title}")
57
58 except KeyboardInterrupt:
59     break
60 except Exception as e:
61     print(f"Ошибка генерации: {e}")
```

ПРИЛОЖЕНИЕ Ж

Seq2seq

```
1 import torch
2 from torch.utils.data import Dataset
3 from torch.nn.utils.rnn import pad_sequence
4
5 from my_models.Seq2seq.data.vocab import simple_tokenize
6
7
8 class TitleDataset(Dataset):
9     def __init__(self, df, vocab, max_src_len=300, max_trg_len=30):
10         self.df = df
11         self.vocab = vocab
12         self.max_src_len = max_src_len
13         self.max_trg_len = max_trg_len
14
15     def __len__(self):
16         return len(self.df)
17
18     def text_to_indices(self, text, max_len, add_sos_eos=True):
19         tokens = simple_tokenize(str(text))
20
21         if len(tokens) > max_len:
22             tokens = tokens[:max_len]
23
24         indices = [self.vocab.word2index.get(t, 1) for t in tokens]
25
26         if add_sos_eos:
27             indices = [2] + indices + [3]
28
29         return torch.tensor(indices, dtype=torch.long)
30
31     def __getitem__(self, idx):
32         row = self.df.iloc[idx]
33
34         src_tensor = self.text_to_indices(row['text'], self.max_src_len,
35             ↪ add_sos_eos=False)
36         trg_tensor = self.text_to_indices(row['title'], self.max_trg_len,
37             ↪ add_sos_eos=True)
38
39         return src_tensor, trg_tensor
40
41     def collate_fn(batch):
42         """Функция для дополнения батча паддингами"""
43         src_batch, trg_batch = zip(*batch)
44
45         src_padded = pad_sequence(src_batch, padding_value=0, batch_first=True)
46         trg_padded = pad_sequence(trg_batch, padding_value=0, batch_first=True)
47
48         return src_padded, trg_padded
```

```

1 import re
2 from collections import Counter
3 import pickle
4
5 import tiktoken
6
7 import my_models.Seq2seq.config as config
8
9 encoding = tiktoken.get_encoding("cl100k_base")
10 def simple_tokenize(text):
11     """Разбивает текст на слова и знаки препинания, сохраняя регистр или в
        ↪ lower"""
12     tokens_ids = encoding.encode(text.lower())
13     return [encoding.decode([t]) for t in tokens_ids]
14
15 class Vocab:
16     def __init__(self):
17         self.word2index = {"<pad>": 0, "<unk>": 1, "<sos>": 2, "<eos>": 3}
18         self.index2word = {0: "<pad>", 1: "<unk>", 2: "<sos>", 3: "<eos>"}
19         self.n_words = 4
20
21     def build_vocab(self, texts, titles):
22         counter = Counter()
23         for text in texts:
24             counter.update(simple_tokenize(str(text)))
25         for title in titles:
26             counter.update(simple_tokenize(str(title)))
27
28         for word, count in counter.most_common(config.MAX_VOCAB_SIZE - 4):
29             if count >= config.MIN_COUNT:
30                 if word not in self.word2index:
31                     self.word2index[word] = self.n_words
32                     self.index2word[self.n_words] = word
33                     self.n_words += 1
34
35     def save(self, path):
36         with open(path, 'wb') as f:
37             pickle.dump(self, f)
38
39     @staticmethod
40     def load(path):
41         with open(path, 'rb') as f:
42             return pickle.load(f)

```



```

1 import torch
2 from torch import nn
3 import torch.nn.functional as F
4
5
6 class Attention(nn.Module):
7     def __init__(self, hidden_dim: int):
8         super().__init__()
9         self.attn = nn.Linear(hidden_dim * 3, hidden_dim)

```



```

10         self.v = nn.Linear(hidden_dim, 1, bias=False)
11
12     def forward(self, hidden, encoder_outputs):
13         src_len = encoder_outputs.shape[1]
14
15         hidden_repeated = hidden.permute(1, 0, 2).repeat(1, src_len, 1)
16
17         combined = torch.cat((hidden_repeated, encoder_outputs), dim=2)
18
19         energy = torch.tanh(self.attn(combined))
20         attention_weights = self.v(energy).squeeze(2)
21
22         return F.softmax(attention_weights, dim=1)

```



```

1  import torch
2  from torch import nn
3
4  from my_models.Seq2seq.model.attention import Attention
5
6
7  class Decoder(nn.Module):
8      def __init__(self, output_dim: int, emb_dim: int, hidden_dim: int, dropout: float):
9          super().__init__()
10          self.output_dim = output_dim
11          self.attention = Attention(hidden_dim)
12
13          self.embedding = nn.Embedding(output_dim, emb_dim)
14
15          self.lstm = nn.LSTM(emb_dim + (hidden_dim * 2), hidden_dim,
16                               ↪ batch_first=True)
17
18          self.fc_out = nn.Linear(hidden_dim * 3 + emb_dim, output_dim)
19          self.dropout = nn.Dropout(dropout)
20
21     def forward(self, input_tok, hidden, cell, encoder_outputs):
22         input_tok = input_tok.unsqueeze(1)
23         embedded = self.dropout(self.embedding(input_tok))
24
25         a = self.attention(hidden, encoder_outputs).unsqueeze(1)
26
27         context = torch.bmm(a, encoder_outputs)
28         rnn_input = torch.cat((embedded, context), dim=2)
29         output, (hidden, cell) = self.lstm(rnn_input, (hidden, cell))
30
31         prediction = self.fc_out(torch.cat((output, context, embedded),
32                                             ↪ dim=2).squeeze(1))
33
34         return prediction, hidden, cell

```



```

1  import torch
2  import torch.nn as nn
3
4

```

```

5 class Encoder(nn.Module):
6     def __init__(self, input_dim: int, emb_dim: int, hidden_dim: int, dropout: float =
      ↪ 0.3):
7         super().__init__()
8         self.embedding = nn.Embedding(input_dim, emb_dim)
9         self.lstm = nn.LSTM(emb_dim, hidden_dim, batch_first=True,
      ↪ bidirectional=True)
10
11         self.fc_hidden = nn.Linear(hidden_dim * 2, hidden_dim)
12         self.fc_cell = nn.Linear(hidden_dim * 2, hidden_dim)
13
14         self.dropout = nn.Dropout(dropout)
15
16     def forward(self, src):
17         embedded = self.dropout(self.embedding(src))
18
19         outputs, (hidden, cell) = self.lstm(embedded)
20
21         hidden = torch.tanh(self.fc_hidden(torch.cat((hidden[-2, :, :], hidden[-1, :, :]),
      ↪ dim=1)))
22         cell = torch.tanh(self.fc_cell(torch.cat((cell[-2, :, :], cell[-1, :, :]), dim=1)))
23
24         return outputs, hidden.unsqueeze(0), cell.unsqueeze(0)

```



```

1 import random
2
3 import torch
4 from torch import nn
5
6
7 class Seq2Seq(nn.Module):
8     def __init__(self, encoder, decoder, device):
9         super().__init__()
10        self.encoder = encoder
11        self.decoder = decoder
12        self.device = device
13
14    def forward(self, src, trg, teacher_forcing_ratio: float = 0.5):
15        batch_size = src.shape[0]
16        trg_len = trg.shape[1]
17        vocab_size = self.decoder.output_dim
18
19        outputs = torch.zeros(batch_size, trg_len, vocab_size).to(self.device)
20
21        encoder_outputs, hidden, cell = self.encoder(src)
22
23        input_tok = trg[:, 0]
24
25        for t in range(1, trg_len):
26            output, hidden, cell = self.decoder(input_tok, hidden, cell, encoder_outputs)
27            outputs[:, t] = output
28
29            teacher_force = random.random() < teacher_forcing_ratio

```

```

30         top1 = output.argmax(1)
31         input_tok = trg[:, t] if teacher_force else top1
32
33     return outputs

1  import os
2  import random
3
4  import numpy as np
5  from tqdm import tqdm
6
7  from my_models.Seq2seq.config import *
8  from my_models.Seq2seq.data.vocab import Vocab, simple_tokenize
9  from my_models.Seq2seq.model.decoder import Decoder
10 from my_models.Seq2seq.model.encoder import Encoder
11 from my_models.Seq2seq.model.seq2seq import Seq2Seq
12
13
14 def set_seed(seed: int = 42):
15     random.seed(seed)
16     np.random.seed(seed)
17     torch.manual_seed(seed)
18     torch.cuda.manual_seed_all(seed)
19
20
21 def train_epoch(model, dataloader, optimizer, criterion, clip):
22     model.train()
23     epoch_loss = 0.0
24
25     pbar = tqdm(dataloader, desc=" Batch", leave=False)
26
27     for src, trg in pbar:
28         src, trg = src.to(model.device), trg.to(model.device)
29         optimizer.zero_grad()
30
31         outputs = model(src, trg)
32
33         out_dim = outputs.shape[-1]
34
35         outputs = outputs[:, 1:].reshape(-1, out_dim)
36         trg_flat = trg[:, 1:].reshape(-1)
37
38         loss = criterion(outputs, trg_flat)
39         loss.backward()
40
41         torch.nn.utils.clip_grad_norm_(model.parameters(), clip)
42         optimizer.step()
43
44         epoch_loss += loss.item()
45
46     pbar.set_postfix({'loss': f'{loss.item():.4f}'})
47
48     return epoch_loss / len(dataloader)

```

```

49
50
51 def evaluate(model, dataloader, criterion):
52     model.eval()
53     epoch_loss = 0.0
54
55     with torch.no_grad():
56         for src, trg in dataloader:
57             src, trg = src.to(model.device), trg.to(model.device)
58
59             outputs = model(src, trg, teacher_forcing_ratio=0.0)
60
61             out_dim = outputs.shape[-1]
62             outputs = outputs[:, 1:].reshape(-1, out_dim)
63             trg_flat = trg[:, 1:].reshape(-1)
64
65             loss = criterion(outputs, trg_flat)
66             epoch_loss += loss.item()
67
68     return epoch_loss / len(dataloader)
69
70
71 def generate_response(model, vocab, text, beam_width=3,
72     ↪ max_len=MAX_TRG_LEN):
73     model.eval()
74
75     sos_idx = vocab.word2index["<sos>"]
76     eos_idx = vocab.word2index["<eos>"]
77
78     with torch.no_grad():
79         tokens = simple_tokenize(str(text))
80         src_indices = [vocab.word2index.get(t, 1) for t in tokens[:MAX_SRC_LEN]]
81         src_tensor = torch.tensor([src_indices], dtype=torch.long).to(device)
82
83         encoder_outputs, hidden, cell = model.encoder(src_tensor)
84
85         beams = [(0.0, [sos_idx], hidden, cell)]
86
87     for _ in range(max_len):
88         new_beams = []
89
90         for score, tokens, h, c in beams:
91             if tokens[-1] == eos_idx:
92                 new_beams.append((score, tokens, h, c))
93                 continue
94
95             input_tok = torch.tensor([tokens[-1]]).to(device)
96
97             output, new_h, new_c = model.decoder(input_tok, h, c, encoder_outputs)
98
99             log_probs = torch.log_softmax(output, dim=1)
100             top_v, top_i = torch.topk(log_probs, beam_width)

```

```

101         for i in range(beam_width):
102             next_score = score + top_v[0, i].item()
103             next_tokens = tokens + [top_i[0, i].item()]
104             new_beams.append((next_score, next_tokens, new_h, new_c))
105
106         beams = sorted(new_beams, key=lambda x: x[0], reverse=True)[:beam_width]
107
108         if all(b[1][-1] == eos_idx for b in beams):
109             break
110
111     best_tokens = beams[0][1]
112
113     result_words = []
114     for idx in best_tokens:
115         if idx == eos_idx:
116             break
117         if idx != sos_idx:
118             result_words.append(vocab.index2word.get(idx, "<unk>"))
119
120     return "".join(result_words)
121
122
123 def load_model(vocab_path=VOCAB_PATH, model=MODEL_SAVE_PATH):
124     """Загружает обученную модель и словарь"""
125     if not os.path.exists(vocab_path):
126         raise FileNotFoundError(f"Файл словаря не найден: {vocab_path}")
127     vocab = Vocab.load(vocab_path)
128
129     if not os.path.exists(model):
130         raise FileNotFoundError(f"Файл модели не найден: {model}")
131
132     checkpoint = torch.load(model, map_location=device)
133
134     if isinstance(checkpoint, dict) and 'model_state_dict' in checkpoint:
135         model_vocab_size = checkpoint.get('vocab_size',
136             ↪ checkpoint['model_state_dict']['encoder.embedding.weight'].shape[0])
137         model_state = checkpoint['model_state_dict']
138         meta_info = f"epoch={checkpoint.get('epoch', '?')},
139             ↪ val_loss={checkpoint.get('val_loss', 0):.4f}"
140     else:
141         model_vocab_size = checkpoint['encoder.embedding.weight'].shape[0]
142         model_state = checkpoint
143         meta_info = "старый формат"
144
145     if model_vocab_size != vocab.n_words:
146         print(f"ВНИМАНИЕ: несоответствие размеров словаря.")
147         print(f" В файле vocab.pkl: {vocab.n_words}")
148         print(f" В checkpoint модели: {model_vocab_size}")
149         print(f" Используем {model_vocab_size} для инициализации модели.")
150         model_vocab_size = model_vocab_size
151     else:
152         print(f"Размер словаря совпадает: {model_vocab_size}")

```

```

152     encoder = Encoder(model_vocab_size, ENC_EMB_DIM, HID_DIM,
    ↪     ENC_DROPOUT)
153     decoder = Decoder(model_vocab_size, DEC_EMB_DIM, HID_DIM,
    ↪     DEC_DROPOUT)
154     model = Seq2Seq(encoder, decoder, device).to(device)
155
156     model.load_state_dict(model_state)
157     model.eval()
158     print(f"Модель загружена успешно! ({meta_info})")
159
160     return model, vocab

1 from __future__ import annotations
2
3 import torch
4
5 TRAIN_DATA_PATH = "../data/training_data/train_df.csv"
6 VAL_DATA_PATH = "../data/training_data/val_df_chunked.csv"
7 MODEL_SAVE_PATH = "../models/seq2seq/best_model_seq2seq.pt"
8 VOCAB_PATH = "../models/seq2seq/vocab.pkl"
9
10 MAX_VOCAB_SIZE = 100_000
11 MIN_COUNT = 1
12 LIMIT = 200_000
13 N_SAMPLES = 10
14
15 MAX_SRC_LEN = 300
16 MAX_TRG_LEN = 30
17 BATCH_SIZE = 128
18 NUM_EPOCHS = 30
19 CLIP = 1.0
20
21 ENC_EMB_DIM = 256
22 DEC_EMB_DIM = 256
23 HID_DIM = 256
24 ENC_DROPOUT = 0.5
25 DEC_DROPOUT = 0.5
26 LR = 5e-4
27 PATIENCE = 4
28
29 SEED = 42
30
31 device = torch.device(
32     "cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available()
    ↪     else "cpu")
33 print(f"Используемое устройство: {device}")

1 import time
2 import torch
3 import torch.nn as nn
4 from torch import optim
5 from torch.utils.data import DataLoader
6 import pandas as pd

```

```

7  from tqdm import tqdm
8
9  import config
10 from data.vocab import Vocab
11 from data.dataset import TitleDataset, collate_fn
12 from model.encoder import Encoder
13 from model.decoder import Decoder
14 from model.seq2seq import Seq2Seq
15 from my_models.Seq2seq.model.utils import train_epoch, evaluate
16
17 # import os
18 # os.environ['PYTORCH_MPS_HIGH_WATERMARK_RATIO'] = '0.0'
19
20 full_train = pd.read_csv(config.TRAIN_DATA_PATH)
21 vocab = Vocab()
22 vocab.build_vocab(full_train['text'].values, full_train['title'].values)
23 vocab.save(config.VOCAB_PATH)
24
25 train_df = pd.read_csv(config.TRAIN_DATA_PATH)
26
27 train_df = train_df.groupby('title').head(config.N_SAMPLES)
28
29 train_df = train_df.sample(frac=1,
    ↪ random_state=config.SEED).reset_index(drop=True)
30
31 train_df = train_df.head(config.LIMIT)
32 val_df = pd.read_csv(config.VAL_DATA_PATH).sample(frac=1,
    ↪ random_state=config.SEED).head(int(len(train_df) * 0.4))
33 print(f"Количество объектов в train: {len(train_df)}, val: {len(val_df)}")
34
35 train_dataset = TitleDataset(train_df, vocab, config.MAX_SRC_LEN,
    ↪ config.MAX_TRG_LEN)
36 val_dataset = TitleDataset(val_df, vocab, config.MAX_SRC_LEN,
    ↪ config.MAX_TRG_LEN)
37
38 del train_df
39 del val_df
40 del full_train
41
42 import gc
43 gc.collect()
44
45 train_loader = DataLoader(train_dataset, batch_size=config.BATCH_SIZE,
46                             shuffle=True, collate_fn=collate_fn)
47 val_loader = DataLoader(val_dataset, batch_size=config.BATCH_SIZE,
48                             shuffle=False, collate_fn=collate_fn)
49
50 encoder = Encoder(vocab.n_words, config.ENC_EMB_DIM, config.HID_DIM,
    ↪ config.ENC_DROPOUT)
51 decoder = Decoder(vocab.n_words, config.DEC_EMB_DIM, config.HID_DIM,
    ↪ config.DEC_DROPOUT)
52 model = Seq2Seq(encoder, decoder, config.device).to(config.device)
53

```

```

54 PAD_IDX = vocab.word2index["<pad>"]
55 criterion = nn.CrossEntropyLoss(ignore_index=PAD_IDX, label_smoothing=0.0)
56 optimizer = optim.AdamW(model.parameters(), lr=config.LR)
57 scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=3,
    ↪ factor=0.5)
58
59 best_val_loss = float('inf')
60 history = []
61
62 epochs_range = range(1, config.NUM_EPOCHS + 1)
63 # pbar = tqdm(epochs_range, desc="Training", unit="epoch")
64 epochs_no_improve = 0
65 if torch.backends.mps.is_available():
66     torch.mps.empty_cache()
67
68 for epoch in epochs_range:
69     start_time = time.time()
70
71     train_loss = train_epoch(model, train_loader, optimizer, criterion, config.CLIP)
72     val_loss = evaluate(model, val_loader, criterion)
73
74     scheduler.step(val_loss)
75
76     history.append({
77         'epoch': epoch,
78         'train_loss': train_loss,
79         'val_loss': val_loss,
80         'lr': optimizer.param_groups[0]['lr']
81     })
82     checkpoint = {
83         'model_state_dict': model.state_dict(),
84         'vocab_size': vocab.n_words,
85         'epoch': epoch,
86         'val_loss': val_loss,
87         'train_loss': train_loss,
88     }
89     if val_loss < best_val_loss:
90         best_val_loss = val_loss
91         torch.save(checkpoint, config.MODEL_SAVE_PATH)
92         print(f"\nЛучшая модель сохранена (vocab_size={vocab.n_words},
    ↪ epoch={epoch:02d}, val_loss={val_loss:.4f})")
93     else:
94         epochs_no_improve += 1
95
96     torch.save(checkpoint, config.MODEL_SAVE_PATH.replace(".pt",
    ↪ "_overfitting.pt"))
97
98     print(f"epoch: {epoch:02d}, train_loss: {train_loss:.4f}, val_loss: {val_loss:.4f}, lr:
    ↪ {optimizer.param_groups[0]['lr']:.2e}, time: {time.time() - start_time:.2f}")
99
100 if epochs_no_improve >= config.PATIENCE:
101     print(f"Ранняя остановка на {epoch:02} эпохе | Best val loss:
    ↪ {best_val_loss:.4f}")

```



```

102         break
103
104 history_path = config.MODEL_SAVE_PATH.replace('.pt', '_history.csv')
105 pd.DataFrame(history).to_csv(history_path, index=False)
106 print(f"\nОбучение завершено. История сохранена в {history_path}")

1 import sys
2 import os
3
4 from my_models.Seq2seq.data.vocab import simple_tokenize
5 from my_models.Seq2seq.model.utils import load_model, generate_response
6
7 sys.path.append(os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__)
    ↪   _file_))))))
8
9 model, vocab =
    ↪   load_model(model=" ../models/seq2seq/best_model_seq2seq_finetuned_2.pt")
10
11 print("\nВведите текст для генерации заголовка.")
12 print("Для выхода введите: exit или quit")
13
14 while True:
15     try:
16         user_input = input("\n>>> Введите текст: ").strip()
17
18         if user_input.lower() in ['exit', 'quit', 'выход']:
19             break
20
21         if not user_input:
22             print("Пожалуйста, введите текст.")
23             continue
24
25         tokens = simple_tokenize(user_input)
26         debug_info = []
27
28         response = generate_response(model, vocab, user_input)
29
30         if response:
31             print(f"\nОтвет: {response}")
32         else:
33             print(f"\nОтвет: [не удалось сгенерировать]")
34
35     except KeyboardInterrupt:
36         break
37     except Exception as e:
38         print(f"\nОшибка: {e}")

```